

```

/* =====
   Man'ouché RH – Application logic
   Vanilla JS · Firebase Realtime DB · PWA-ready
   ===== */

// ----- CONFIG -----
const FB_CONFIG = {
  apiKey: "AIzaSyDPQz5sLRsSp9kaqG8JLyOW_FeqszkcCUg",
  authDomain: "manouche75003.firebaseio.com",
  databaseURL: "https://manouche75003-default-rtdb.europe-west1.firebaseio.com",
  projectId: "manouche75003",
  storageBucket: "manouche75003.firebaseio.com",
  messagingSenderId: "620239670518",
  appId: "1:620239670518:web:ec6249a4b8522506b5f0d7"
};

const ADMIN_USER = 'rayess';
const ADMIN_CODE = '541194J';

// Default seed data (matches existing RH_ULTIME.html structure)
const DEFAULT_EMPLOYEES = [
  {id:1,prenom:"Johnny",nom:"Salemneh",poste:"Cuisinier",contrat:"CDI",heures:35,
  {id:3,prenom:"Ahmad",nom:"Yaggi",poste:"Cuisinier",contrat:"CDI",heures:35,taux
  {id:4,prenom:"Jeremie",nom:"Dacosta",poste:"Cuisinier",contrat:"CDI",heures:35,
  {id:5,prenom:"Oussama",nom:"Bouaziz",poste:"Cuisinier",contrat:"CDI",heures:35,
  {id:6,prenom:"Dababo",nom:"Yahya",poste:"Cuisinier",contrat:"CDI",heures:35,tau
  {id:7,prenom:"Omar",nom:"Kozbari",poste:"Cuisinier",contrat:"CDI",heures:35,tau
];

const DAYS = ['Lundi','Mardi','Mercredi','Jeudi','Vendredi','Samedi','Dimanche'];
const DAYS_SHORT = ['Lun','Mar','Mer','Jeu','Ven','Sam','Dim'];
const MONTHS_FR = ['janvier','février','mars','avril','mai','juin','juillet','aoû

// Leave types (mutually exclusive with regular shift)
const LEAVE_TYPES = {
  cp: { label: 'Congé payé', short: 'CP', color:
  absent_justifie: { label: 'Absent justifié', short: 'Abs. just.', color:
  absent_injustifie: { label: 'Absent injustifié', short: 'Abs. inj.', color:
  arret_maladie: { label: 'Arrêt maladie', short: 'AM', color:
  rtt: { label: 'RTT', short: 'RTT', color:
  recup: { label: 'Récupération', short: 'Récup', color:
};

// Normalize a shift – converts legacy {absent:true} / {conge:true} to new leaveT

```

```

function normShift(s) {
  if (!s) return s;
  if (s.leaveType) return s;
  if (s.absent) return { ...s, leaveType: 'absent_justifie' };
  if (s.conge) return { ...s, leaveType: 'cp' };
  return s;
}

function isLeave(s) { return !(s && (s.leaveType || s.absent || s.conge)); }

// ----- JOURS FÉRIÉS (France) -----
// Calcul de Pâques – algorithme de Butcher/Meeus
function easterDate(year) {
  const a = year % 19;
  const b = Math.floor(year / 100);
  const c = year % 100;
  const d = Math.floor(b / 4);
  const e = b % 4;
  const f = Math.floor((b + 8) / 25);
  const g = Math.floor((b - f + 1) / 3);
  const h = (19 * a + b - d - g + 15) % 30;
  const i = Math.floor(c / 4);
  const k = c % 4;
  const l = (32 + 2 * e + 2 * i - h - k) % 7;
  const m = Math.floor((a + 11 * h + 22 * l) / 451);
  const month = Math.floor((h + l - 7 * m + 114) / 31);
  const day = ((h + l - 7 * m + 114) % 31) + 1;
  return new Date(year, month - 1, day);
}

function holidaysForYear(year) {
  const e = easterDate(year);
  const easterMon = new Date(e); easterMon.setDate(e.getDate() + 1);
  const ascension = new Date(e); ascension.setDate(e.getDate() + 39);
  const pentecost = new Date(e); pentecost.setDate(e.getDate() + 50);
  return {
    [`${year}-01-01`]: { short: "Jour de l'An", long: "Jour de l'An" },
    [dateISO(easterMon)]: { short: 'Lundi de Pâques', long: 'Lundi de Pâques' },
    [`${year}-05-01`]: { short: 'Fête du Travail', long: 'Fête du Travail' },
    [`${year}-05-08`]: { short: 'Victoire 1945', long: 'Victoire 1945' },
    [dateISO(ascension)]: { short: 'Ascension', long: "Jeudi de l'Ascension" },
    [dateISO(pentecost)]: { short: 'L. Pentecôte', long: 'Lundi de Pentecôte' },
    [`${year}-07-14`]: { short: '14 Juillet', long: 'Fête nationale' },
    [`${year}-08-15`]: { short: 'Assomption', long: 'Assomption' },
    [`${year}-11-01`]: { short: 'Toussaint', long: 'Toussaint' },
    [`${year}-11-11`]: { short: '11 Novembre', long: 'Armistice 1918' },
    [`${year}-12-25`]: { short: 'Noël', long: 'Noël' },
  };
}

```

```

}
const _holidayCache = {};
function holidayFor(date) {
  const y = date.getFullYear();
  if (!_holidayCache[y]) _holidayCache[y] = holidaysForYear(y);
  return _holidayCache[y][dateISO(date)] || null;
}

// ————— VACANCES SCOLAIRES (Zone C – Paris) —————
// Source : éducation nationale, dates officielles
const SCHOOL_HOLIDAYS_ZONE_C = [
  { name: 'Toussaint', start: '2025-10-18', end: '2025-11-02' },
  { name: 'Noël', start: '2025-12-20', end: '2026-01-04' },
  { name: 'Hiver', start: '2026-02-14', end: '2026-03-01' },
  { name: 'Printemps', start: '2026-04-11', end: '2026-04-26' },
  { name: 'Été', start: '2026-07-04', end: '2026-08-31' },
  { name: 'Toussaint', start: '2026-10-17', end: '2026-11-02' },
  { name: 'Noël', start: '2026-12-19', end: '2027-01-03' },
  { name: 'Hiver', start: '2027-02-06', end: '2027-02-21' },
  { name: 'Printemps', start: '2027-04-03', end: '2027-04-18' },
  { name: 'Été', start: '2027-07-03', end: '2027-08-31' },
];

function schoolHolidayFor(date) {
  const iso = dateISO(date);
  for (const h of SCHOOL_HOLIDAYS_ZONE_C) {
    if (iso >= h.start && iso <= h.end) return h;
  }
  return null;
}

function schoolHolidaysOverlappingWeek(weekStart) {
  const startISO = dateISO(weekStart);
  const endISO = dateISO(addDays(weekStart, 6));
  return SCHOOL_HOLIDAYS_ZONE_C.filter(h =>
    !(h.end < startISO || h.start > endISO)
  );
}

// ————— BROUILLONS / PUBLICATIONS —————
function weekIsPublished(wk) {
  if (state.publications[wk]) return true;
  // Les semaines déjà passées (de plus de 2 jours) sont visibles
  // automatiquement – pratique pour les données historiques importées
  const wkDate = new Date(wk);
  const today = new Date();
  today.setHours(0,0,0,0);
  const wkEnd = new Date(wkDate); wkEnd.setDate(wkDate.getDate() + 6);
  return wkEnd < today; // semaine complètement passée
}

```

```

}

// ----- EXPORTS -----
function csvEsc(s) {
  s = String(s ?? '');
  if (s.includes(';') || s.includes('"') || s.includes('\n') || s.includes(','))
    return '"' + s.replace(/"/g, '""') + '"';
  }
  return s;
}

function exportPlanningCSV() {
  const wk = weekKey(state.weekStart);
  const lines = ['Salaarié;Poste;Date;Jour;Statut;Type;Début;Fin;Pause (mn);Heures
  const actives = state.employees.filter(e => e.statut === 'Actif');

  actives.forEach(emp => {
    for (let d = 0; d < 7; d++) {
      const date = addDays(state.weekStart, d);
      const fer = holidayFor(date);
      const ferStr = fer ? fer.long : '';
      const shifts = (state.shifts[`_${emp.id}_${d}_${wk}`] || []).map(normShift);
      if (shifts.length === 0) {
        lines.push([
          csvEsc(emp.prenom + ' ' + emp.nom),
          csvEsc(emp.poste || ''),
          dateISO(date),
          DAYS[d],
          'Repos',
          '', '', '', '0', '0', '',
          csvEsc(ferStr)
        ].join(';'));
      } else {
        shifts.forEach(s => {
          if (s.leaveType) {
            const lt = LEAVE_TYPES[s.leaveType];
            lines.push([
              csvEsc(emp.prenom + ' ' + emp.nom),
              csvEsc(emp.poste || ''),
              dateISO(date),
              DAYS[d],
              csvEsc(lt?.label || s.leaveType),
              '', '', '', '0', '0',
              csvEsc(s.label || ''),
              csvEsc(ferStr)
            ].join(';'));
          } else {

```

```

        lines.push([
            csvEsc(emp.prenom + ' ' + emp.nom),
            csvEsc(emp.poste || ''),
            dateISO(date),
            DAYS[d],
            'Travaille',
            s.type || '',
            s.start || '',
            s.end || '',
            String(s.pauseDuration || 0),
            shiftHours(s).toFixed(2).replace('.', ','),
            csvEsc(s.label || ''),
            csvEsc(ferStr)
        ].join(';'));
    }
    });
}

// Section totaux
lines.push('');
lines.push('TOTAUX SEMAINE');
lines.push('Salarié;Contrat;Heures planifiées;Écart contrat');
actives.forEach(emp => {
    let h = 0;
    for (let d = 0; d < 7; d++) {
        (state.shifts[`${emp.id}_${d}_${wk}`] || []).forEach(s => h += shiftHours(s)
        )
    }
    const gap = h - (emp.heures || 35);
    lines.push([
        csvEsc(emp.prenom + ' ' + emp.nom),
        (emp.heures || 35) + 'h',
        h.toFixed(2).replace('.', ','),
        (gap >= 0 ? '+' : '-') + gap.toFixed(2).replace('.', ','),
    ].join(';'));
});

const csv = lines.join('\r\n');
const blob = new Blob(['\u00eff' + csv], { type: 'text/csv;charset=utf-8' });
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = `planning_${wk}.csv`;
document.body.appendChild(a);
a.click();
document.body.removeChild(a);

```

```

    setTimeout(() => URL.revokeObjectURL(url), 1000);
    toast('CSV téléchargé', 'good');
}

function exportPlanningPDF() {
    document.body.classList.add('printing');
    // Brief delay to let the class apply
    setTimeout(() => {
        window.print();
        setTimeout(() => document.body.classList.remove('printing'), 500);
    }, 100);
}

// Seed planning (template based on the RH_ULTIME schedule) – used by "Charger un
const SEED_SHIFTS = {
    // Ahmad Yaggi (3)
    '3_0': [{type: 'ar', start: '11:00', end: '23:00', label: 'Patron', pauseDuration: 120}],
    '3_1': [{type: 'ar', start: '11:00', end: '23:00', label: 'Patron', pauseDuration: 120}],
    '3_2': [{type: 'ar', start: '11:00', end: '23:00', label: 'Patron', pauseDuration: 120}],
    '3_5': [{type: 'soir', start: '18:00', end: '23:00'}],
    // Jeremie (4)
    '4_0': [{type: 'soir', start: '17:00', end: '23:00'}],
    '4_4': [{type: 'ar', start: '11:00', end: '23:59', pauseDuration: 120}],
    '4_5': [{type: 'ar', start: '12:00', end: '23:59', pauseDuration: 120}],
    '4_6': [{type: 'soir', start: '15:00', end: '23:00'}],
    // Oussama (5)
    '5_3': [{type: 'ar', start: '11:00', end: '23:00', pauseDuration: 120}],
    '5_4': [{type: 'ar', start: '12:00', end: '23:59', pauseDuration: 120}],
    '5_5': [{type: 'midi', start: '12:00', end: '18:00'}],
    '5_6': [{type: 'ar', start: '12:00', end: '23:00', pauseDuration: 120}],
    // Yahya / Dababo (6)
    '6_3': [{type: 'ar', start: '12:00', end: '23:00', pauseDuration: 120}],
    '6_4': [{type: 'soir', start: '17:00', end: '23:00'}],
    '6_5': [{type: 'ar', start: '11:00', end: '23:59', pauseDuration: 120}],
    '6_6': [{type: 'ar', start: '12:00', end: '23:00', pauseDuration: 120}],
    // Omar (7)
    '7_0': [{type: 'midi', start: '12:00', end: '17:00'}],
    '7_1': [{type: 'ar', start: '12:00', end: '23:00', pauseDuration: 120}],
    '7_2': [{type: 'ar', start: '12:00', end: '23:00', pauseDuration: 120}]
};

// _____ STATE _____
const state = {
    user: null,          // null | {role: 'admin'} | {role: 'emp', empId, ...}
    employees: [],
    shifts: {},          // key `${empId}_${dayIdx}_${weekKey}` → [{type, start, end, ...}
    punches: {},         // key `${empId}_${dateISO}` → [{in: '08:32', out: '14:15', meta

```

```

publications: {}, // key weekKey → {publishedAt, by, note}
absenceRequests: {}, // keyed by request id
emargements: {}, // key `${empId}_${weekKey}` → {signedAt, userAgent}
pourboires: {}, // key `${empId}_${yyyy-mm}` → amount in € (monthly)
duerp: null, // {updatedAt, docId, dueAt}
affichages: {}, // key affichage name → {present, photoDocId, updatedAt}
payslips: {}, // key `${empId}_${yyyy-mm}` → {docId, sentAt, ackBy?}
adminCredentials: null, // {username, passwordHash}
weekStart: getMonday(new Date()),
fbReady: false,
page: null, // current page id
loading: true,
monthView: false,
monthAnchor: new Date(),
empDetail: null, // current employee id being viewed in detail (null = list)
empTab: 'info', // active tab in employee detail
rhTab: 'entries', // active tab in Suivi RH page
reqTab: 'pending', // active tab in absence requests
hoursView: 'cards', // 'cards' | 'table'
};

// ————— EMPLOYEE NORMALIZATION —————
function normEmp(e) {
  if (!e) return null;
  const defaults = {
    nomNaissance: '', genre: '', dateNaissance: '', paysNaissance: '',
    departementNaissance: '', communeNaissance: '', nationalite: '',
    email: '', telMobile: '', telFixe: '', notifSMS: false,
    adresse: '', complementAdresse: '', codePostal: '', ville: '', pays: 'France'
    contratDebut: '', contratFin: '', periodeEssaiJours: 60,
    remuneration: 0, joursTravailles: 5, navigoMensuel: 0,
    dateSortie: '', motifSortie: '',
    cpAcquisN: e.cpAcquis ?? 25,
    cpPrisN: e.cpPris ?? 0,
    cpAcquisNml: 0, cpPrisNml: 0,
    reposCompensateur: 0, recupJoursFeries: 0,
    travailleurEtranger: false, titreSejourType: '',
    titreSejourNumero: '', titreSejourDebut: '', titreSejourFin: '',
    // NOUVEAU – Conformité
    periodeEssaiRenouvelable: false,
    periodeEssaiRenouveleeAt: '', // date du renouvellement si fait
    contratSigneAt: '', // date de signature du contrat
    autorisationTravailNumero: '', // numéro autorisation prefecture (étrangers)
    autorisationTravailFin: '', // date d'expiration
    navigoDernierRenouvellement: '', // date du dernier achat/renouvellement annue
    visiteMedicaleAt: '', // date de la dernière visite médicale
    visiteMedicaleType: '', // 'embauche' | 'periodique' | 'reprise'
  };

```

```

    visiteMedicaleProchaine: '', // date prévue prochaine visite
    mutuelleAdhesionAt: '',
    mutuelleDispensee: false,
    mutuelleDispenseRaison: '',
    dpaeFaiteAt: '', // date DPAE
    certifHACCP: false, // formation hygiène
    certifHACCPDate: '',
    permisB: false, permisBFin: '',
    peutSeConnecter: true,
    // NOUVEAU – Modalités de paie spécifiques
    exclureCP: false, // pas de gestion de congés payés (extra, etc.)
    sansHeuresSupp: false, // toutes les heures au taux normal
    payeNet: false, // le taux saisi est un net (pas un brut)
    exclurePaie: false, // exclu du récap paie (gérant non salarié, stagia
    forfaitJours: false, // forfait jours – pas de décompte horaire
    sansAvantageRepas: false, // pas d'avantage en nature repas
    sansNavigo: false, // pas de remboursement transport
    dispos: { 0: true, 1: true, 2: true, 3: true, 4: true, 5: true, 6: true },
    avenants: [], documents: [],
  };
  return { ...defaults, ...e };
}

function isProfileComplete(e) {
  const n = normEmp(e);
  return !! (n.email && n.telMobile && n.dateNaissance && n.adresse && n.codePosta
}

// ————— RAPPELS / ÉCHÉANCES —————
function daysBetween(fromISO, toISO) {
  const a = new Date(fromISO + 'T00:00:00');
  const b = new Date(toISO + 'T00:00:00');
  return Math.round((b - a) / (1000 * 60 * 60 * 24));
}

function computeRappels() {
  const out = [];
  const todayISO = dateISO(new Date());
  const today = new Date(todayISO + 'T00:00:00');

  state.employees.forEach(e => {
    if (e.statut !== 'Actif') return;
    const n = normEmp(e);
    const empLabel = `${e.prenom} ${e.nom}`;

    // 1. Contrat signé ?
    if (n.contratDebut && !n.contratSigneAt) {

```



```

const ds = daysBetween(todayISO, n.contratDebut);
if (ds <= 7) {
  out.push({
    type: 'contrat_signe',
    empId: e.id, empLabel,
    label: 'Contrat à faire signer',
    detail: `Début de contrat ${n.contratDebut ? fmtDateShort(n.contratDebut) : ''}`,
    dueDate: n.contratDebut, daysLeft: ds,
    severity: ds < 0 ? 'alert' : (ds < 3 ? 'warn' : 'info'),
    action: 'Marquer le contrat comme signé dans la fiche salarié'
  });
}
}

// 2. Fin de période d'essai
if (n.contratDebut && n.périodeEssaiJours > 0 && !n.dateSortie) {
  const fin = new Date(n.contratDebut + 'T00:00:00');
  fin.setDate(fin.getDate() + n.périodeEssaiJours);
  const finISO = dateISO(fin);
  const ds = daysBetween(todayISO, finISO);
  if (ds <= 14 && ds >= -7) {
    out.push({
      type: 'fin_essai',
      empId: e.id, empLabel,
      label: ds < 0 ? 'Période d\'essai terminée' : 'Fin de période d\'essai',
      detail: `${n.périodeEssaiJours} jours · échéance ${fmtDateShort(finISO)}`,
      dueDate: finISO, daysLeft: ds,
      severity: ds < 0 ? 'info' : (ds <= 3 ? 'alert' : (ds <= 7 ? 'warn' : 'info')),
      action: ds > 0 ? 'Décider : valider, rompre, ou renouveler avant cette date' : ''
    });
  }
}

// Renouvellement à anticiper (avant le terme initial)
if (n.périodeEssaiRenouvelable && !n.périodeEssaiRenouveleeAt && ds >= 0 && !n.dateSortie) {
  out.push({
    type: 'renouvellement_essai',
    empId: e.id, empLabel,
    label: 'Renouvellement période d\'essai possible',
    detail: `À décider avant ${fmtDateShort(finISO)} (envoi LRAR + signature)`,
    dueDate: finISO, daysLeft: ds,
    severity: ds <= 2 ? 'alert' : 'warn',
    action: 'Préparer la LRAR de renouvellement et la faire signer'
  });
}
}

// 3. Fin de CDD
if (n.contratFin && !n.dateSortie) {

```

```

const ds = daysBetween(todayISO, n.contratFin);
if (ds <= 60 && ds >= -30) {
  out.push({
    type: 'fin_cdd',
    empId: e.id, empLabel,
    label: ds < 0 ? 'Contrat échu' : 'Fin de CDD',
    detail: `${esc(e.contrat||'CDD')}` - terme ${fmtDateShort(n.contratFin)}
    dueDate: n.contratFin, daysLeft: ds,
    severity: ds < 0 ? 'alert' : (ds <= 7 ? 'alert' : (ds <= 30 ? 'warn' :
    action: ds > 0 ? 'Préparer STC, certificat travail, attestation France
  });
}
}

// 4. Titre de séjour
if (n.travailleurEtranger && n.titreSejourFin) {
  const ds = daysBetween(todayISO, n.titreSejourFin);
  if (ds <= 120) {
    out.push({
      type: 'titre_sejour',
      empId: e.id, empLabel,
      label: ds < 0 ? 'Titre de séjour expiré' : 'Titre de séjour à renouvele
      detail: `${esc(n.titreSejourType||'Titre')}` n° ${esc(n.titreSejourNumer
      dueDate: n.titreSejourFin, daysLeft: ds,
      severity: ds < 0 ? 'alert' : (ds <= 30 ? 'alert' : (ds <= 60 ? 'warn' :
      action: ds < 0 ? 'URGENT : suspendre le salarié et l\'orienter en préfe
    });
  }
}

// 5. Autorisation de travail
if (n.travailleurEtranger && n.autorisationTravailFin) {
  const ds = daysBetween(todayISO, n.autorisationTravailFin);
  if (ds <= 90) {
    out.push({
      type: 'autorisation_travail',
      empId: e.id, empLabel,
      label: ds < 0 ? 'Autorisation de travail expirée' : 'Autorisation de tr
      detail: `N° ${esc(n.autorisationTravailNumero||'-')}` · expire ${fmtDate
      dueDate: n.autorisationTravailFin, daysLeft: ds,
      severity: ds < 0 ? 'alert' : (ds <= 15 ? 'alert' : (ds <= 45 ? 'warn' :
      action: ds < 0 ? 'URGENT : ne plus faire travailler le salarié, lourde
    });
  }
}

// 6. Pass Navigo annuel (Imagine R / Navigo Annuel renouvellement)

```

```

if (n.navigoDernierRenouvellement) {
    const lastDate = new Date(n.navigoDernierRenouvellement + 'T00:00:00');
    const nextDate = new Date(lastDate); nextDate.setFullYear(nextDate.getFully
    const nextISO = dateISO(nextDate);
    const ds = daysBetween(todayISO, nextISO);
    if (ds <= 45) {
        out.push({
            type: 'navigo',
            empId: e.id, empLabel,
            label: ds < 0 ? 'Navigo annuel expiré' : 'Renouvellement Navigo annuel'
            detail: `Dernier renouvellement ${fmtDateShort(n.navigoDernierRenouvell
            dueDate: nextISO, daysLeft: ds,
            severity: ds < 0 ? 'warn' : (ds <= 15 ? 'warn' : 'info'),
            action: 'Refaire la prise en charge employeur (remboursement 50% obliga
        });
    }
} else if (n.navigoMensuel > 0 && n.contratDebut) {
    // Pas de date de renouvellement enregistrée mais le salarié a un navigo →
    out.push({
        type: 'navigo',
        empId: e.id, empLabel,
        label: 'Navigo : date à renseigner',
        detail: `${n.navigoMensuel.toFixed(2)} €/mois remboursés – pas de date d'
        dueDate: todayISO, daysLeft: 0,
        severity: 'info',
        action: 'Renseigner la date du dernier renouvellement dans la fiche'
    });
}

// 7. Visite médicale du travail (à faire dans les 3 mois après embauche, pui
if (n.contratDebut && !n.visiteMedicaleAt && !n.dateSortie) {
    const startDate = new Date(n.contratDebut + 'T00:00:00');
    const deadline = new Date(startDate); deadline.setMonth(deadline.getMonth()
    const ds = daysBetween(todayISO, dateISO(deadline));
    if (ds <= 60) {
        out.push({
            type: 'visite_medicale',
            empId: e.id, empLabel,
            label: ds < 0 ? 'Visite médicale d\'embauche en retard' : 'Visite médic
            detail: `À organiser sous 3 mois après embauche · échéance ${fmtDateSho
            dueDate: dateISO(deadline), daysLeft: ds,
            severity: ds < 0 ? 'alert' : (ds <= 15 ? 'warn' : 'info'),
            action: 'Programmer un RDV avec la médecine du travail (Centre Médical
        });
    }
}
}
if (n.visiteMedicaleProchaine && !n.dateSortie) {

```

```

const ds = daysBetween(todayISO, n.visiteMedicaleProchaine);
if (ds <= 30) {
  out.push({
    type: 'visite_medicale_periodique',
    empId: e.id, empLabel,
    label: ds < 0 ? 'Visite médicale périodique en retard' : 'Visite médica
    detail: `Prochaine visite ${fmtDateShort(n.visiteMedicaleProchaine)}`,
    dueDate: n.visiteMedicaleProchaine, daysLeft: ds,
    severity: ds < 0 ? 'alert' : (ds <= 7 ? 'warn' : 'info'),
    action: 'Programmer un RDV avec la médecine du travail'
  });
}
}

// 8. HACCP / Formation hygiène (obligatoire en restauration commerciale)
if (!n.certifHACCP && n.contratDebut && !n.dateSortie) {
  // Au moins 1 personne du staff doit être HACCP – alerte légère
  // On laisse au manager de vérifier
}

// 9. DPAE
if (n.contratDebut && !n.dpaeFaiteAt) {
  const ds = daysBetween(todayISO, n.contratDebut);
  if (ds >= -7 && ds <= 1) {
    out.push({
      type: 'dpae',
      empId: e.id, empLabel,
      label: 'DPAE à confirmer',
      detail: `Aucune trace de DPAE enregistrée pour ce salarié`,
      dueDate: n.contratDebut, daysLeft: ds,
      severity: ds <= 0 ? 'alert' : 'warn',
      action: 'Vérifier la DPAE sur net-entreprises.fr ou marquer comme faite
    });
  }
}

// 10. Entretien professionnel obligatoire tous les 2 ans
if (n.contratDebut && !n.dateSortie) {
  const refDate = n.dernierEntretienPro || n.contratDebut;
  const next = new Date(refDate + 'T00:00:00');
  next.setFullYear(next.getFullYear() + 2);
  const nextISO = dateISO(next);
  const ds = daysBetween(todayISO, nextISO);
  if (ds <= 90) {
    out.push({
      type: 'entretien_pro',
      empId: e.id, empLabel,

```

```

        label: ds < 0 ? 'Entretien pro EN RETARD' : 'Entretien professionnel à
        detail: `Échéance ${fmtDateShort(nextISO)} · obligatoire tous les 2 ans
        dueDate: nextISO, daysLeft: ds,
        severity: ds < 0 ? 'alert' : (ds <= 30 ? 'warn' : 'info'),
        action: ds < 0 ? 'URGENT : pénalité 3000 € abondement CPF si défaut' :
    });
}
}

// 11. Visite médicale de reprise après arrêt maladie ≥ 30 jours
// Détection : on regarde les arrêts maladie consécutifs de l'employé
if (n.contratDebut && !n.dateSortie) {
    const amDays = [];
    Object.entries(state.shifts).forEach(([key, shifts]) => {
        if (!key.startsWith(`${e.id}_`)) return;
        const parts = key.split('_');
        if (parts.length !== 3) return;
        const dayIdx = parseInt(parts[1]);
        const wk = parts[2];
        if (!wk || isNaN(dayIdx)) return;
        (shifts||[]).forEach(s => {
            if (normShift(s).leaveType === 'arret_maladie') {
                const d = addDays(new Date(wk + 'T00:00:00'), dayIdx);
                amDays.push(dateISO(d));
            }
        });
    });
    amDays.sort();
    // Find longest consecutive streak
    let maxStreak = 0, curStreak = 0, streakEnd = null;
    for (let i = 0; i < amDays.length; i++) {
        if (i === 0 || daysBetween(amDays[i-1], amDays[i]) === 1) curStreak++;
        else curStreak = 1;
        if (curStreak > maxStreak) { maxStreak = curStreak; streakEnd = amDays[i]
    }
    if (maxStreak >= 30 && streakEnd) {
        const reprise = addDays(new Date(streakEnd + 'T00:00:00'), 1);
        const ds = daysBetween(todayISO, dateISO(reprise));
        // Visit doit avoir lieu dans les 8 jours suivant la reprise
        const deadline = addDays(reprise, 8);
        const dsDeadline = daysBetween(todayISO, dateISO(deadline));
        // Trigger only if visit not done after the streak
        const visitOK = n.visiteMedicaleAt && n.visiteMedicaleAt > streakEnd && n
        if (!visitOK && dsDeadline <= 30 && dsDeadline >= -60) {
            out.push({
                type: 'visite_reprise',
                empId: e.id, empLabel,

```

```

        label: dsDeadline < 0 ? 'Visite de reprise EN RETARD' : 'Visite médic
        detail: `Arrêt maladie de ${maxStreak} jours · échéance visite ${fmtD
        dueDate: dateISO(deadline), daysLeft: dsDeadline,
        severity: dsDeadline < 0 ? 'alert' : 'warn',
        action: 'Programmer RDV médecine du travail dans les 8 jours suivant
    });
  }
}
}
});

// Sort by urgency: alert first, then by daysLeft
out.sort((a, b) => {
  const sev = { alert: 0, warn: 1, info: 2 };
  if (sev[a.severity] !== sev[b.severity]) return sev[a.severity] - sev[b.sever
  return a.daysLeft - b.daysLeft;
});
return out;
}

const RAPPEL_ICONS = {
  contrat_signe: '📝',
  fin_essai: '🕒',
  renouvellement_essai: '🔄',
  fin_cdd: '📅',
  titre_sejour: '🏠',
  autorisation_travail: '👤',
  navigo: '🚆',
  visite_medicale: '⌘',
  visite_medicale_periodique: '⌘',
  visite_reprise: '⌘',
  dpae: '📄',
  entretien_pro: '🤝',
};

// Compute CP pris from shifts for current year (auto-calculate from leave shifts
function computeCpPris(empId) {
  let count = 0;
  const yr = new Date().getFullYear();
  Object.entries(state.shifts).forEach(([key, shifts]) => {
    if (!key.startsWith(`${empId}_`)) return;
    const wk = key.split('_')[2];
    if (!wk || !wk.startsWith(String(yr))) return;
    (shifts || []).forEach(s => {
      const n = normShift(s);
      if (n.leaveType === 'cp') count++;
    });
  });
}

```

```

    });
    return count;
}

// ----- UTILS -----
function $(sel, root=document) { return root.querySelector(sel); }
function $$ (sel, root=document) { return [...root.querySelectorAll(sel)]; }
function esc(s) { return String(s ?? '').replace(/[\<>"']/g, c => ({'&':'&amp;',
function pad(n) { return String(n).padStart(2, '0'); }
function initials(e) { return `${(e.prenom||'').charAt(0)}${(e.nom||'').charAt(0)

function getMonday(d) {
    const x = new Date(d);
    x.setHours(0,0,0,0);
    const day = x.getDay();
    const diff = day === 0 ? -6 : 1 - day;
    x.setDate(x.getDate() + diff);
    return x;
}

function dateISO(d) { return `${d.getFullYear()}-${pad(d.getMonth()+1)}-${pad(d.g
function weekKey(d) { return dateISO(getMonday(d)); }
function addDays(d, n) { const x = new Date(d); x.setDate(x.getDate()+n); return
function fmtDateLong(d) {
    return d.toLocaleDateString('fr-FR', { weekday: 'long', day: 'numeric', month:
}

function fmtDateShort(d) {
    if (!d) return '-';
    if (typeof d === 'string') {
        const parts = d.split('-');
        if (parts.length === 3) return `${parts[2]}/${parts[1]}/${parts[0]}`;
        return d;
    }
    return d.toLocaleDateString('fr-FR');
}

function fmtRange(d1, d2) {
    const sameMonth = d1.getMonth() === d2.getMonth();
    if (sameMonth) {
        return `${d1.getDate()} - ${d2.getDate()} ${d2.toLocaleDateString('fr-FR',{mo
    }
    return `${d1.toLocaleDateString('fr-FR',{day:'numeric',month:'short'})} - ${d2.
}

function timeToMin(t) { if (!t) return 0; const [h,m] = t.split(':').map(Number);
function minToTime(m) { return `${pad(Math.floor(m/60))}:${pad(m%60)}`; }
function shiftHours(sh) {
    if (!sh) return 0;
    if (isLeave(sh)) return 0;
    let dur = timeToMin(sh.end) - timeToMin(sh.start);

```

```

    if (dur < 0) dur += 24*60;
    if (sh.pauseDuration) dur -= sh.pauseDuration;
    return Math.max(0, dur) / 60;
}

function shiftsForCell(empId, dayIdx, wkKey=weekKey(state.weekStart)) {
    return state.shifts[`${empId}_${dayIdx}_${wkKey}`] || [];
}

// ----- HCR - Avantage en nature repas, coupures, repos -----
// Convention HCR : 4,15 € par repas en 2025 (valeur DSN)
const REPAS_HCR_VALEUR = 4.15;

// Compte les repas pour un shift donné (1 si shift midi ou soir avec pause, 2 si
function repasPourShift(sh) {
    if (!sh || isLeave(sh)) return 0;
    const s = timeToMin(sh.start);
    const e = timeToMin(sh.end);
    const fin = e < s ? e + 24*60 : e;
    // Repas du midi : si shift inclut 12:00-14:00
    const midiOK = s <= 12*60 + 30 && fin >= 13*60 + 30;
    // Repas du soir : si shift inclut 19:00-21:00
    const soirOK = s <= 19*60 + 30 && fin >= 20*60 + 30;
    return (midiOK ? 1 : 0) + (soirOK ? 1 : 0);
}

// Détecte une coupure : 2 shifts ou plus dans la même journée avec >1h d'écart
function coupurePourJour(shifts) {
    const realShifts = shifts.filter(s => !isLeave(s)).sort((a,b) => timeToMin(a.start));
    if (realShifts.length < 2) return 0;
    let totalCoupure = 0;
    for (let i = 1; i < realShifts.length; i++) {
        const prevEnd = timeToMin(realShifts[i-1].end);
        const curStart = timeToMin(realShifts[i].start);
        const gap = curStart - prevEnd;
        if (gap > 60) totalCoupure += gap;
    }
    return totalCoupure / 60; // en heures
}

// Calcule le repos hebdo le plus long (en heures) pour une semaine
function reposHebdoMaximum(empId, wkStart) {
    const wk = weekKey(wkStart);
    let maxRepos = 0;
    // On scan de la fin du dimanche précédent jusqu'au lundi suivant pour détecter
    const events = []; // {ts, type:'end' | 'start'}
    for (let d = -1; d <= 7; d++) {
        const day = addDays(wkStart, d);

```



```

const dayIdx = d < 0 ? 6 : (d > 6 ? 0 : d);
const wkRef = d < 0 ? weekKey(addDays(wkStart, -7)) : (d > 6 ? weekKey(addDay
const shifts = (state.shifts[`${empId}_${dayIdx}_${wkRef}`] || []).filter(s =
shifts.forEach(s => {
    const startMin = day.getTime()/60000 + timeToMin(s.start);
    let endMin = day.getTime()/60000 + timeToMin(s.end);
    if (timeToMin(s.end) < timeToMin(s.start)) endMin += 24*60;
    events.push({ ts: startMin, type: 'start' });
    events.push({ ts: endMin, type: 'end' });
});
}
events.sort((a, b) => a.ts - b.ts);
for (let i = 0; i < events.length - 1; i++) {
    if (events[i].type === 'end' && events[i+1].type === 'start') {
        const gap = events[i+1].ts - events[i].ts;
        // On garde uniquement les gaps qui chevauchent la semaine
        const wkStartMin = wkStart.getTime()/60000;
        const wkEndMin = addDays(wkStart, 7).getTime()/60000;
        if (events[i+1].ts > wkStartMin && events[i].ts < wkEndMin) {
            maxRepos = Math.max(maxRepos, gap / 60);
        }
    }
}
return maxRepos;
}

// Récap mensuel HCR par salarié pour la paie
function hcrMensuelEmp(empId, year, month) {
    const monthStart = new Date(year, month, 1);
    const monthEnd = new Date(year, month+1, 0);
    let repas = 0, coupures = 0;
    for (let d = new Date(monthStart); d <= monthEnd; d = addDays(d, 1)) {
        const dayIdx = (d.getDay() + 6) % 7;
        const wk = weekKey(d);
        const shifts = (state.shifts[`${empId}_${dayIdx}_${wk}`] || []).map(normShift
        shifts.forEach(s => repas += repasPourShift(s));
        coupures += coupurePourJour(shifts);
    }
    return { repas, coupures, valeurRepas: repas * REPAS_HCR_VALEUR };
}

// _____ TOAST _____
function toast(msg, type='', dur=3000) {
    const el = document.createElement('div');
    el.className = `toast${type ? ' '+type : ''}`;
    el.textContent = msg;
    $('#toast-host').appendChild(el);

```

```

    setTimeout(() => {
        el.style.animation = 'toastOut .25s forwards';
        setTimeout(() => el.remove(), 250);
    }, dur);
}

// ----- MODAL -----
function openModal({ title, body, footer, onClose }) {
    const bg = document.createElement('div');
    bg.className = 'modal-bg';
    bg.innerHTML = `
        <div class="modal">
            <div class="modal-h">
                <h3>${esc(title)}</h3>
                <button class="btn-ghost" data-close aria-label="Fermer">x</button>
            </div>
            <div class="modal-b">${body}</div>
            ${footer ? `<div class="modal-f">${footer}</div>` : ''}
        </div>`;
    document.body.appendChild(bg);
    document.body.style.overflow = 'hidden';

    const close = () => {
        if (bg.classList.contains('closing')) return;
        bg.classList.add('closing');
        document.removeEventListener('keydown', escHandler);
        setTimeout(() => {
            bg.remove();
            document.body.style.overflow = '';
            onClose && onClose();
        }, 250);
    };

    bg.addEventListener('click', (e) => { if (e.target === bg) close(); });
    $$(['data-close', bg]).forEach(b => b.addEventListener('click', close));
    const escHandler = (e) => { if (e.key === 'Escape') close(); };
    document.addEventListener('keydown', escHandler);
    return { bg, close };
}

// ----- FIREBASE -----
let db = null;
async function initFirebase() {
    if (typeof firebase === 'undefined') {
        console.warn('Firebase not available - local mode');
        return false;
    }
}

```

```

try {
  if (!firebase.apps.length) firebase.initializeApp(FB_CONFIG);
  await firebase.auth().signInAnonymously();
  db = firebase.database();
  state.fbReady = true;
  return true;
} catch (e) {
  console.error('Firebase init failed:', e);
  toast('Connexion Firebase impossible - mode local', 'error');
  return false;
}
}

// ----- DOCUMENTS - stockage base64 dans RTDB (sans Firebase Storage) -----
const MAX_DOC_SIZE = 2 * 1024 * 1024; // 2 Mo limite côté upload (un peu plus en
const MAX_RTDB_SIZE = 8 * 1024 * 1024; // 8 Mo dur (limite RTDB par node = 10 Mo)

async function compressImageIfNeeded(file, maxDim = 1800, quality = 0.85) {
  if (!file.type || !file.type.startsWith('image/')) return file;
  return new Promise((resolve) => {
    const img = new Image();
    img.onload = () => {
      let { width, height } = img;
      if (width <= maxDim && height <= maxDim && file.size < 600*1024) {
        URL.revokeObjectURL(img.src); resolve(file); return;
      }
      const canvas = document.createElement('canvas');
      if (width > height) {
        if (width > maxDim) { height = (maxDim / width) * height; width = maxDim;
        } else {
          if (height > maxDim) { width = (maxDim / height) * width; height = maxDim;
          }
        }
      }
      canvas.width = width; canvas.height = height;
      canvas.getContext('2d').drawImage(img, 0, 0, width, height);
      canvas.toBlob(blob => {
        URL.revokeObjectURL(img.src);
        if (!blob) { resolve(file); return; }
        const out = new File([blob], file.name.replace(/\.[^.]*/, '') + '.jpg',
          resolve(out);
        }, 'image/jpeg', quality);
      });
      img.onerror = () => { URL.revokeObjectURL(img.src); resolve(file); };
      img.src = URL.createObjectURL(file);
    });
  });
}

async function fileToDataURL(file) {

```

```

return new Promise((resolve, reject) => {
  const r = new FileReader();
  r.onload = () => resolve(r.result);
  r.onerror = () => reject(r.error);
  r.readAsDataURL(file);
});
}

async function uploadEmployeeDoc(empId, file, category) {
  if (!db) { toast('Firebase non disponible', 'error'); return null; }

  // Auto-compress images
  let workFile = file;
  if (file.type && file.type.startsWith('image/')) {
    try { workFile = await compressImageIfNeeded(file); } catch(e) { /* ignore */
    }

    if (workFile.size > MAX_DOC_SIZE) {
      toast(`Fichier trop gros (${formatBytes(workFile.size)}). Limite 2 Mo - compr
      return null;
    }

    try {
      const dataUrl = await fileToDataUrl(workFile);
      if (dataUrl.length > MAX_RTDB_SIZE) {
        toast(`Fichier trop volumineux après encodage. Réduis encore.`, 'error', 60
        return null;
      }

      const docId = 'd' + Date.now() + Math.random().toString(36).slice(2,7);
      // Save the blob in a SEPARATE node (not loaded on startup)
      await db.ref(`docBlobs/${docId}`).set(dataUrl);
      return {
        id: docId,
        name: workFile.name,
        size: workFile.size,
        type: workFile.type,
        category,
        uploadedAt: new Date().toISOString(),
        // Note: no url field, fetched on demand from docBlobs/{id}
      };
    } catch (e) {
      console.error('Upload failed', e);
      toast('Échec upload : ' + (e.message || e.code), 'error', 6000);
      return null;
    }
  }
}

```

```

async function fetchDocBlob(docId) {
  if (!db) return null;
  try {
    const snap = await db.ref(`docBlobs/${docId}`).once('value');
    return snap.val();
  } catch (e) {
    console.error('Fetch blob failed', e);
    return null;
  }
}

async function openDocInNewTab(docId, fileName) {
  toast('Ouverture du document...', '');
  const dataUrl = await fetchDocBlob(docId);
  if (!dataUrl) { toast('Document introuvable', 'error'); return; }
  const win = window.open('', '_blank');
  if (!win) { toast('Pop-up bloquée – autorise les pop-ups pour ouvrir le documen
  // Detect if image, PDF, or other
  const isImage = dataUrl.startsWith('data:image/');
  const isPdf = dataUrl.startsWith('data:application/pdf');
  win.document.title = fileName;
  if (isImage) {
    win.document.body.style.margin = '0';
    win.document.body.style.background = '#000';
    win.document.body.innerHTML = `
```

```

}

async function deleteDocBlob(docId) {
  if (!db) return;
  try { await db.ref(`docBlobs/${docId}`).remove(); }
  catch (e) { console.warn(e); }
}

function formatBytes(b) {
  if (b < 1024) return b + ' o';
  if (b < 1024*1024) return (b/1024).toFixed(1) + ' Ko';
  return (b/(1024*1024)).toFixed(1) + ' Mo';
}

const DOC_CATEGORIES = {
  contrat:      { label: 'Contrat de travail',    icon: '📄' },
  avenant:      { label: 'Avenant',              icon: '📝' },
  fiche_paie:   { label: 'Fiche de paie',        icon: '💰' },
  arret_maladie: { label: 'Arrêt maladie',       icon: '🏠' },
  navigo:       { label: 'Pass Navigo',          icon: '🚆' },
  titre_sejour: { label: 'Titre de séjour',      icon: '🛂' },
  carte_identite: { label: "Pièce d'identité",   icon: '🪪' },
  rib:          { label: 'RIB',                  icon: '💰' },
  certificat:   { label: 'Certificat médical',   icon: '🩺' },
  diplome:      { label: 'Diplôme',              icon: '🎓' },
  formation:    { label: 'Attestation formation', icon: '📖' },
  autre:        { label: 'Autre document',       icon: '📎' },
};

function fbListen() {
  if (!db) return;
  ['employees', 'shifts', 'punches', 'publications', 'absenceRequests', 'emargements',
  db.ref(k).on('value', snap => {
    const v = snap.val();
    if (k === 'employees' && v) {
      if (Array.isArray(v)) {
        state.employees = v.filter(Boolean);
      } else {
        state.employees = Object.values(v).filter(Boolean);
      }
      state.employees.forEach(e => {
        if (!e.username) e.username = (e.prenom || '').toLowerCase();
        if (!e.code) e.code = '1234';
      });
      console.log('[fb] employees loaded:', state.employees.length);
    }
    if (k === 'shifts' && v) {

```

```

        state.shifts = v;
        migrateLegacyShifts();
        console.log('[fb] shifts loaded:', Object.keys(state.shifts).length);
    }
    if (k === 'punches' && v) state.punches = v;
    if (k === 'publications' && v) state.publications = v;
    if (k === 'absenceRequests' && v) state.absenceRequests = v;
    if (k === 'emargements' && v) state.emargements = v;
    if (k === 'pourboires' && v) state.pourboires = v;
    if (k === 'duerp' && v) state.duerp = v;
    if (k === 'affichages' && v) state.affichages = v;
    if (k === 'payslips' && v) state.payslips = v;
    if (k === 'adminCredentials' && v) state.adminCredentials = v;
    render();
}, err => {
    console.error('[fb] read error on', k, err);
    toast(`Lecture Firebase refusée (${k}) – vérifie les règles`, 'error', 6000
    });
});
}

// Migrate legacy RH_ULTIME shift keys (empId_dayIdx) to new format (empId_dayIdx
// for the current week, so existing planning shows up immediately.
function migrateLegacyShifts() {
    if (!state.shifts) return;
    const currentWk = weekKey(state.weekStart);
    const updates = {};
    let count = 0;
    Object.keys(state.shifts).forEach(key => {
        const parts = key.split('_');
        if (parts.length === 2 && !isNaN(parts[0]) && !isNaN(parts[1])) {
            const newKey = `${key}_${currentWk}`;
            if (!state.shifts[newKey]) {
                state.shifts[newKey] = state.shifts[key];
                updates[newKey] = state.shifts[key];
                count++;
            }
        }
    });
    if (count && db) {
        console.log(`[migration] copying ${count} legacy shifts to current week`);
        db.ref('shifts').update(updates).catch(e => console.warn('migration write fai
        toast(`${count} shifts récupérés de l'ancien planning`, 'good', 4000);
    }
}

function fbSave(path, value) {

```

```

    if (!db) return;
    return db.ref(path).set(value).catch(e => {
        console.error(e); toast('Erreur de synchronisation', 'error');
    });
}

async function seedIfEmpty() {
    if (!db) return;
    const snap = await db.ref('employees').once('value');
    if (!snap.exists()) {
        await db.ref('employees').set(DEFAULT_EMPLOYEES);
        state.employees = DEFAULT_EMPLOYEES;
    }
}

// _____ AUTH _____
// SHA-256 helper for password hashing
async function sha256(str) {
    const buf = new TextEncoder().encode(str);
    const hashBuf = await crypto.subtle.digest('SHA-256', buf);
    return [...new Uint8Array(hashBuf)].map(b => b.toString(16).padStart(2, '0')).join('');
}

async function authAdmin(u, c) {
    // 1. Vérifier les credentials stockés dans Firebase
    const stored = state.adminCredentials;
    if (stored && stored.username && stored.passwordHash) {
        if (u === stored.username) {
            const hash = await sha256(c);
            if (hash === stored.passwordHash) {
                state.user = { role: 'admin' };
                sessionStorage.setItem('mu_user', JSON.stringify(state.user));
                return true;
            }
        }
    }
    return false;
}

// 2. Fallback sur les credentials par défaut hardcodés (premier login)
if (u === ADMIN_USER && c === ADMIN_CODE) {
    state.user = { role: 'admin' };
    sessionStorage.setItem('mu_user', JSON.stringify(state.user));
    return true;
}
return false;
}

function authEmp(u, c) {
    const e = state.employees.find(x => x.username === u && String(x.code) === Stri

```



```

    if (e) {
        state.user = { role: 'emp', empId: e.id, prenom: e.prenom, nom: e.nom };
        sessionStorage.setItem('mu_user', JSON.stringify(state.user));
        return e;
    }
    return null;
}

function logout() {
    state.user = null;
    sessionStorage.removeItem('mu_user');
    state.page = null;
    render();
}

// ----- ANOMALY DETECTION -----
function detectAnomalies(empId=null, fromDate=null, toDate=null) {
    const anomalies = [];
    const from = fromDate || addDays(state.weekStart, -28);
    const to = toDate || addDays(state.weekStart, 7);

    const emps = empId ? state.employees.filter(e => e.id === empId) : state.employ

    emps.forEach(emp => {
        // Walk every day between from..to
        for (let d = new Date(from); d <= to; d = addDays(d, 1)) {
            const iso = dateISO(d);
            const dayIdx = (d.getDay() + 6) % 7;
            const wk = weekKey(d);
            const planned = state.shifts[`${emp.id}_${dayIdx}_${wk}`] || [];
            const punches = state.punches[`${emp.id}_${iso}`] || [];

            // Rule 1: planned but not pointed
            if (planned.length > 0 && punches.length === 0 && d < new Date()) {
                const isToday = iso === dateISO(new Date());
                if (!isToday) {
                    anomalies.push({
                        type: 'oubli',
                        severity: 'alert',
                        empId: emp.id,
                        date: iso,
                        title: `Oubli de pointage`,
                        detail: `${emp.prenom} ${emp.nom} - ${fmtDateLong(d)} (${planned[0].s
                    });
                }
            }
        }
    });

    // Rule 2: pointed but no plan

```

```

if (punches.length > 0 && planned.length === 0) {
  anomalies.push({
    type: 'horsplan',
    severity: 'warn',
    empId: emp.id,
    date: iso,
    title: `Pointage hors planning`,
    detail: `${emp.prenom} ${emp.nom} - ${fmtDateLong(d)} (aucun shift prév
  });
}

// Rule 3: incomplete punch (entrée sans sortie pour un jour passé)
const lastP = punches[punches.length-1];
if (lastP && lastP.in && !lastP.out && d < new Date(new Date()).setHours(0,0
  anomalies.push({
    type: 'incomplet',
    severity: 'alert',
    empId: emp.id,
    date: iso,
    title: `Pointage incomplet`,
    detail: `${emp.prenom} ${emp.nom} - entrée à ${lastP.in} le ${fmtDateLo
  });
}

// Rule 4: écart contrat/réel sur la journée > 1h
if (planned.length > 0 && punches.length > 0 && lastP.out) {
  const plannedH = planned.reduce((s, sh) => s + shiftHours(sh), 0);
  const realH = punches.reduce((s, p) => {
    if (!p.in || !p.out) return s;
    let d = timeToMin(p.out) - timeToMin(p.in);
    if (d < 0) d += 24*60;
    return s + d/60;
  }, 0);
  const gap = realH - plannedH;
  if (Math.abs(gap) >= 1) {
    anomalies.push({
      type: 'ecart',
      severity: gap > 0 ? 'warn' : 'alert',
      empId: emp.id,
      date: iso,
      title: `Écart contrat/réel : ${gap > 0 ? '+' : ''}${gap.toFixed(1)}h`
      detail: `${emp.prenom} ${emp.nom} - ${fmtDateLong(d)} : prévu ${plann
    });
  }
}
}
}

```

```

// Rule 5: dépassement horaire hebdo contrat
const weeks = new Set();
for (let d = new Date(from); d <= to; d = addDays(d, 1)) weeks.add(weekKey(d))
weeks.forEach(wk => {
  let totalReal = 0;
  for (let i = 0; i < 7; i++) {
    const d = addDays(new Date(wk), i);
    const punches = state.punches[`${emp.id}_${dateISO(d)}`] || [];
    punches.forEach(p => {
      if (p.in && p.out) {
        let dur = timeToMin(p.out) - timeToMin(p.in);
        if (dur < 0) dur += 24*60;
        totalReal += dur / 60;
      }
    });
  }
  const limit = (emp.heures || 35);
  if (totalReal > limit + 3) {
    anomalies.push({
      type: 'depassement',
      severity: 'warn',
      empId: emp.id,
      date: wk,
      title: `Heures sup. non déclarées`,
      detail: `${emp.prenom} ${emp.nom} - semaine du ${new Date(wk).toLocaleD
    });
  }
}

// Rule 6 (HCR): repos hebdomadaire < 35h consécutives
const wkDate = new Date(wk + 'T00:00:00');
const wkEnd = addDays(wkDate, 6);
const todayD = new Date(); todayD.setHours(0,0,0,0);
// On ne checke que pour les semaines complétées (passées) ou en cours
if (wkDate <= todayD) {
  const repos = reposHebdoMaximum(emp.id, wkDate);
  // S'il y a au moins 1 shift cette semaine
  let hasShifts = false;
  for (let i = 0; i < 7; i++) {
    if (((state.shifts[`${emp.id}_${i}_${wk}`]) || []).some(s => !isLeave(s))
  }
  if (hasShifts && repos > 0 && repos < 35) {
    anomalies.push({
      type: 'repos_hebdo',
      severity: repos < 24 ? 'alert' : 'warn',
      empId: emp.id,
      date: wk,
      title: `Repos hebdomadaire insuffisant`,

```

```

        detail: `${emp.prenom} ${emp.nom} - semaine du ${new Date(wk).toLocale
    });
    }
    }
    });
});

return anomalies;
}

// ----- ROUTING / RENDER -----
function render() {
    const app = $('#app');
    if (state.loading) {
        return; // boot screen stays
    }
    if (!state.user) {
        app.innerHTML = viewLogin();
        bindLogin();
        return;
    }
    if (state.user.role === 'admin') {
        if (!state.page) state.page = 'dashboard';
        app.innerHTML = viewAdminShell();
        bindAdmin();
        return;
    }
    if (state.user.role === 'emp') {
        if (!state.page) state.page = 'home';
        app.innerHTML = viewEmpShell();
        bindEmp();
        return;
    }
}

// ----- LOGIN VIEW -----
let loginMode = 'emp'; // default to employee (mobile use case)

function viewLogin() {
    return `
        <div class="login-screen">
            <div class="login-card">
                <div class="login-mark">M</div>
                <h1 class="login-h">Man'ouché</h1>
                <div class="login-sub">${loginMode === 'admin' ? 'Console de gestion' : '
                    <div class="tab-switch" data-active="${loginMode === 'admin' ? '1' : '0'}

```

```

        <button data-mode="emp" class="${loginMode=== 'emp'? 'active': ''}">Salari
        <button data-mode="admin" class="${loginMode=== 'admin'? 'active': ''}">Ma
    </div>

    <form id="loginForm">
        <div class="field">
            <label class="field-label">${loginMode === 'admin' ? 'Identifiant' :
            <input class="input" id="loginUser" autocomplete="username" placeholder
        </div>
        <div class="field">
            <label class="field-label">${loginMode === 'admin' ? 'Code' : 'Code P
            <input class="input mono" id="loginCode" type="password" ${loginMode
        </div>
        <button class="btn-pri" type="submit">Se connecter</button>
    </form>

    <div class="login-status">
        <span class="dot ${state.fbReady ? 'on' : 'off'}"></span>
        <span>${state.fbReady ? 'Synchronisation active' : 'Mode local'}</span>
    </div>
</div>`;
}

function bindLogin() {
    $$('[data-mode]').forEach(b => b.addEventListener('click', e => {
        loginMode = e.target.dataset.mode;
        render();
    }));
    $('#loginForm').addEventListener('submit', async (e) => {
        e.preventDefault();
        const u = $('#loginUser').value.trim().toLowerCase();
        const c = $('#loginCode').value.trim();
        if (!u || !c) { toast('Renseigne tous les champs', 'error'); return; }
        if (loginMode === 'admin') {
            const ok = await authAdmin(u, c);
            if (ok) { toast('Bienvenue', 'good'); render(); }
            else toast('Identifiants invalides', 'error');
        } else {
            const emp = authEmp(u, c);
            if (emp) { toast(`Bonjour ${emp.prenom}`, 'good'); render(); }
            else toast('Identifiants invalides', 'error');
        }
    });
}

// ————— ADMIN SHELL —————

```

```

function viewAdminShell() {
  const anomalies = detectAnomalies();
  const pendingReqs = Object.values(state.absenceRequests || {}).filter(r => r &&
  const rappels = computeRappels();
  const rappelsCount = rappels.filter(r => r.severity !== 'info').length;
  const unsignedCount = countUnsignedEmargements();

  // Top-level sections - like Combo
  const TOP_SECTIONS = [
    { key: 'dashboard',    label: 'Aperçu',      pages: ['dashboard'],
    { key: 'planning',     label: 'Planning',    pages: ['planning', 'month'],
    { key: 'team',         label: 'Équipe',      pages: ['employees'],
    { key: 'hours',        label: 'Heures',      pages: ['hours', 'pointages'],
    { key: 'rappels',      label: 'Rappels',      pages: ['rappels'],
    { key: 'emargements',  label: 'Émargements', pages: ['emargements'],
    { key: 'rh',           label: 'RH',          pages: ['requests', 'cp', 'rh'],
  ];

  const currentSection = TOP_SECTIONS.find(s => s.pages.includes(state.page)) ||
  const showSubNav = currentSection.key === 'rh';

  return `
    <div class="shell">
      <header class="topnav">
        <div class="topnav-inner">
          <div class="topnav-brand">
            <span class="brand-mark">M</span>
            <span class="brand-name">Man'ouché</span>
          </div>
          <nav class="topnav-links">
            ${TOP_SECTIONS.map(s => {
              const active = s.key === currentSection.key;
              const firstPage = s.pages[0];
              return `<button class="topnav-link ${active?'active':''}" data-page=
            }).join('')}
          </nav>
          <div class="topnav-right">
            <button class="topnav-icon" title="${state.fbReady?'Connecté':'Hors l
              <span class="status-dot-tiny" style="background:${state.fbReady?'#2
            </button>
            ${anomalies.length ? `<button class="topnav-icon" data-page="alerts"
              <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke=
              <span class="dot-badge">${anomalies.length}</span>
            </button>` : ''}
            <button class="topnav-icon" data-logout title="Déconnexion">
              <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke=
            </button>

```

```

        <button class="topnav-avatar" id="openAdminSettings" title="Paramètre
    </div>
</div>
<div class="topnav-mobile">
    <select id="mobNav">
        <optgroup label="Tableau de bord">
            <option value="dashboard" ${state.page==='dashboard'? 'selected':''}>
        </optgroup>
        <optgroup label="Planning">
            <option value="planning" ${state.page==='planning'? 'selected':''}>P
            <option value="month" ${state.page==='month'? 'selected':''}>Vue moi
        </optgroup>
        <optgroup label="Équipe">
            <option value="employees" ${state.page==='employees'? 'selected':''}>
        </optgroup>
        <optgroup label="Heures">
            <option value="hours" ${state.page==='hours'? 'selected':''}>Prépara
            <option value="pointages" ${state.page==='pointages'? 'selected':''}>
        </optgroup>
        <optgroup label="Conformité">
            <option value="rappels" ${state.page==='rappels'? 'selected':''}>Rap
            <option value="emargements" ${state.page==='emargements'? 'selected'
            <option value="etablissement" ${state.page==='etablissement'? 'selec
        </optgroup>
        <optgroup label="RH">
            <option value="requests" ${state.page==='requests'? 'selected':''}>D
            <option value="cp" ${state.page==='cp'? 'selected':''}>Compteurs CP<
            <option value="rh" ${state.page==='rh'? 'selected':''}>Suivi salarié
            <option value="alerts" ${state.page==='alerts'? 'selected':''}>Alert
        </optgroup>
    </select>
</div>
</header>

<div class="shell-body">
    ${showSubNav ? `
        <aside class="subnav">
            <div class="subnav-section">Suivi RH</div>
            <button class="subnav-item ${state.page==='requests'? 'active':''}" da
                Demandes d'absence
                ${pendingReqs > 0 ? `<span class="subnav-badge">${pendingReqs}</spa
            </button>
            <button class="subnav-item ${state.page==='cp'? 'active':''}" data-pag
            <button class="subnav-item ${state.page==='rh'? 'active':''}" data-pag
            <div class="subnav-section" style="margin-top:18px;">Conformité</div>
            <button class="subnav-item ${state.page==='etablissement'? 'active':''
                Établissement
    `}

```

```

        </button>
        <button class="subnav-item ${state.page==='alerts'?'active':''}" data
            Alertes pointage
            ${anomalies.length ? `<span class="subnav-badge alert">${anomalies.
        </button>
    </aside>
    ` : ''}

    <main class="main ${showSubNav?'has-subnav':''}">
        <div class="page-pad fade-in" id="adminBody">
            ${renderAdminPage()}
        </div>
    </main>
</div>
</div>`;
}

```

```

function renderAdminPage() {
    switch (state.page) {
        case 'dashboard': return pageDashboard();
        case 'planning': return pagePlanning();
        case 'month': return pageMonth();
        case 'hours': return pageHours();
        case 'pointages': return pagePointages();
        case 'alerts': return pageAlerts();
        case 'employees': return state.empDetail ? pageEmployeeDetail() : pageEmployee
        case 'requests': return pageAbsenceRequests();
        case 'cp': return pageCpCompteurs();
        case 'rh': return pageRH();
        case 'rappels': return pageRappels();
        case 'emargements': return pageEmargements();
        case 'etablissement': return pageEtablissement();
        default: return pageDashboard();
    }
}

```

```

function bindAdmin() {
    $$('[data-page]').forEach(b => b.addEventListener('click', e => {
        state.page = e.currentTarget.dataset.page;
        state.empDetail = null;
        render();
    }));
    $$('[data-logout]').forEach(b => b.addEventListener('click', logout));
    const mob = $('#mobNav');
    if (mob) mob.addEventListener('change', e => {
        state.page = e.target.value;
        state.empDetail = null;
    });
}

```



```

    render();
  });
  const settingsBtn = $('#openAdminSettings');
  if (settingsBtn) settingsBtn.addEventListener('click', () => openAdminSettings(

switch (state.page) {
  case 'dashboard': bindDashboard(); break;
  case 'planning': bindPlanning(); break;
  case 'month': bindMonth(); break;
  case 'hours': bindHours(); break;
  case 'pointages': bindPointages(); break;
  case 'alerts': bindAlerts(); break;
  case 'employees': state.empDetail ? bindEmployeeDetail() : bindEmployees(); b
  case 'requests': bindAbsenceRequests(); break;
  case 'cp': bindCpCompteurs(); break;
  case 'rh': bindRH(); break;
  case 'rappels': bindRappels(); break;
  case 'emargements': bindEmargements(); break;
  case 'etablissement': bindEtablissement(); break;
}
}

// ----- DASHBOARD -----
function pageDashboard() {
  const actives = state.employees.filter(e => e.statut === 'Actif');
  const wk = weekKey(state.weekStart);
  let totalHours = 0;
  actives.forEach(e => {
    for (let d = 0; d < 7; d++) {
      const sh = state.shifts[`${e.id}_${d}_${wk}`] || [];
      sh.forEach(s => totalHours += shiftHours(s));
    }
  });
  const anomalies = detectAnomalies();
  const todayISO = dateISO(new Date());
  const todayDay = (new Date().getDay() + 6) % 7;
  const todayShifts = actives.filter(e => (state.shifts[`${e.id}_${todayDay}_${wk}`

return `
  <div class="page-head">
    <div>
      <div class="uppercase-eyebrow">${fmtDateLong(new Date())}</div>
      <h1 class="h-display">Bonjour Jad.</h1>
    </div>
  </div>

  <div class="kpi-grid">

```

```

<div class="kpi">
  <div class="kpi-label">Salariés actifs</div>
  <div class="kpi-value">${actives.length}</div>
  <div class="kpi-meta">${state.employees.length - actives.length} inactifs
</div>
<div class="kpi">
  <div class="kpi-label">Heures planifiées</div>
  <div class="kpi-value">${Math.round(totalHours)}<span style="font-size:18">
  <div class="kpi-meta">Semaine du ${state.weekStart.toLocaleDateString('fr
</div>
<div class="kpi">
  <div class="kpi-label">Au travail aujourd'hui</div>
  <div class="kpi-value">${todayShifts}</div>
  <div class="kpi-meta">sur ${actives.length} actifs</div>
</div>
<div class="kpi ${anomalies.length ? 'alert' : ''}">
  <div class="kpi-label">Anomalies</div>
  <div class="kpi-value">${anomalies.length}</div>
  <div class="kpi-meta">${anomalies.length ? 'à examiner' : 'tout est bon'}
</div>
</div>

<div class="panel">
  <div class="panel-head">
    <h3>Équipe en service aujourd'hui</h3>
    <button class="btn-ghost" data-page="planning">Voir le planning →</button>
  </div>
  <div class="panel-body nopad">
    ${renderTodayTeam(actives, todayDay, wk)}
  </div>
</div>

${anomalies.length ? `
  <div class="panel" style="margin-top:16px;">
    <div class="panel-head">
      <h3>Anomalies récentes</h3>
      <button class="btn-ghost" data-page="alerts">Tout voir →</button>
    </div>
    <div class="panel-body nopad">
      ${anomalies.slice(0, 5).map(a => renderAlertRow(a)).join('')}
    </div>
  </div>` : ''}
`;
}

function renderTodayTeam(actives, dayIdx, wk) {
  const onShift = actives

```

```

    .map(e => ({ e, shifts: state.shifts[`${e.id}_${dayIdx}_${wk}`] || [] }))
    .filter(x => x.shifts.length);
if (!onShift.length) {
    return `<div class="empty"><h4>Personne aujourd'hui</h4><div>Aucun shift plan
}
return `<table class="tbl">
  <thead><tr><th>Salarié</th><th>Horaire</th><th>Type</th><th>Heures</th></tr><
  <tbody>
    ${onShift.map(({e, shifts}) => shifts.map(s => `
      <tr>
        <td><div class="emp-cell"><div class="av-emp">${initials(e)}</div><div><d
        <td class="mono">${s.start} → ${s.end}</td>
        <td><span class="chip">${s.type === 'midi' ? 'Midi' : s.type === 'soir' ?
        <td class="mono tabular">${shiftHours(s).toFixed(1)} h</td>
      </tr>
    `).join('')).join('')}
  </tbody>
</table>`;
}

```

```

function renderAlertRow(a) {
  const e = state.employees.find(x => x.id === a.empId);
  return `
    <div class="alert-row ${a.severity === 'warn' ? 'warn' : ''}">
      <div class="alert-icon">!</div>
      <div class="alert-body">
        <div class="alert-title">${esc(a.title)}</div>
        <div class="alert-meta">${esc(a.detail)}</div>
      </div>
    </div>`;
}

```

```

function bindDashboard() {
  $$('[data-page]').forEach(b => b.addEventListener('click', e => {
    const p = e.currentTarget.dataset.page;
    if (p) { state.page = p; render(); }
  }));
}

```

// ————— PLANNING —————

```

function pagePlanning() {
  const wk = weekKey(state.weekStart);
  const actives = state.employees.filter(e => e.statut === 'Actif');
  const wkEnd = addDays(state.weekStart, 6);
  const todayISO = dateISO(new Date());

  let weekShiftCount = 0;

```

```

actives.forEach(e => {
  for (let d = 0; d < 7; d++) {
    weekShiftCount += (state.shifts[`${e.id}_${d}_${wk}`] || []).length;
  }
});

const prevWk = weekKey(addDays(state.weekStart, -7));
let prevWeekShiftCount = 0;
actives.forEach(e => {
  for (let d = 0; d < 7; d++) {
    prevWeekShiftCount += (state.shifts[`${e.id}_${d}_${prevWk}`] || []).length
  }
});

const pub = state.publications[wk];
const isPublished = !!pub;

// Total heures planifiées
let totalH = 0;
actives.forEach(e => {
  for (let d = 0; d < 7; d++) {
    const shifts = state.shifts[`${e.id}_${d}_${wk}`] || [];
    shifts.forEach(s => totalH += shiftHours(s));
  }
});

// Jours fériés de la semaine
const weekFeries = [];
for (let i = 0; i < 7; i++) {
  const d = addDays(state.weekStart, i);
  const f = holidayFor(d);
  if (f) weekFeries.push({ date: d, dayIdx: i, ...f });
}

return `
  <div class="page-head">
    <div>
      <div class="uppercase-eyebrow">Planning</div>
      <h1 class="h-1">${fmtRange(state.weekStart, wkEnd)}</h1>
      <div class="row" style="gap:8px;margin-top:10px;flex-wrap:wrap;">
        <span class="chip">${Math.round(totalH)} h planifiées</span>
        ${isPublished
          ? `<span class="chip good">✓ Publié ${new Date(pub.publishedAt).toLocaleDateString()}</span>`
          : `<span class="chip draft">● Brouillon – invisible des salariés</span>`
        }
        <span class="chip">Semaine vide</span>`
    </div>
    ${schoolHolidaysOverlappingWeek(state.weekStart).map(h =>

```

```

        `<span class="chip accent" title="Zone C · ${new Date(h.start).toLocaleDate()}.join('')}`
        ${weekFeries.map(f =>
            `<span class="chip warn"> 17 ${esc(f.short)} (${DAYS_SHORT[f.dayIdx]})`
        ).join('')}
    </div>
</div>
<div class="page-actions">
    <div class="week-nav">
        <button class="btn-icon" data-week="prev" title="Semaine précédente">←<
        <span class="week-label">Semaine du ${pad(state.weekStart.getDate())}/${
        <button class="btn-icon" data-week="next" title="Semaine suivante">→</b>
        <button class="btn-sec" data-week="today">Aujourd'hui</button>
    </div>
</div>
</div>

${!isPublished && weekShiftCount > 0 ? `
    <div class="info-banner draft-banner">
        <span style="font-size:18px;"></span>
        <div style="flex:1;">
            <strong>Cette semaine est un brouillon</strong>
            <div style="font-size:12px;margin-top:2px;opacity:.85;">Les salariés ne
        </div>
        <button class="btn-link" id="dismissDraftInfo">×</button>
    </div>
` : ``}

${weekFeries.length ? `
    <div class="ferie-banner">
        <span class="ferie-icon"></span>
        <div>
            <strong>Jour${weekFeries.length>1?'s':''} férié${weekFeries.length>1?'s'
            <div style="font-size:12px;margin-top:2px;">${weekFeries.map(f => ` ${es
        </div>
    </div>
` : ``}

<div class="row plan-actions">
    <button class="btn-sec" id="dupFromPrev" ${prevWeekShiftCount > 0 ? '' : 'd
    <button class="btn-sec" id="clearWeek" ${weekShiftCount > 0 ? '' : 'disable
    <button class="btn-sec" id="exportCsv" ${weekShiftCount > 0 ? '' : 'disable
    <button class="btn-sec" id="exportPdf" ${weekShiftCount > 0 ? '' : 'disable
    <div class="spacer"></div>
    ${isPublished ? `<button class="btn-sec" id="unpublishBtn">Dépublier (repas
    <button class="btn-pri" id="publishBtn" style="width:auto;padding:9px 16px;
        ${isPublished ? '🔄 Republier le planning' : '📢 Publier le planning'}`

```

```
</button>
```

```
</div>
```

```
`${weekShiftCount} === 0 ? `
```

```
<div class="panel" style="margin-bottom:14px;background:#fafafa;border-styl
```

```
<div class="panel-body" style="text-align:center;padding:24px 18px;">
```

```
<div class="serif" style="font-size:22px;letter-spacing:-0.02em;margin-
```

```
<div class="text-mute" style="font-size:13px;margin-bottom:16px;">Cliqu
```

```
<button class="btn-pri" id="seedDefault" style="width:auto;padding:10px
```

```
</div>
```

```
</div>` : ''}
```

```
<div class="panel">
```

```
<div class="panel-body nopad">
```

```
<div class="plan-wrap">
```

```
<div class="plan">
```

```
<div class="plan-cell head"><div class="day">Salarié</div></div>
```

```
`${[0,1,2,3,4,5,6].map(i => {
```

```
  const d = addDays(state.weekStart, i);
```

```
  const isToday = dateISO(d) === todayISO;
```

```
  const fer = holidayFor(d);
```

```
  const vac = schoolHolidayFor(d);
```

```
  return `<div class="plan-cell head ${fer?'ferie':''} ${vac} && !fer?
```

```
    <div class="day">${DAYS_SHORT[i]}</div>
```

```
    <div class="date ${isToday?'today':''}>${d.getDate()}</div>
```

```
    ${fer ? `<div class="ferie-tag" title="${esc(fer.long)}">${esc(fe
```

```
    </div>`;
```

```
  })).join('')}
```

```
`${actives.map(e => {
```

```
  let empWeekH = 0;
```

```
  for (let d = 0; d < 7; d++) {
```

```
    const shifts = state.shifts[`${e.id}_${d}_${wk}`] || [];
```

```
    shifts.forEach(s => empWeekH += shiftHours(s));
```

```
  }
```

```
  const contractH = e.heures || 35;
```

```
  const gap = empWeekH - contractH;
```

```
  const gapClass = Math.abs(gap) < 0.5 ? 'gap-ok' : (gap > 0 ? 'gap-o
```

```
  const empWeekHStr = empWeekH > 0 ? (Math.floor(empWeekH) + 'h' + (e
```

```
  return `
```

```
<div class="plan-cell emp">
```

```
<div class="emp-row">
```

```
<div class="av-emp sm">${initials(e)}</div>
```

```
<div class="emp-nm-wrap">
```

```
<div class="nm">${esc(e.prenom)}</div>
```

```
<div class="emp-stats">
```

```
<span class="chip-tiny">${contractH}h</span>
```

```
<span class="chip-tiny ${gapClass}">${empWeekHStr}</span>
```

```

        </div>
      </div>
    </div>
  </div>
  ${[0,1,2,3,4,5,6].map(d => {
    const dateD = addDays(state.weekStart, d);
    const ferD = holidayFor(dateD);
    const vacD = schoolHolidayFor(dateD);
    const shifts = (state.shifts[`${e.id}_${d}_${wk}`] || []).map(nor
return `<div class="plan-cell cell ${shifts.length?'has':''} ${fe
    ${shifts.map(s => renderShiftCell(s)).join('')}
    </div>`;
  }).join('')}
`)).join('')}
</div>
</div>
</div>
</div>

<div class="row" style="margin-top:12px; color:var(--c-ink-4); font-size:12px
  <span>Astuce :</span><span>clicquez sur une cellule pour ajouter ou modifier
</div>
`;
}

function renderShiftCell(s) {
  if (s.leaveType) {
    const lt = LEAVE_TYPES[s.leaveType] || { short: s.label || 'Absent', color: '
    return `<div class="shift ${lt.color}">${esc(lt.short)}</div>`;
  }
  const totalMin = Math.round(shiftHours(s) * 60);
  const h = Math.floor(totalMin / 60);
  const m = totalMin % 60;
  const durStr = h > 0 ? (m > 0 ? `${h}h${pad(m)}` : `${h}h`) : `${m}mn`;
  const label = s.label || (s.type==='midi'? 'Ouverture':s.type==='soir'? 'Fermeture'
  const pauseStr = s.pauseDuration ? ` (${s.pauseDuration}mn)` : '';
  return `
    <div class="shift ${s.type}">
      <div class="shift-title">${esc(label)}</div>
      <div class="shift-time">${s.start} - ${s.end}${pauseStr}</div>
      <div class="shift-dur">${durStr}</div>
    </div>
  `;
}

function bindPlanning() {
  $$('[data-week]').forEach(b => b.addEventListener('click', e => {

```

```

    const a = e.currentTarget.dataset.week;
    if (a === 'prev') state.weekStart = addDays(state.weekStart, -7);
    else if (a === 'next') state.weekStart = addDays(state.weekStart, 7);
    else state.weekStart = getMonday(new Date());
    render();
  }));
  $$('[data-edit]').forEach(c => c.addEventListener('click', e => {
    const [empId, dayIdx] = e.currentTarget.dataset.edit.split('_').map(Number);
    openShiftEditor(empId, dayIdx);
  }));

  const seedBtn = $('#seedDefault');
  if (seedBtn) seedBtn.addEventListener('click', () => seedPlanningFromTemplate());

  const dupBtn = $('#dupFromPrev');
  if (dupBtn) dupBtn.addEventListener('click', () => duplicatePreviousWeek());

  const clearBtn = $('#clearWeek');
  if (clearBtn) clearBtn.addEventListener('click', () => clearCurrentWeek());

  const pubBtn = $('#publishBtn');
  if (pubBtn) pubBtn.addEventListener('click', () => publishWeek());

  const unpubBtn = $('#unpublishBtn');
  if (unpubBtn) unpubBtn.addEventListener('click', () => unpublishWeek());

  const csvBtn = $('#exportCsv');
  if (csvBtn) csvBtn.addEventListener('click', () => exportPlanningCSV());

  const pdfBtn = $('#exportPdf');
  if (pdfBtn) pdfBtn.addEventListener('click', () => exportPlanningPDF());
}

function publishWeek() {
  const wk = weekKey(state.weekStart);
  const payload = { publishedAt: new Date().toISOString(), by: 'admin' };
  state.publications[wk] = payload;
  fbSave(`publications/${wk}`, payload);
  toast(`Planning publié – visible par les salariés`, 'good');
  render();
}

function unpublishWeek() {
  const wk = weekKey(state.weekStart);
  if (!confirm(`Repasser ce planning en brouillon ?\n\nIl ne sera plus visible pa
delete state.publications[wk];
  fbSave(`publications/${wk}`, null);

```



```

    toast('Repasé en brouillon', '');
    render();
}

```

```

function clearCurrentWeek() {
    const wk = weekKey(state.weekStart);
    const actives = state.employees.filter(e => e.statut === 'Actif');
    if (!confirm(`Vider tout le planning de la semaine du ${state.weekStart.toLocal
const updates = {};
actives.forEach(e => {
    for (let d = 0; d < 7; d++) {
        const k = `${e.id}_${d}_${wk}`;
        if (state.shifts[k]) {
            delete state.shifts[k];
            updates[k] = null;
        }
    }
});
if (db && Object.keys(updates).length) db.ref('shifts').update(updates).catch(e
toast('Semaine vidée', '');
render();
}

```

```

function seedPlanningFromTemplate() {
    const wk = weekKey(state.weekStart);
    const updates = {};
    let count = 0;
    Object.entries(SEED_SHIFTS).forEach(([k, v]) => {
        const [empId, dayIdx] = k.split('_');
        const emp = state.employees.find(e => String(e.id) === empId && e.statut ===
        if (!emp) return;
        const newKey = `${empId}_${dayIdx}_${wk}`;
        state.shifts[newKey] = v;
        updates[newKey] = v;
        count++;
    });
    if (db) db.ref('shifts').update(updates).catch(e => console.warn(e));
    toast(`${count} shifts chargés`, 'good');
    render();
}

```

```

function duplicatePreviousWeek() {
    const prevWk = weekKey(addDays(state.weekStart, -7));
    const curWk = weekKey(state.weekStart);
    const updates = {};
    let count = 0;
    state.employees.forEach(emp => {

```

```

    for (let d = 0; d < 7; d++) {
      const prevShifts = state.shifts[`${emp.id}_${d}_${prevWk}`];
      if (prevShifts && prevShifts.length) {
        const newKey = `${emp.id}_${d}_${curWk}`;
        state.shifts[newKey] = JSON.parse(JSON.stringify(prevShifts));
        updates[newKey] = state.shifts[newKey];
        count++;
      }
    }
  });
  if (db) db.ref('shifts').update(updates).catch(e => console.warn(e));
  toast(`${count} shifts dupliqués`, 'good');
  render();
}

function openShiftEditor(empId, dayIdx) {
  const wk = weekKey(state.weekStart);
  const key = `${empId}_${dayIdx}_${wk}`;
  const existing = (state.shifts[key] || []).map(normShift);
  const e = state.employees.find(x => x.id === empId);
  const sh = existing[0] || { type: 'soir', start: '17:00', end: '23:00', pauseDu
  const d = addDays(state.weekStart, dayIdx);
  const currentLeave = sh.leaveType || '';
  const initialDate = dateISO(d);

  const body = `
    <div class="uppercase-eyebrow" style="margin-bottom:4px;">${esc(e.prenom)} ${
    <div style="font-family:var(--f-display);font-size:24px;letter-spacing:-0.015

  <div class="form-grid">
    <div class="field full">
      <label class="field-label">Date</label>
      <input class="input" id="shDate" type="date" value="${initialDate}">
    </div>

    <div class="field full">
      <label class="field-label">Statut</label>
      <select class="input" id="shLeaveType">
        <option value="">- Travaille (shift normal) -</option>
        ${Object.entries(LEAVE_TYPES).map(([k,v]) => `<option value="${k}" ${cu
      </select>
    </div>

    <div class="field full" id="shRangeWrap" ${currentLeave?'':'style="display:
      <label class="field-label">Date de fin (optionnel, pour étaler sur plusie
      <input class="input" id="shDateEnd" type="date" value="${initialDate}">
      <div class="text-mute" style="font-size:11.5px;margin-top:6px;">Laisse id

```

```

    </div>
</div>

<div id="shFieldsWrap" ${currentLeave?'style="display:none;":""}>
  <div class="form-grid" style="margin-top:14px;">
    <div class="field full">
      <label class="field-label">Type de service</label>
      <div class="tab-switch" id="shType" data-active="${sh.type==='midi'?'0'
        <button type="button" data-t="midi" class="${sh.type==='midi'?'active'
        <button type="button" data-t="soir" class="${sh.type==='soir'?'active'
        <button type="button" data-t="ar" class="${sh.type==='ar'?'active':''
      </div>
    </div>
    <div class="field">
      <label class="field-label">Début</label>
      <input class="input mono" id="shStart" type="time" value="${sh.start ||
    </div>
    <div class="field">
      <label class="field-label">Fin</label>
      <input class="input mono" id="shEnd" type="time" value="${sh.end || '23
    </div>
    <div class="field">
      <label class="field-label">Pause (mn)</label>
      <input class="input mono" id="shPauseDur" type="number" min="0" step="1
    </div>
    <div class="field">
      <label class="field-label">Pause début</label>
      <input class="input mono" id="shPauseStart" type="time" value="${sh.pau
    </div>
    <div class="field full">
      <label class="field-label">Pause fin</label>
      <input class="input mono" id="shPauseEnd" type="time" value="${sh.pause
    </div>
    <div class="field full">
      <label class="field-label">Libellé (optionnel)</label>
      <input class="input" id="shLabel" placeholder="ex: Cuisine, Salle, Renf
    </div>
  </div>
</div>
`;

const footer = `
  ${existing.length ? `<button class="btn-danger" id="shDel">Supprimer</button>
  <div class="spacer"></div>
  <button class="btn-sec" data-close>Annuler</button>
  <button class="btn-pri" id="shSave" style="width:auto;">Enregistrer</button>
  `;

```

```

const { close } = openModal({ title: 'Édition du jour', body, footer });

let chosenType = sh.type || 'soir';
$$('#shType [data-t]').forEach(b => b.addEventListener('click', e => {
  chosenType = e.target.dataset.t;
  $$('#shType [data-t]').forEach(x => x.classList.toggle('active', x === e.targ
  const idx = chosenType === 'midi' ? 0 : chosenType === 'soir' ? 1 : 2;
  $('#shType').setAttribute('data-active', idx);
}));

$('#shLeaveType').addEventListener('change', ev => {
  const isLeave = !!ev.target.value;
  $('#shFieldsWrap').style.display = isLeave ? 'none' : '';
  $('#shRangeWrap').style.display = isLeave ? '' : 'none';
  if (isLeave) $('#shDateEnd').value = $('#shDate').value;
});

// Sync end date with start when start changes (default to same day)
$('#shDate').addEventListener('change', ev => {
  const endVal = $('#shDateEnd').value;
  if (!endVal || endVal < ev.target.value) {
    $('#shDateEnd').value = ev.target.value;
  }
});

$('#shSave').addEventListener('click', () => {
  const leave = $('#shLeaveType').value;
  const newDateISO = $('#shDate').value;
  if (!newDateISO) { toast('Date invalide', 'error'); return; }

  let newSh;
  if (leave) {
    newSh = { leaveType: leave, label: LEAVE_TYPES[leave].label };
  } else {
    newSh = {
      type: chosenType,
      start: $('#shStart').value,
      end: $('#shEnd').value,
      pauseDuration: parseInt($('#shPauseDur').value) || 0,
    };
    const ps = $('#shPauseStart').value;
    const pe = $('#shPauseEnd').value;
    const lbl = $('#shLabel').value.trim();
    if (ps) newSh.pauseStart = ps;
    if (pe) newSh.pauseEnd = pe;
    if (lbl) newSh.label = lbl;
  }

```

```

}

// Delete original entry if date changed (we move it)
const dateMoved = newDateISO !== initialDate;
if (dateMoved || existing.length) {
  delete state.shifts[key];
  fbSave(`shifts/${key}`, null);
}

// Determine days to save (range support for leaves)
const endDateISO = leave ? ($('#shDateEnd').value || newDateISO) : newDateISO
if (endDateISO < newDateISO) { toast('La date de fin doit être ≥ date de débu

const daysToSave = [];
let cur = new Date(newDateISO + 'T00:00:00');
const end = new Date(endDateISO + 'T00:00:00');
while (cur <= end) {
  daysToSave.push(new Date(cur));
  cur = addDays(cur, 1);
}

daysToSave.forEach(targetDate => {
  const targetMon = getMonday(targetDate);
  const targetWk = weekKey(targetMon);
  const targetDayIdx = (targetDate.getDay() + 6) % 7;
  const newKey = `${empId}_${targetDayIdx}_${targetWk}`;
  state.shifts[newKey] = [newSh];
  fbSave(`shifts/${newKey}`, [newSh]);
});

// If we moved to a different week, jump there to show the result
if (dateMoved) {
  state.weekStart = getMonday(new Date(newDateISO + 'T00:00:00'));
}

const msg = daysToSave.length > 1 ? `${daysToSave.length} jours enregistrés`
toast(msg, 'good');
close();
render();
});

if (existing.length) {
  $('#shDel').addEventListener('click', () => {
    if (!confirm('Supprimer ce shift ?')) return;
    delete state.shifts[key];
    fbSave(`shifts/${key}`, null);
    toast('Supprimé', '');
  });
}

```

```

        close();
        render();
    });
}
}

// ----- MONTH VIEW -----
function pageMonth() {
    const anchor = state.monthAnchor;
    const year = anchor.getFullYear();
    const month = anchor.getMonth();
    const monthName = MONTHS_FR[month];
    const firstDay = new Date(year, month, 1);
    const lastDay = new Date(year, month+1, 0);
    // First Monday on or before firstDay
    const gridStart = getMonday(firstDay);
    // Last Sunday on or after lastDay
    const lastDayOfWeek = (lastDay.getDay() + 6) % 7;
    const gridEnd = addDays(lastDay, 6 - lastDayOfWeek);
    const totalDays = Math.round((gridEnd - gridStart) / 86400000) + 1;

    const actives = state.employees.filter(e => e.statut === 'Actif');
    const todayISO = dateISO(new Date());

    // Aggregate per day
    const daysData = [];
    let monthTotalH = 0;
    for (let i = 0; i < totalDays; i++) {
        const d = addDays(gridStart, i);
        const wk = weekKey(d);
        const dayIdx = (d.getDay() + 6) % 7;
        const inMonth = d.getMonth() === month;
        let dayH = 0, workers = 0, leaves = [];
        actives.forEach(e => {
            const shifts = (state.shifts[`${e.id}_${dayIdx}_${wk}`] || []).map(normShif
            shifts.forEach(s => {
                if (s.leaveType) {
                    leaves.push({emp:e, leaveType:s.leaveType});
                } else {
                    dayH += shiftHours(s);
                    workers++;
                }
            });
        });
        if (inMonth) monthTotalH += dayH;
        daysData.push({date:d, inMonth, dayH, workers, leaves});
    }

```

```

return `
  <div class="page-head">
    <div>
      <div class="uppercase-eyebrow">Vue mois</div>
      <h1 class="h-1" style="text-transform:capitalize;">${monthName} ${year}</
      <div class="chip" style="margin-top:6px;">${Math.round(monthTotalH)} h pl
    </div>
    <div class="page-actions">
      <div class="week-nav">
        <button class="btn-icon" data-mnav="prev">⏪</button>
        <button class="btn-sec" data-mnav="today">Ce mois</button>
        <button class="btn-icon" data-mnav="next">⏩</button>
      </div>
    </div>
  </div>

  <div class="panel">
    <div class="panel-body nopad">
      <div class="month-grid">
        ${DAYS_SHORT.map(d => `<div class="mc head"><div class="day">${d}</div>
        ${daysData.map(d => {
          const isToday = dateISO(d.date) === todayISO;
          const fer = holidayFor(d.date);
          return `
            <div class="mc ${d.inMonth?'':'out'} ${isToday?'today':''} ${fer?'f
              <div class="mc-date ${isToday?'today':''}">${d.date.getDate()}</d
              ${fer ? `<div class="mc-ferie" title="${esc(fer.long)}">${esc(fer
                ${d.workers > 0 ? `<div class="mc-stat"><strong>${d.dayH.toFixed(
                  ${d.leaves.length > 0 ? `<div class="mc-leaves">${d.leaves.map(1
                </div>
              `;
            }).join('')}
          </div>
        </div>
      </div>
    </div>

    <div class="row" style="margin-top:14px;gap:14px;flex-wrap:wrap;font-size:12p
      <span><span class="dot-color" style="background:#dbeafe;"></span> CP</span>
      <span><span class="dot-color" style="background:#fef3c7;"></span> Absent ju
      <span><span class="dot-color" style="background:#fee2e2;"></span> Absent in
      <span><span class="dot-color" style="background:#ede9fe;"></span> Arrêt mal
      <span><span class="dot-color" style="background:#cffafe;"></span> RTT</span>
    </div>
  `;
}

```

```

function bindMonth() {
  $$('[data-mnav]').forEach(b => b.addEventListener('click', e => {
    const a = e.currentTarget.dataset.mnav;
    if (a === 'prev') state.monthAnchor = new Date(state.monthAnchor.getFullYear(
    else if (a === 'next') state.monthAnchor = new Date(state.monthAnchor.getFull
    else state.monthAnchor = new Date();
    render();
  }));
  $$('[data-mday]').forEach(c => c.addEventListener('click', e => {
    const iso = e.currentTarget.dataset.mday;
    state.weekStart = getMonday(new Date(iso));
    state.page = 'planning';
    render();
  }));
}

// _____ HOURS TRACKING _____
// _____ HOURS TRACKING - VUE COMPTA _____
function pageHours() {
  const anchor = state.monthAnchor || new Date();
  const year = anchor.getFullYear();
  const month = anchor.getMonth();
  const actives = state.employees.filter(e => e.statut === 'Actif');
  const monthStart = new Date(year, month, 1);
  const monthEnd = new Date(year, month+1, 0);

  // Build detailed data for each employee
  const data = actives.map(emp => {
    const n = normEmp(emp);
    let plannedTotal = 0, realTotal = 0;
    let supplH25 = 0, supplH50 = 0, normalH = 0;
    const leaves = { cp: 0, absent_justifie: 0, absent_injustifie: 0, arret_malad
    let workedDays = 0;
    let ferieWorked = 0; // jours fériés travaillés

    // Iterate week by week to compute heures sup correctly
    const firstMonday = getMonday(monthStart);
    for (let wkDate = new Date(firstMonday); wkDate <= monthEnd; wkDate = addDays
      const wk = weekKey(wkDate);
      let wkPlanned = 0;
      for (let d = 0; d < 7; d++) {
        const dayDate = addDays(wkDate, d);
        if (dayDate.getMonth() !== month) continue; // only count days in this mo
        const shifts = (state.shifts[`${emp.id}_${d}_${wk}`] || []).map(normShift
        const fer = holidayFor(dayDate);
        shifts.forEach(s => {
          if (s.leaveType) {

```



```

        leaves[s.leaveType] = (leaves[s.leaveType] || 0) + 1;
    } else {
        const h = shiftHours(s);
        wkPlanned += h;
        plannedTotal += h;
        if (h > 0) workedDays++;
        if (fer && h > 0) ferieWorked++;
    }
});
// Real hours from punches
const pks = state.punches[`${emp.id}_${dateISO(dayDate)}`] || [];
pks.forEach(p => {
    if (p.in && p.out) {
        let dur = timeToMin(p.out) - timeToMin(p.in);
        if (dur < 0) dur += 24*60;
        realTotal += dur / 60;
    }
});
}
// Compute heures sup for this week based on contract
const contractH = n.heures || 35;
if (n.sansHeuresSup) {
    // Toutes les heures comptées comme normales (forfait, accord spécifique)
    normalH += wkPlanned;
} else {
    const wkNormal = Math.min(wkPlanned, contractH);
    const wkSupp25 = Math.min(Math.max(wkPlanned - contractH, 0), 8);
    const wkSupp50 = Math.max(wkPlanned - contractH - 8, 0);
    normalH += wkNormal;
    supplH25 += wkSupp25;
    supplH50 += wkSupp50;
}
}

const taux = n.taux || 12;
const salaireNormal = normalH * taux;
const salaireS25 = supplH25 * taux * 1.25;
const salaireS50 = supplH50 * taux * 1.50;
const totalBrutPlanifie = salaireNormal + salaireS25 + salaireS50;
const navigoMensuel = n.sansNavigo ? 0 : (n.navigoMensuel || 0);

// HCR - repas et coupures (calcul auto sur les shifts du mois)
const hcr = n.sansAvantageRepas
    ? { repas: 0, valeurRepas: 0, coupures: hcrMensuelEmp(emp.id, year, month).
        : hcrMensuelEmp(emp.id, year, month);

// Pourboires manuels (stockés dans state.pourboires[`${empId}_${year}-${mont

```

```

const moisKey = `${emp.id}_${year}-${pad(month+1)}`;
const pourboires = (state.pourboires && state.pourboires[moisKey]) || 0;

// 8 dernières semaines (heures planifiées) pour mini-graphique
const sparkWeeks = [];
for (let i = 7; i >= 0; i--) {
  const w = addDays(getMonday(new Date()), -7 * i);
  const wk = weekKey(w);
  let h = 0;
  for (let d = 0; d < 7; d++) {
    (state.shifts[`${emp.id}_${d}_${wk}`] || []).map(normShift).forEach(s =>
    )
    sparkWeeks.push({ date: w, hours: h });
  }
}

return {
  emp, n,
  plannedTotal, realTotal,
  normalH, supplH25, supplH50,
  salaireNormal, salaireS25, salaireS50, totalBrutPlanifie,
  leaves, workedDays, ferieWorked,
  navigoMensuel,
  repas: hcr.repas, valeurRepas: hcr.valeurRepas, coupures: hcr.coupures,
  pourboires, sparkWeeks,
};
}).filter(d => !d.n.exclurePaie);

// Team-wide totals for KPI cards
const teamTotalBrut = data.reduce((s,d) => s + d.totalBrutPlanifie, 0);
const teamTotalH = data.reduce((s,d) => s + d.normalH + d.supplH25 + d.supplH50
const teamSuppH = data.reduce((s,d) => s + d.supplH25 + d.supplH50, 0);
const teamCP = data.reduce((s,d) => s + d.leaves.cp, 0);

return `
<div class="page-head">
  <div>
    <div class="uppercase-eyebrow">Heures</div>
    <h1 class="h-1">Préparation paie</h1>
    <div class="text-mute" style="margin-top:6px;font-size:13px;">Récapitulat
  </div>
  <div class="page-actions">
    <div class="week-nav">
      <button class="btn-icon" data-hnav="prev" title="Mois précédent"></but
      <span class="week-label" style="text-transform:capitalize;min-width:170
      <button class="btn-icon" data-hnav="next" title="Mois suivant"></butto
      <button class="btn-sec" data-hnav="today">Mois en cours</button>
    </div>

```

```

    </div>
</div>

<div class="kpi-grid" style="grid-template-columns:repeat(4,1fr);">
  <div class="kpi">
    <div class="kpi-label">Masse salariale brute</div>
    <div class="kpi-value">${Math.round(teamTotalBrut).toLocaleString('fr-FR')}
    <div class="kpi-meta">Estimation pour ${MONTHS_FR[month]}</div>
  </div>
  <div class="kpi">
    <div class="kpi-label">Total heures</div>
    <div class="kpi-value">${Math.round(teamTotalH)}<span style="font-size:18px">
    <div class="kpi-meta">${data.length} salarié${data.length>1?'s':''}</div>
  </div>
  <div class="kpi ${teamSuppH > 0 ? 'warn' : ''}">
    <div class="kpi-label">Heures supplémentaires</div>
    <div class="kpi-value">${teamSuppH.toFixed(1)}<span style="font-size:18px">
    <div class="kpi-meta">au-delà du contrat</div>
  </div>
  <div class="kpi">
    <div class="kpi-label">Jours de CP pris</div>
    <div class="kpi-value">${teamCP}</div>
    <div class="kpi-meta">cumul équipe</div>
  </div>
</div>

<div class="row" style="margin-bottom:18px;gap:8px;flex-wrap:wrap;">
  <button class="btn-pri" id="exportPaieCsv" style="width:auto;">↓ Export paie
  <button class="btn-sec" id="exportPaieDetaillée">↓ Export détaillé</button>
  <div class="spacer"></div>
  <button class="btn-ghost" id="toggleHoursView" data-view="${state.hoursView}
    ${((state.hoursView || 'cards') === 'cards' ? '≡ Vue tableau' : '☐ Vue car
  </button>
</div>

${((state.hoursView || 'cards') === 'cards' ? `
  <div class="pcards">
    ${data.map(d => {
      const totalAbs = (d.leaves.absent_justifie||0) + (d.leaves.absent_injus
      const totalH = d.normalH + d.supplH25 + d.supplH50;
      const mods = [];
      if (d.n.sansHeuresSupp) mods.push('Sans heures sup');
      if (d.n.payeNet) mods.push('Payé net');
      if (d.n.exclureCP) mods.push('Sans CP');
      if (d.n.forfaitJours) mods.push('Forfait jours');
      if (d.n.sansAvantageRepas) mods.push('Sans repas');
      if (d.n.sansNavigo) mods.push('Sans Navigo');
    })}
  </div>
`

```

```

return `
<article class="pc">
  <div class="pc-head">
    <div class="pc-who">
      <div class="av-emp sm">${initials(d.emp)}</div>
      <div>
        <div class="pc-name">${esc(d.emp.prenom)} ${esc(d.emp.nom)}</div>
        <div class="pc-sub">${esc(d.emp.poste||'-')} · ${esc(d.emp.co
      </div>
    </div>
    <button class="btn-ghost" data-detail-emp="${d.emp.id}" title="Vo
  </div>

  <div class="pc-stats">
    <div class="pc-stat">
      <div class="pc-stat-n">${d.normalH.toFixed(1)}<span>h</span></div>
      <div class="pc-stat-l">Heures normales</div>
    </div>
    <div class="pc-stat">
      <div class="pc-stat-n ${d.supplH25>0?'c-amber':'c-mute'}">${d.s
      <div class="pc-stat-l">Sup. +25%</div>
    </div>
    <div class="pc-stat">
      <div class="pc-stat-n ${d.supplH50>0?'c-red':'c-mute'}">${d.sup
      <div class="pc-stat-l">Sup. +50%</div>
    </div>
    <div class="pc-stat">
      <div class="pc-stat-n">${totalH.toFixed(1)}<span>h</span></div>
      <div class="pc-stat-l">Total heures</div>
    </div>
    <div class="pc-stat">
      <div class="pc-stat-n">${d.repas}<span>repas · ${d.valeurRepas.
      <div class="pc-stat-l">Avantage repas</div>
    </div>
    <div class="pc-stat pc-stat-key">
      <div class="pc-stat-n c-green">${Math.round(d.totalBrutPlanifie
      <div class="pc-stat-l">Brut estimé</div>
    </div>
  </div>

  <div class="pc-info">
    <span>Coupures <b class="${d.coupures>0?'c-amber':''}">${d.coupur
    <span>Navigo <b>${d.navigoMensuel?d.navigoMensuel.toFixed(0)+'€':
    <span>CP <b>${d.leaves.cp||'0'}j</b></span>
    <span>Arrêt maladie <b class="${d.leaves.arret_maladie>0?'c-red':
    <span>Absences <b class="${totalAbs>0?'c-red':''}">${totalAbs||'0
    <span>Fériés trav. <b class="${d.ferieWorked>0?'c-green':''}">${d

```

```

    <span class="pc-tip">Pourboires <input type="number" data-tip="${
</div>

<div class="pc-weeks">
  <div class="pc-weeks-1">8 dernières semaines · contrat ${d.n.heur
  <div class="pc-weeks-row">
    ${d.sparkWeeks.map(w => {
      const gap = w.hours - (d.n.heures||35);
      const cls = w.hours === 0 ? 'empty' : Math.abs(gap) < 1 ? 'ok
      return `<div class="pc-wk"><div class="pc-wk-n ${cls}">${w.ho
    }).join('')}
  </div>
</div>
</article>
`;
}).join('')}

${data.length > 0 ? `
  <article class="pc pc-total">
    <div class="pc-head">
      <div class="pc-who">
        <div>
          <div class="pc-name">Total équipe</div>
          <div class="pc-sub">${data.length} salariés · ${MONTHS_FR[month
        </div>
      </div>
    </div>
    <div class="pc-stats">
      <div class="pc-stat">
        <div class="pc-stat-n">${teamTotalH.toFixed(0)}<span>h</span></di
        <div class="pc-stat-l">Heures totales</div>
      </div>
      <div class="pc-stat">
        <div class="pc-stat-n ${teamSupH>0?'c-amber':'c-mute'}">${teamSu
        <div class="pc-stat-l">Heures sup.</div>
      </div>
      <div class="pc-stat">
        <div class="pc-stat-n">${data.reduce((s,d)=>s+d.valeurRepas,0).to
        <div class="pc-stat-l">Repas</div>
      </div>
      <div class="pc-stat">
        <div class="pc-stat-n">${data.reduce((s,d)=>s+(d.pourboires||0),0
        <div class="pc-stat-l">Pourboires</div>
      </div>
      <div class="pc-stat">
        <div class="pc-stat-n">${data.reduce((s,d)=>s+(d.navigoMensuel||0
        <div class="pc-stat-l">Navigo</div>

```

```

    </div>
    <div class="pc-stat pc-stat-key">
      <div class="pc-stat-n c-green">${Math.round(teamTotalBrut).toLocaleString()}
      <div class="pc-stat-l">Masse brute</div>
    </div>
  </div>
</article>
` : ''}
</div>
` : `
<!-- Vue tableau comtable, pour export visuel -->
<div class="panel">
  <div class="panel-head">
    <h3>Vue comtable détaillée</h3>
    <span class="text-mute" style="font-size:11.5px;">HCR · sup +25% (36→43)
  </div>
  <div class="panel-body nopad" style="overflow-x:auto;">
    <table class="tbl tbl-compact">
      <thead>
        <tr>
          <th>Salarié</th>
          <th>Contrat</th>
          <th>Taux €/h</th>
          <th>H. norm.</th>
          <th>Sup +25%</th>
          <th>Sup +50%</th>
          <th>Total H</th>
          <th>Brut €</th>
          <th>Repas</th>
          <th>Coupures</th>
          <th>Pourboires</th>
          <th>Navigo</th>
          <th>CP</th>
          <th>AM</th>
          <th>Abs.</th>
          <th>Fériés</th>
        </tr>
      </thead>
      <tbody>
        ${data.map(d => {
          const totalAbs = (d.leaves.absent_justifie||0) + (d.leaves.absent_justifie||0)
          return `
            <tr>
              <td>
                <div class="with-av">
                  <div class="av-emp sm">${initials(d.emp)}</div>
                  <div class="stack">

```

```

        <div style="font-weight:600;">${esc(d.emp.prenom)} ${es
        <div class="sub" style="text-decoration:none;cursor:def
    </div>
</div>
</td>
<td><span class="chip">${esc(d.emp.contrat||'-' )} · ${d.n.heu
<td class="mono tabular">${d.n.taux ? d.n.taux.toFixed(2)+' €
<td class="mono tabular"><strong>${d.normalH.toFixed(1)} h</s
<td class="mono tabular ${d.supplH25>0?'over':''}">${d.supplH
<td class="mono tabular ${d.supplH50>0?'over':''}">${d.supplH
<td class="mono tabular"><strong>${(d.normalH+d.supplH25+d.su
<td class="mono tabular"><strong>${d.totalBrutPlanifie.toFixe
<td class="mono tabular" title="${d.repas} repas × ${REPAS_HC
<td class="mono tabular ${d.coupures > 0 ? 'over' : ''}">${d.
<td class="mono tabular"><input type="number" class="input mi
<td class="mono tabular">${d.navigoMensuel ? d.navigoMensuel.
<td class="mono tabular">${d.leaves.cp || '-'}</td>
<td class="mono tabular">${d.leaves.arret_maladie || '-'}</td>
<td class="mono tabular">${totalAbs || '-'}</td>
<td class="mono tabular">${d.ferieWorked || '-'}</td>
</tr>
`
    }).join('')
    ${data.length > 0 ? `
    <tr class="row-total">
        <td><strong style="font-family:var(--f-display);font-size:15px;
        <td>—</td>
        <td>—</td>
        <td class="mono tabular">${data.reduce((s,d) => s + d.normalH,
        <td class="mono tabular">${data.reduce((s,d) => s + d.supplH25,
        <td class="mono tabular">${data.reduce((s,d) => s + d.supplH50,
        <td class="mono tabular"><strong>${teamTotalH.toFixed(1)} h</st
        <td class="mono tabular"><strong>${teamTotalBrut.toFixed(2)} €<
        <td class="mono tabular"><strong>${data.reduce((s,d) => s + d.v
        <td class="mono tabular">${data.reduce((s,d) => s + d.coupures,
        <td class="mono tabular"><strong>${data.reduce((s,d) => s + (d.
        <td class="mono tabular">${data.reduce((s,d) => s + (d.navigoMe
        <td class="mono tabular">${data.reduce((s,d) => s + d.leaves.cp
        <td class="mono tabular">${data.reduce((s,d) => s + d.leaves.ar
        <td class="mono tabular">${data.reduce((s,d) => s + (d.leaves.a
        <td class="mono tabular">${data.reduce((s,d) => s + d.ferieWork
    </tr>
    ` : ''}
</tbody>
</table>
</div>
</div>

```

```

    `}

    <div class="row text-mute" style="margin-top:16px;font-size:11.5px;gap:14px;f
      <span><strong>Brut estimé</strong> = (H. normales × taux) + (H. sup 25% × t
      <span><strong>Hors</strong> : primes, ancienneté, mutuelle, charges patrona
    </div>
  `;
}

```

```

function bindHours() {
  $$('[data-hnav]').forEach(b => b.addEventListener('click', e => {
    const a = e.currentTarget.dataset.hnav;
    if (!state.monthAnchor) state.monthAnchor = new Date();
    if (a === 'prev') state.monthAnchor = new Date(state.monthAnchor.getFullYear(
    else if (a === 'next') state.monthAnchor = new Date(state.monthAnchor.getFull
    else state.monthAnchor = new Date();
    render();
  }));
  $$('[data-detail-emp]').forEach(b => b.addEventListener('click', e => {
    state.empDetail = parseInt(e.currentTarget.dataset.detailEmp);
    state.empTab = 'temps';
    state.page = 'employees';
    render();
  }));
  const toggleBtn = $('#toggleHoursView');
  if (toggleBtn) toggleBtn.addEventListener('click', () => {
    state.hoursView = (state.hoursView || 'cards') === 'cards' ? 'table' : 'cards
    render();
  });
  // Pourboires inputs - save on blur
  $$('[data-tip]').forEach(inp => {
    inp.addEventListener('change', ev => {
      const empId = parseInt(ev.currentTarget.dataset.tip);
      const val = parseFloat(ev.currentTarget.value) || 0;
      const anchor = state.monthAnchor || new Date();
      const moisKey = `${empId}_${anchor.getFullYear()}-${pad(anchor.getMonth()+1
      state.pourboires = state.pourboires || {};
      if (val > 0) {
        state.pourboires[moisKey] = val;
        if (db) db.ref(`pourboires/${moisKey}`).set(val);
      } else {
        delete state.pourboires[moisKey];
        if (db) db.ref(`pourboires/${moisKey}`).remove();
      }
      toast('Pourboires enregistrés', 'good');
    });
  });
}

```



```

const exp1 = $('#exportPaieCsv');
if (exp1) exp1.addEventListener('click', () => exportPaieCSV(false));
const exp2 = $('#exportPaieDetaillée');
if (exp2) exp2.addEventListener('click', () => exportPaieCSV(true));
}

function exportPaieCSV(detailed) {
  const anchor = state.monthAnchor || new Date();
  const year = anchor.getFullYear();
  const month = anchor.getMonth();
  const actives = state.employees.filter(e => e.statut === 'Actif');
  const monthEnd = new Date(year, month+1, 0);
  const firstMonday = getMonday(new Date(year, month, 1));

  const lines = [];
  if (detailed) {
    lines.push('Salarié;Poste;Type contrat;H. contrat hebdo;H. contrat mensuel;Ta
  } else {
    lines.push('Salarié;Type contrat;H. contrat;Taux €/h;H. normales;H. sup +25%;
  }

  actives.forEach(emp => {
    const n = normEmp(emp);
    let plannedTotal = 0, normalH = 0, supplH25 = 0, supplH50 = 0;
    const leaves = { cp: 0, absent_justifie: 0, absent_injustifie: 0, arret_malad
    let ferieWorked = 0;
    for (let wkDate = new Date(firstMonday); wkDate <= monthEnd; wkDate = addDays
      const wk = weekKey(wkDate);
      let wkPlanned = 0;
      for (let d = 0; d < 7; d++) {
        const dayDate = addDays(wkDate, d);
        if (dayDate.getMonth() !== month) continue;
        const shifts = (state.shifts[`${emp.id}_${d}_${wk}`] || []).map(normShift
        const fer = holidayFor(dayDate);
        shifts.forEach(s => {
          if (s.leaveType) leaves[s.leaveType] = (leaves[s.leaveType] || 0) + 1;
          else {
            const h = shiftHours(s);
            wkPlanned += h; plannedTotal += h;
            if (fer && h > 0) ferieWorked++;
          }
        });
      }

      const contractH = n.heures || 35;
      normalH += Math.min(wkPlanned, contractH);
      supplH25 += Math.min(Math.max(wkPlanned - contractH, 0), 8);
      supplH50 += Math.max(wkPlanned - contractH - 8, 0);

```

```

}
const taux = n.taux || 12;
const contractMonthly = (n.heures||35) * 52 / 12;
const brutN = normalH * taux;
const brutS25 = supplH25 * taux * 1.25;
const brutS50 = supplH50 * taux * 1.50;
const total = brutN + brutS25 + brutS50;
const fr = v => String(v.toFixed(2)).replace('.', ',');
const fr1 = v => String(v.toFixed(1)).replace('.', ',');

if (detailed) {
  lines.push([
    csvEsc(`${emp.prenom} ${emp.nom}`),
    csvEsc(emp.poste||''),
    csvEsc(emp.contrat||''),
    n.heures||35,
    fr1(contractMonthly),
    fr(taux),
    fr1(normalH),
    fr1(supplH25),
    fr1(supplH50),
    fr1(normalH+supplH25+supplH50),
    fr(brutN),
    fr(brutS25),
    fr(brutS50),
    fr(total),
    fr(n.navigoMensuel||0),
    leaves.cp,
    leaves.arret_maladie,
    leaves.absent_justifie,
    leaves.absent_injustifie,
    leaves.rtt,
    leaves.recup,
    ferieWorked,
  ]).join(';');
} else {
  lines.push([
    csvEsc(`${emp.prenom} ${emp.nom}`),
    csvEsc(emp.contrat||''),
    `${n.heures||35}h`,
    fr(taux),
    fr1(normalH),
    fr1(supplH25),
    fr1(supplH50),
    fr1(normalH+supplH25+supplH50),
    fr(total),
    fr(n.navigoMensuel||0),
  ]

```

```

    leaves.cp,
    leaves.arret_maladie,
  ].join(';'));
}
});

const csv = lines.join('\r\n');
const blob = new Blob(['\uffeff' + csv], { type: 'text/csv;charset=utf-8' });
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = `paie_${MONTHS_FR[month]}_${year}${detailed? '_detail':''}.csv`;
document.body.appendChild(a); a.click(); document.body.removeChild(a);
setTimeout(() => URL.revokeObjectURL(url), 1000);
toast('CSV téléchargé', 'good');
}

// ----- POINTAGES -----
function pagePointages() {
  // Show last 14 days
  const days = [];
  for (let i = 13; i >= 0; i--) days.push(addDays(new Date(), -i));
  const actives = state.employees.filter(e => e.statut === 'Actif');

  return `
    <div class="page-head">
      <div>
        <div class="uppercase-eyebrow">Pointages</div>
        <h1 class="h-1">14 derniers jours</h1>
      </div>
    </div>

    <div class="panel">
      <div class="panel-body nopad">
        <table class="tbl">
          <thead><tr><th>Date</th><th>Salarié</th><th>Entrée</th><th>Sortie</th></tr>
          <tbody>
            ${days.flatMap(d => {
              const iso = dateISO(d);
              const dayIdx = (d.getDay() + 6) % 7;
              const wk = weekKey(d);
              return actives.flatMap(e => {
                const punches = state.punches[`${e.id}_${iso}`] || [];
                const planned = state.shifts[`${e.id}_${dayIdx}_${wk}`] || [];
                if (!punches.length && !planned.length) return [];
                if (!punches.length) {
                  return [ `<tr>

```

```

        <td class="mono">${pad(d.getDate())}/${pad(d.getMonth()+1)}</
        <td><div class="emp-cell"><div class="av-emp sm">${initials(e
        <td class="text-dim"></td><td class="text-dim"></td><td cla
        <td><span class="chip alert">Pas de pointage</span></td>
      </tr>`;
    }
    return punches.map(p => `
      <tr>
        <td class="mono">${pad(d.getDate())}/${pad(d.getMonth()+1)}</
        <td><div class="emp-cell"><div class="av-emp sm">${initials(e
        <td class="mono">${p.in || '-'}</td>
        <td class="mono">${p.out || '<span class="text-dim">en cours<
        <td class="mono tabular">${p.in && p.out ? ((timeToMin(p.out)
        <td>${p.in && p.out ? '<span class="chip good">Complet</span>
      </tr>
    `);
  });
  }).join('') || `<tr><td colspan="6"><div class="empty"><h4>Aucun poin
</tbody>
</table>
</div>
</div>
`;
}
function bindPointages() {}

// _____ ALERTS _____
function pageAlerts() {
  const all = detectAnomalies();
  return `
    <div class="page-head">
      <div>
        <div class="uppercase-eyebrow">Surveillance</div>
        <h1 class="h-1">Anomalies de pointage</h1>
      </div>
      <div class="page-actions">
        <span class="chip ${all.length?'alert':'good'}">${all.length} ${all.lengt
      </div>
    </div>

    <div class="panel">
      <div class="panel-body nopad">
        ${all.length === 0
        ? `<div class="empty"><h4>Tout est en ordre</h4><div>Aucune anomalie dé
        : all.map(a => renderAlertRow(a)).join('')`
        </div>
      </div>
    </div>
  `;
}

```

```

<div class="panel" style="margin-top:16px;">
  <div class="panel-head"><h3>Règles de détection</h3></div>
  <div class="panel-body" style="font-size:13px;line-height:1.7;color:var(--c-ink)">
    <div><strong style="color:var(--c-ink);">Oubli de pointage</strong> · pla
    <div><strong style="color:var(--c-ink);">Pointage hors planning</strong>
    <div><strong style="color:var(--c-ink);">Pointage incomplet</strong> · en
    <div><strong style="color:var(--c-ink);">Écart contrat/réel</strong> · di
    <div><strong style="color:var(--c-ink);">Heures sup. non déclarées</strong>
  </div>
</div>
`;
}
function bindAlerts() {}

// _____ EMPLOYEES _____
// _____ EMPLOYEES - LIST _____
function pageEmployees() {
  const sorted = [...state.employees].sort((a,b) => {
    if (a.statut !== b.statut) return a.statut === 'Actif' ? -1 : 1;
    return (a.prenom + a.nom).localeCompare(b.prenom + b.nom);
  });
  const actifs = sorted.filter(e => e.statut === 'Actif').length;
  return `
    <div class="page-head">
      <div>
        <div class="uppercase-eyebrow">Équipe</div>
        <h1 class="h-1">Salariés</h1>
        <div class="text-mute" style="margin-top:6px;font-size:13px;">${actifs} a
      </div>
      <div class="page-actions">
        <button class="btn-pri" id="addEmp" style="width:auto;">+ Ajouter un coll
      </div>
    </div>

    <div class="panel">
      <div class="panel-body nopad">
        <table class="tbl">
          <thead><tr>
            <th>Collaborateur</th>
            <th>Rôle</th>
            <th>Contrat</th>
            <th>Email</th>
            <th>Mobile</th>
            <th>Statut</th>
            <th></th>
          </tr></thead>

```

```

<tbody>
  ${sorted.map(e => {
    const n = normEmp(e);
    const complete = isProfileComplete(n);
    const inactif = e.statut !== 'Actif';
    const poste = e.poste || 'Non renseigné';
    const posteChipClass = !e.poste ? '' : (e.poste.toLowerCase()).inclu
    return `
    <tr class="row-clickable" data-view-emp="${e.id}">
      <td>
        <div class="with-av">
          <div class="av-emp">${initials(e)}</div>
          <div class="stack">
            <div style="font-weight:600;">${esc(e.prenom)} ${esc(e.nom)}
            <div class="sub">${e.poste ? esc(e.poste) : 'Non renseigné'}
          </div>
        </div>
      </td>
      <td><span class="chip ${posteChipClass}" style="font-size:11.5px;
      <td>
        <div class="stack">
          <span style="font-weight:500;">${esc(e.contrat || '-')}</span>
          <span class="sub" style="text-decoration:none;cursor:default;
        </div>
      </td>
      <td style="font-size:12.5px;color:var(--c-ink-3);">${esc(n.email)}
      <td style="font-size:12.5px;color:var(--c-ink-3);">${esc(n.telMob)}
      <td>${inactif ? '<span class="chip">Inactif</span>' : (complete ?
      <td style="text-align:right;color:var(--c-ink-4);font-size:18px;"
    </tr>
    `)).join('')}
  </tbody>
</table>
</div>
</div>
`;
}

function bindEmployees() {
  $('#addEmp').addEventListener('click', () => openEmployeeCreator());
  $('[data-view-emp]').forEach(b => b.addEventListener('click', e => {
    const id = parseInt(e.currentTarget.dataset.viewEmp);
    state.empDetail = id;
    state.empTab = 'info';
    render();
  }));
}

```

```

function openEmployeeCreator() {
  const newId = Math.max(0, ...state.employees.map(x => x.id||0)) + 1;
  const body = `
    <div class="form-grid">
      <div class="field"><label class="field-label">Prénom *</label><input class=
      <div class="field"><label class="field-label">Nom *</label><input class="in
      <div class="field"><label class="field-label">Poste</label><input class="in
      <div class="field"><label class="field-label">Contrat</label>
        <select class="input" id="ncContrat">
          ${[['CDI', 'CDD', 'Extra', 'Apprenti', 'Stage']].map(c => `<option>${c}</opti
        </select>
      </div>
      <div class="field"><label class="field-label">Heures hebdo</label><input cl
      <div class="field"><label class="field-label">Email</label><input class="in
    </div>
    <div class="text-mute" style="margin-top:12px;font-size:12px;">Les autres inf
  `;
  const footer = `
    <div class="spacer"></div>
    <button class="btn-sec" data-close>Annuler</button>
    <button class="btn-pri" id="ncSave" style="width:auto;padding:10px 18px;">Cré
  `;
  const { close } = openModal({ title: 'Nouveau salarié', body, footer });
  $('#ncSave').addEventListener('click', () => {
    const prenom = $('#ncPrenom').value.trim();
    const nom = $('#ncNom').value.trim();
    if (!prenom || !nom) { toast('Prénom et nom requis', 'error'); return; }
    const emp = normEmp({
      id: newId, prenom, nom,
      poste: $('#ncPoste').value.trim() || 'Cuisinier',
      contrat: $('#ncContrat').value,
      heures: parseInt($('#ncHeures').value) || 35,
      email: $('#ncEmail').value.trim(),
      taux: 12, statut: 'Actif',
      username: prenom.toLowerCase().replace(/^[a-z0-9]/g, ''),
      code: '1234',
    });
    state.employees.push(emp);
    fbSave('employees', state.employees);
    toast('Salarié créé', 'good');
    close();
    state.empDetail = newId;
    state.empTab = 'info';
    render();
  });
}

```

```
// ----- EMPLOYEE DETAIL -----

function pageEmployeeDetail() {
  const e = state.employees.find(x => x.id === state.empDetail);
  if (!e) { state.empDetail = null; return pageEmployees(); }
  const n = normEmp(e);
  const tab = state.empTab || 'info';

  return `
    <div class="row" style="margin-bottom:14px;">
      <button class="btn-ghost" id="backToList">← Retour à la liste</button>
    </div>

    <div class="emp-header">
      <div class="emp-header-top">
        <div class="av-emp lg">${initials(e)}</div>
        <div style="flex:1;min-width:0;">
          <div class="emp-header-name">${esc(e.prenom)} ${esc(e.nom)}</div>
          <div class="emp-header-sub">${esc(e.poste||'Non renseigné')} · ${esc(e.
        </div>
        <div style="display:flex;flex-direction:column;align-items:flex-end;gap:8
          <span class="chip ${e.statut==='Actif'? 'good': 'alert'}">${esc(e.statut|
          ${e.statut !== 'Actif' ? `<button class="btn-sec" id="reactivateEmp" st
        </div>
      </div>
      <div class="emp-header-grid">
        <div><div class="ehg-label">Début contrat</div><div class="ehg-value">${n
        <div><div class="ehg-label">Fin contrat</div><div class="ehg-value">${n.c
        <div><div class="ehg-label">Type</div><div class="ehg-value">${esc(e.cont
        <div><div class="ehg-label">Établissement</div><div class="ehg-value">Man
        <div><div class="ehg-label">Solde CP</div><div class="ehg-value">${(n.cpA
      </div>
    </div>

    <div class="emp-tabs">
      <button class="emp-tab ${tab==='info'? 'active': ''}" data-emp-tab="info">Inf
      <button class="emp-tab ${tab==='contrats'? 'active': ''}" data-emp-tab="contr
      <button class="emp-tab ${tab==='temps'? 'active': ''}" data-emp-tab="temps">T
      <button class="emp-tab ${tab==='conges'? 'active': ''}" data-emp-tab="conges"
      <button class="emp-tab ${tab==='docs'? 'active': ''}" data-emp-tab="docs">Doc
      <button class="emp-tab ${tab==='role'? 'active': ''}" data-emp-tab="role">Rôl
    </div>

    <div class="emp-tabbody">
      ${tab==='info' ? empTabInfo(n) :
      tab==='contrats' ? empTabContrats(n) :
      tab==='temps' ? empTabTemps(n) :

```



```

        tab=== 'conges' ? empTabConges(n) :
        tab=== 'docs' ? empTabDocs(n) :
        tab=== 'role' ? empTabRole(n) : ''
    </div>
    `;
}

function bindEmployeeDetail() {
    $('#backToList').addEventListener('click', () => {
        state.empDetail = null;
        render();
    });
    $$('[data-emp-tab]').forEach(b => b.addEventListener('click', ev => {
        state.empTab = ev.currentTarget.dataset.empTab;
        render();
    }));
    const reactBtn = $('#reactivateEmp');
    if (reactBtn) reactBtn.addEventListener('click', () => {
        const e = state.employees.find(x => x.id === state.empDetail);
        if (!e) return;
        if (!confirm(`Réactiver ${e.prenom} ${e.nom} ?\n\nLe salarié redevient actif`)) return;
        const updated = { ...e, statut: 'Actif', dateSortie: '', motifSortie: '' };
        state.employees = state.employees.map(x => x.id === e.id ? updated : x);
        fbSave('employees', state.employees);
        toast(`${e.prenom} réactivé(e)`, 'good');
        render();
    });
    // Tab-specific bindings
    const tab = state.empTab || 'info';
    if (tab === 'info') bindEmpTabInfo();
    if (tab === 'contrats') bindEmpTabContrats();
    if (tab === 'temps') bindEmpTabTemps();
    if (tab === 'conges') bindEmpTabConges();
    if (tab === 'docs') bindEmpTabDocs();
    if (tab === 'role') bindEmpTabRole();
}

// — INFOS PERSO TAB —
function empTabInfo(n) {
    return `
        <div class="info-grid">
            <div class="panel">
                <div class="panel-head"><h3>État civil</h3></div>
                <div class="panel-body">
                    ${kvRow('Genre', n.genre || 'Non renseigné')}
                    ${kvRow('Prénom', n.prenom)}
                    ${kvRow('Nom de naissance', n.nomNaissance || 'Non renseigné')}
                </div>
            </div>
        </div>
    `;
}

```

```

    ${kvRow('Nom de famille', n.nom)}
    ${kvRow('Nationalité', n.nationalite || 'Non renseigné')}
    ${kvRow('Date de naissance', n.dateNaissance ? fmtDateShort(n.dateNaiss
    ${kvRow('Pays de naissance', n.paysNaissance || 'Non renseigné')}
    ${kvRow('Département de naissance', n.departementNaissance || 'Non rens
    ${kvRow('Commune de naissance', n.communeNaissance || 'Non renseigné')}
  </div>
</div>
<div class="panel">
  <div class="panel-head"><h3>Coordonnées</h3></div>
  <div class="panel-body">
    ${kvRow('Email', n.email || 'Non renseigné')}
    ${kvRow('Tél. mobile', n.telMobile || 'Non renseigné')}
    ${kvRow('Notifications SMS', n.notifSMS ? 'Oui' : 'Non')}
    ${kvRow('Tél. fixe', n.telFixe || 'Non renseigné')}
    ${kvRow('Adresse', n.adresse || 'Non renseigné')}
    ${kvRow('Complément', n.complementAdresse || 'Non renseigné')}
    ${kvRow('Code postal', n.codePostal || 'Non renseigné')}
    ${kvRow('Ville', n.ville || 'Non renseigné')}
    ${kvRow('Pays', n.pays || 'France')}
  </div>
</div>
</div>
<div class="row" style="justify-content:center;margin-top:18px;">
  <button class="btn-pri" id="editInfo" style="width:auto;padding:10px 18px;"
</div>
`;
}

```

```

function bindEmpTabInfo() {
  $('#editInfo').addEventListener('click', () => openInfoEditor());
}

```

```

function openInfoEditor() {
  const e = state.employees.find(x => x.id === state.empDetail);
  const n = normEmp(e);
  const body = `
    <div class="modal-section-title">État civil</div>
    <div class="form-grid">
      <div class="field"><label class="field-label">Genre</label>
      <select class="input" id="iGenre">
        <option value="" ${!n.genre?'selected':''}>Non renseigné</option>
        <option value="M" ${n.genre==='M'?'selected':''}>Masculin</option>
        <option value="F" ${n.genre==='F'?'selected':''}>Féminin</option>
      </select>
    </div>
    <div class="field"><label class="field-label">Prénom</label><input class="i
  `;
}

```

```

    <div class="field"><label class="field-label">Nom de naissance</label><input
    <div class="field"><label class="field-label">Nom de famille</label><input
    <div class="field"><label class="field-label">Nationalité</label><input cla
    <div class="field"><label class="field-label">Date de naissance</label><inp
    <div class="field"><label class="field-label">Pays de naissance</label><inp
    <div class="field"><label class="field-label">Département de naissance</lab
    <div class="field full"><label class="field-label">Commune de naissance</la
  </div>

  <div class="modal-section-title">Coordonnées</div>
  <div class="form-grid">
    <div class="field"><label class="field-label">Email</label><input class="in
    <div class="field"><label class="field-label">Tél. mobile</label><input cla
    <div class="field"><label class="field-label">Tél. fixe</label><input class
    <div class="field"><label class="field-label"><input type="checkbox" id="iN
    <div class="field full"><label class="field-label">Adresse</label><input cl
    <div class="field full"><label class="field-label">Complément d'adresse</la
    <div class="field"><label class="field-label">Code postal</label><input cla
    <div class="field"><label class="field-label">Ville</label><input class="in
    <div class="field"><label class="field-label">Pays</label><input class="inp
  </div>
`;
const footer = `
  <div class="spacer"></div>
  <button class="btn-sec" data-close>Annuler</button>
  <button class="btn-pri" id="iSave" style="width:auto;padding:10px 18px;">Enre
`;
const { close } = openModal({ title: 'Modifier les informations personnelles',
$('#iSave').addEventListener('click', () => {
  const updated = {
    ...e,
    genre: $('#iGenre').value,
    prenom: $('#iPrenom').value.trim(),
    nomNaissance: $('#iNomNaiss').value.trim(),
    nom: $('#iNom').value.trim(),
    nationalite: $('#iNat').value.trim(),
    dateNaissance: $('#iDOB').value,
    paysNaissance: $('#iPaysN').value.trim(),
    departementNaissance: $('#iDeptN').value.trim(),
    communeNaissance: $('#iComN').value.trim(),
    email: $('#iEmail').value.trim(),
    telMobile: $('#iTelM').value.trim(),
    telFixe: $('#iTelF').value.trim(),
    notifSMS: $('#iNotif').checked,
    adresse: $('#iAdr').value.trim(),
    complementAdresse: $('#iAdr2').value.trim(),
    codePostal: $('#iCP').value.trim(),
  };

```

```

        ville: $('#iVille').value.trim(),
        pays: $('#iPays').value.trim() || 'France',
    });
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Informations mises à jour', 'good');
    close();
    render();
});
}

// — CONTRATS TAB —
function empTabContrats(n) {
    const avenants = n.avenants || [];
    const today = new Date();
    // Compute essai end
    let essaiFin = null;
    if (n.contratDebut && n.periodeEssaiJours > 0) {
        essaiFin = new Date(n.contratDebut + 'T00:00:00');
        essaiFin.setDate(essaiFin.getDate() + n.periodeEssaiJours);
    }
    const essaiInCours = essaiFin && today < essaiFin;
    return `
        <div class="panel">
            <div class="panel-head"><h3>Contrat en cours</h3></div>
            <div class="panel-body">
                ${kvRow('Type', n.contrat || 'Non renseigné')}
                ${kvRow('Début du contrat', n.contratDebut ? fmtDateShort(n.contratDebut)
                ${kvRow('Fin du contrat', n.contratFin ? fmtDateShort(n.contratFin) : (n.
                ${kvRow('Période d\'essai', n.periodeEssaiJours ? `${n.periodeEssaiJours}
                ${kvRow('Poste', n.poste || 'Non renseigné')}
                ${kvRow('Rémunération mensuelle brute', n.remuneration ? `${n.remuneration
                ${kvRow('Taux horaire', n.taux ? `${n.taux} €/h` : 'Non renseigné')}
                ${kvRow('Durée de travail hebdomadaire', `${n.heures || 0}h`)}
                ${kvRow('Nb. de jours travaillés par semaine', `${n.joursTravaillés || 5}
                ${kvRow('Remboursement Navigo / mois', n.navigoMensuel ? `${n.navigoMensuel
            </div>
        </div>

        <div class="panel" style="margin-top:14px;">
            <div class="panel-head"><h3>Conformité & administratif</h3></div>
            <div class="panel-body">
                ${kvRow('Contrat signé', n.contratSigneAt ? `<span class="chip good">✓ ${
                ${kvRow('DPAE faite', n.dpaeFaiteAt ? `<span class="chip good">✓ ${fmtDate
                ${kvRow('Période d\'essai renouvelable', n.periodeEssaiRenouvelable ? `Ou
                ${kvRow('Visite médicale d\'embauche', n.visiteMedicaleAt ? `<span class=
                ${kvRow('Prochaine visite médicale', n.visiteMedicaleProchaine ? fmtDateS

```

```

    ${kvRow('Mutuelle', n.mutuelleDispensee ? `Dispensé(e)` ${n.mutuelleDispens
    ${kvRow('Dernier entretien professionnel', n.dernierEntretienPro ? fmtDat
    ${kvRow('Formation HACCP', n.certifHACCP ? `<span class="chip good">✓` ${n
    ${n.travailleurEtranger ? `
        ${kvRow('Titre de séjour', n.titreSejourFin ? `${esc(n.titreSejourType|
        ${kvRow('Autorisation de travail', n.autorisationTravailFin ? `n° ${esc
        : ''}
    ${kvRow('Pass Navigo renouvelé le', n.navigoDernierRenouvellement ? fmtDa
</div>
</div>

```

```

<div class="panel" style="margin-top:14px;">
  <div class="panel-head">
    <h3>Tous les contrats et avenants</h3>
    <button class="btn-sec" id="addAvenant">+ Nouvel avenant</button>
  </div>
  <div class="panel-body ${avenants.length?'nopad':''}">
    ${avenants.length === 0 ? '<div class="text-mute" style="text-align:cente
    <table class="tbl">
      <thead><tr><th>Date</th><th>Type</th><th>Modification</th><th>Détail<
      <tbody>
        ${avenants.map((a,i) => `
          <tr>
            <td class="mono">${a.date ? fmtDateShort(a.date) : '-'}</td>
            <td>${esc(a.type||'-')}</td>
            <td>${esc(a.modif||'-')}</td>
            <td class="text-mute">${esc(a.detail||'-')}</td>
            <td><button class="btn-ghost" data-del-avenant="${i}">Supprimer
          </tr>
        `).join('')}
      </tbody>
    </table>
  `}
  </div>
</div>

```

```

<div class="row" style="justify-content:center;gap:8px;margin-top:18px;flex-w
  <button class="btn-pri" id="editContrat" style="width:auto;">✎ Modifier le
  <button class="btn-sec" id="editConformite" style="width:auto;">⚖️ Conformi
  <button class="btn-sec" id="editModalites" style="width:auto;">📄 Modalités
  ${!n.dateSortie
    ? `<button class="btn-sec" id="openPackSortie" style="width:auto;">📦 Pac
    : `<button class="btn-sec" id="cancelSortie" style="width:auto;">🚫 Annule
</div>
${n.dateSortie ? `
  <div style="text-align:center;margin-top:12px;">
    <span class="chip alert">Sortie enregistrée le ${fmtDateShort(n.dateSorti

```

```

        </div>
        ` : ''}
    `;
}

```

```

function bindEmpTabContrats() {
    $('#editContrat').addEventListener('click', () => openContratEditor());
    const addBtn = $('#addAvenant');
    if (addBtn) addBtn.addEventListener('click', () => openAvenantEditor());
    const confBtn = $('#editConformite');
    if (confBtn) confBtn.addEventListener('click', () => openConformiteEditor());
    const modBtn = $('#editModalites');
    if (modBtn) modBtn.addEventListener('click', () => openModalitesPaie());
    const psBtn = $('#openPackSortie');
    if (psBtn) psBtn.addEventListener('click', () => openPackSortie(state.empDetail));
    const cancelBtn = $('#cancelSortie');
    if (cancelBtn) cancelBtn.addEventListener('click', () => {
        const e = state.employees.find(x => x.id === state.empDetail);
        if (!e) return;
        if (!confirm(`Annuler la sortie de ${e.prenom} ${e.nom} et le réactiver ?\n\n`)) return;
        const updated = { ...e, statut: 'Actif', dateSortie: '', motifSortie: '' };
        state.employees = state.employees.map(x => x.id === e.id ? updated : x);
        fbSave('employees', state.employees);
        toast(`${e.prenom} réactivé(e)`, 'good');
        render();
    });
    $$('[data-del-avenant]').forEach(b => b.addEventListener('click', ev => {
        const i = parseInt(ev.currentTarget.dataset.delAvenant);
        if (!confirm('Supprimer cet avenant ?')) return;
        const e = state.employees.find(x => x.id === state.empDetail);
        const avenants = [...(e.avenants||[])];
        avenants.splice(i, 1);
        const updated = { ...e, avenants };
        state.employees = state.employees.map(x => x.id === e.id ? updated : x);
        fbSave('employees', state.employees);
        toast('Avenant supprimé', '');
        render();
    }));
}

```

```

function openContratEditor() {
    const e = state.employees.find(x => x.id === state.empDetail);
    const n = normEmp(e);
    const body = `
        <div class="form-grid">
            <div class="field"><label class="field-label">Type de contrat</label>
                <select class="input" id="cType">

```

```

        ${['CDI', 'CDD', 'Extra', 'Apprenti', 'Stage', 'Interim'].map(c => `<option
        </select>
    </div>
    <div class="field"><label class="field-label">Poste</label><input class="in
    <div class="field"><label class="field-label">Date de début</label><input c
    <div class="field"><label class="field-label">Date de fin (laisser vide pou
    <div class="field"><label class="field-label">Période d'essai (jours)</labe
    <div class="field"><label class="field-label">Heures hebdo</label><input cl
    <div class="field"><label class="field-label">Jours travaillés / semaine</l
    <div class="field"><label class="field-label">Taux horaire (€)</label><inpu
    <div class="field full"><label class="field-label">Rémunération mensuelle b
    <div class="field full"><label class="field-label">Remboursement Navigo / t
    </div>
    `;
const footer = `
    <div class="spacer"></div>
    <button class="btn-sec" data-close>Annuler</button>
    <button class="btn-pri" id="cSave" style="width:auto;padding:10px 18px;">Enre
    `;
const { close } = openModal({ title: 'Modifier le contrat', body, footer });
$('#cSave').addEventListener('click', () => {
    const updated = {
        ...e,
        contrat: $('#cType').value,
        poste: $('#cPoste').value.trim(),
        contratDebut: $('#cDebut').value,
        contratFin: $('#cFin').value,
        periodeEssaiJours: parseInt($('#cEssai').value) || 60,
        heures: parseInt($('#cHeures').value) || 35,
        joursTravaillés: parseInt($('#cJours').value) || 5,
        taux: parseFloat($('#cTaux').value) || 12,
        remuneration: parseFloat($('#cRem').value) || 0,
        navigoMensuel: parseFloat($('#cNavigo').value) || 0,
    };
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Contrat mis à jour', 'good');
    close();
    render();
});
}

function openAvenantEditor() {
    const body = `
        <div class="form-grid">
            <div class="field"><label class="field-label">Date</label><input class="inp
            <div class="field"><label class="field-label">Type</label>

```

```

        <select class="input" id="aType">
          <option>Changement d'horaire</option>
          <option>Augmentation</option>
          <option>Changement de poste</option>
          <option>Renouvellement CDD</option>
          <option>Autre</option>
        </select>
      </div>

      <div class="field full"><label class="field-label">Modification (résumé cou
      <div class="field full"><label class="field-label">Détail (optionnel)</labe
    </div>
  `;
  const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
  const { close } = openModal({ title: 'Nouvel avenant', body, footer });
  $('#aSave').addEventListener('click', () => {
    const e = state.employees.find(x => x.id === state.empDetail);
    const newAv = {
      date: $('#aDate').value,
      type: $('#aType').value,
      modif: $('#aModif').value.trim(),
      detail: $('#aDetail').value.trim(),
    };
    if (!newAv.modif) { toast('Saisis au moins la modification', 'error'); return
    const avenants = [...(e.avenants||[]), newAv].sort((a,b) => (b.date||'').loca
    const updated = { ...e, avenants };
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Avenant ajouté', 'good');
    close();
    render();
  });
}

// — Conformité editor —
function openConformiteEditor() {
  const e = state.employees.find(x => x.id === state.empDetail);
  const n = normEmp(e);
  const body = `
    <div class="modal-section-title">Signature et déclarations</div>
    <div class="form-grid">
      <div class="field"><label class="field-label">Contrat signé le</label><inpu
      <div class="field"><label class="field-label">DPAE faite le</label><input c
      <div class="field full">
        <label style="display:flex;align-items:center;gap:8px;font-size:13px;curs
          <input type="checkbox" id="cfEssaiRenouvelable" ${n.périodeEssaiRenouve
            Période d'essai renouvelable
          </label>

```



```

    </div>
    <div class="field full"><label class="field-label">Renouvellement effectué
</div>

<div class="modal-section-title">Médecine du travail</div>
<div class="form-grid">
    <div class="field"><label class="field-label">Dernière visite médicale</lab
    <div class="field">
        <label class="field-label">Type</label>
        <select class="input" id="cfVMType">
            <option value=""></option>
            <option ${n.visiteMedicaleType==='embauche'? 'selected':''} value="embau
            <option ${n.visiteMedicaleType==='periodique'? 'selected':''} value="per
            <option ${n.visiteMedicaleType==='reprise'? 'selected':''} value="repris
        </select>
    </div>
    <div class="field full"><label class="field-label">Prochaine visite prévue<
</div>

<div class="modal-section-title">Mutuelle</div>
<div class="form-grid">
    <div class="field full">
        <label style="display:flex;align-items:center;gap:8px;font-size:13px;curs
            <input type="checkbox" id="cfMutDisp" ${n.mutuelleDispensee?'checked':''}
            Salarié(e) dispensé(e) de la mutuelle
        </label>
    </div>
    <div class="field"><label class="field-label">Adhésion mutuelle le</label><
    <div class="field"><label class="field-label">Motif de dispense</label>
        <select class="input" id="cfMutRaison">
            <option value=""></option>
            <option ${n.mutuelleDispenseRaison==='CDD < 3 mois'? 'selected':''}>CDD
            <option ${n.mutuelleDispenseRaison==='Couverture conjoint'? 'selected':''}
            <option ${n.mutuelleDispenseRaison==='CSS / ACS'? 'selected':''}>CSS / A
            <option ${n.mutuelleDispenseRaison==='Multi-employeur'? 'selected':''}>M
        </select>
    </div>
</div>

<div class="modal-section-title">Formation et entretiens</div>
<div class="form-grid">
    <div class="field full">
        <label style="display:flex;align-items:center;gap:8px;font-size:13px;curs
            <input type="checkbox" id="cfHACCP" ${n.certifHACCP?'checked':''}>
            Titulaire de la formation HACCP / hygiène alimentaire
        </label>
    </div>
</div>

```

```

        <div class="field"><label class="field-label">Date de la formation HACCP</l
        <div class="field"><label class="field-label">Dernier entretien professionn
    </div>

    <div class="modal-section-title">Travailleur étranger</div>
    <div class="form-grid">
        <div class="field full">
            <label style="display:flex;align-items:center;gap:8px;font-size:13px;curs
                <input type="checkbox" id="cfEtranger" ${n.travailleurEtranger?'checked
                Salarié(e) de nationalité étrangère hors UE/EEE
            </label>
        </div>
        <div class="field"><label class="field-label">N° titre de séjour</label><in
        <div class="field"><label class="field-label">Type titre de séjour</label>
            <select class="input" id="cfTSType">
                <option value="">—</option>
                ${['Carte de séjour temporaire','Carte de résident','APS','Récépissé','
            </select>
        </div>
        <div class="field"><label class="field-label">Titre valide jusqu'au</label>
        <div class="field"><label class="field-label">N° autorisation travail</labe
        <div class="field full"><label class="field-label">Autorisation travail exp
    </div>

    <div class="modal-section-title">Transports</div>
    <div class="form-grid">
        <div class="field full"><label class="field-label">Pass Navigo renouvelé le
    </div>
`;
const footer = `
    <button class="btn-sec" data-close>Annuler</button>
    <button class="btn-pri" id="cfSave" style="width:auto;">Enregistrer</button>
`;
const { close } = openModal({ title: 'Conformité & administratif', body, footer

$('#cfSave').addEventListener('click', () => {
    const updated = {
        ...e,
        contratSigneAt: $('#cfContratSigneAt').value,
        dpaeFaiteAt: $('#cfDpaeFaiteAt').value,
        periodeEssaiRenouvelable: $('#cfEssaiRenouvelable').checked,
        periodeEssaiRenouveleeAt: $('#cfEssaiRenouvAt').value,
        visiteMedicaleAt: $('#cfVMAAt').value,
        visiteMedicaleType: $('#cfVMType').value,
        visiteMedicaleProchaine: $('#cfVMNext').value,
        mutuelleDispensee: $('#cfMutDisp').checked,
        mutuelleAdhesionAt: $('#cfMutAt').value,

```

```

    mutuelleDispenseRaison: $('#cfMutRaison').value,
    certifHACCP: $('#cfHACCP').checked,
    certifHACCPDate: $('#cfHACCPDate').value,
    dernierEntretienPro: $('#cfEntretien').value,
    travailleurEtranger: $('#cfEtranger').checked,
    titreSejourNumero: $('#cfTSNum').value.trim(),
    titreSejourType: $('#cfTSType').value,
    titreSejourFin: $('#cfTSFin').value,
    autorisationTravailNumero: $('#cfATNum').value.trim(),
    autorisationTravailFin: $('#cfATFin').value,
    navigoDernierRenouvellement: $('#cfNavigoAt').value,
  };
  state.employees = state.employees.map(x => x.id === e.id ? updated : x);
  fbSave('employees', state.employees);
  toast('Conformité mise à jour', 'good');
  close();
  render();
});
}

// — Modalités de paie spécifiques —
function openModalitesPaie() {
  const e = state.employees.find(x => x.id === state.empDetail);
  const n = normEmp(e);
  const flag = (id, checked, label, detail) => `
    <label class="modalite-row">
      <input type="checkbox" id="${id}" ${checked?'checked':''}>
      <div>
        <div class="modalite-label">${label}</div>
        <div class="modalite-detail">${detail}</div>
      </div>
    </label>
  `;
  const body = `
    <div class="text-mute" style="font-size:12.5px;line-height:1.5;margin-bottom:
      Ces réglages adaptent le calcul de la paie pour ce salarié. Ils s'appliquen
    </div>
    ${flag('mpSansHS', n.sansHeuresSupp, 'Sans heures supplémentaires', 'Toutes l
    ${flag('mpForfait', n.forfaitJours, 'Forfait jours', 'Cadre au forfait – pas
    ${flag('mpExclureCP', n.exclureCP, 'Sans gestion de congés payés', 'Ce salari
    ${flag('mpPayeNet', n.payeNet, 'Rémunération exprimée en net', 'Le taux horai
    ${flag('mpSansRepas', n.sansAvantageRepas, 'Sans avantage en nature repas', '
    ${flag('mpSansNavigo', n.sansNavigo, 'Sans remboursement transport', 'Pas de
    ${flag('mpExclurePaie', n.exclurePaie, 'Exclure du récap paie', 'Ce salarié n
  `;
  const footer = `
    <button class="btn-sec" data-close>Annuler</button>
  `;

```

```

<button class="btn-pri" id="mpSave" style="width:auto;">Enregistrer</button>
`;
const { close } = openModal({ title: 'Modalités de paie', body, footer });

$('#mpSave').addEventListener('click', () => {
  const updated = {
    ...e,
    sansHeuresSupp: $('#mpSansHS').checked,
    forfaitJours: $('#mpForfait').checked,
    exclureCP: $('#mpExclureCP').checked,
    payeNet: $('#mpPayeNet').checked,
    sansAvantageRepas: $('#mpSansRepas').checked,
    sansNavigo: $('#mpSansNavigo').checked,
    exclurePaie: $('#mpExclurePaie').checked,
  };
  state.employees = state.employees.map(x => x.id === e.id ? updated : x);
  fbSave('employees', state.employees);
  toast('Modalités de paie mises à jour', 'good');
  close();
  render();
});
}

// — TEMPS & PLANIF TAB —
function empTabTemps(n) {
  const now = new Date();
  const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
  let monthH = 0;
  for (let d = new Date(monthStart); d <= new Date(now.getFullYear(), now.getMont
    const wk = weekKey(d);
    const dayIdx = (d.getDay() + 6) % 7;
    (state.shifts[`${n.id}_${dayIdx}_${wk}`] || []).forEach(s => monthH += shiftH
  }
  return `
    <div class="info-grid">
      <div class="panel">
        <div class="panel-head"><h3>Temps de travail</h3></div>
        <div class="panel-body">
          ${kvRow('Mois en cours', `${monthH.toFixed(1)} h`)}
          ${kvRow('Heures contractuelles hebdo', `${n.heures||0} h`)}
          ${kvRow('Estimation mensuelle contractuelle', `~${((n.heures||0) * 4.33
        </div>
      </div>
      <div class="panel">
        <div class="panel-head"><h3>Disponibilités hebdomadaires</h3></div>
        <div class="panel-body" style="padding:0;">
          ${DAYS.map((d,i) => `

```

```

        <div class="row" style="padding:10px 16px;border-bottom:1px solid var
          <span style="flex:1;font-weight:500;">${d}</span>
          <span class="chip ${n.dispos?.[i]!==false?'good':''}" style="font-s
        </div>
      `).join('')}
    </div>
  </div>
</div>
<div class="row" style="justify-content:center;margin-top:18px;">
  <button class="btn-pri" id="editDispos" style="width:auto;padding:10px 18px
</div>
`;
}

```

```

function bindEmpTabTemps() {
  $('#editDispos').addEventListener('click', () => openDisposEditor());
}

```

```

function openDisposEditor() {
  const e = state.employees.find(x => x.id === state.empDetail);
  const n = normEmp(e);
  const body = `
    <div class="text-mute" style="margin-bottom:10px;font-size:12.5px;">Décoche 1
    ${DAYS.map((d,i) => `
      <label class="row" style="padding:10px 0;border-bottom:1px solid var(--c-li
        <input type="checkbox" id="d${i}" ${n.dispos[i]!==false?'checked':''} sty
        <span style="flex:1;font-weight:500;">${d}</span>
      </label>
    `).join('')}
  `;
  const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
  const { close } = openModal({ title: 'Disponibilités', body, footer });
  $('#dSave').addEventListener('click', () => {
    const dispos = {};
    for (let i = 0; i < 7; i++) dispos[i] = $('#d'+i).checked;
    const updated = { ...e, dispos };
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Disponibilités mises à jour', 'good');
    close();
    render();
  });
}

```

// — CONGÉS TAB —

```

function empTabConges(n) {
  const cpPrisAuto = computeCpPris(n.id);

```

```

const soldeN = (n.cpAcquisN||0) - (n.cpPrisN||0);
const soldeNm1 = (n.cpAcquisNm1||0) - (n.cpPrisNm1||0);
const total = soldeN + soldeNm1;
const reqs = Object.values(state.absenceRequests||{}).filter(r => r && r.empId
return `
  <div class="cp-grid">
    <div class="cp-card">
      <div class="cp-card-label">Solde CP total</div>
      <div class="cp-card-value">${total.toFixed(1)} j</div>
      <div class="cp-card-sub">N: ${soldeN.toFixed(1)} · N-1: ${soldeNm1.toFixe
    </div>
    <div class="cp-card">
      <div class="cp-card-label">Repos compensateur</div>
      <div class="cp-card-value">${(n.reposCompensateur||0)} h</div>
    </div>
    <div class="cp-card">
      <div class="cp-card-label">Récupération jours fériés</div>
      <div class="cp-card-value">${(n.recupJoursFeries||0)} h</div>
    </div>
  </div>

  <div class="panel" style="margin-top:14px;">
    <div class="panel-head"><h3>Compteurs CP – Détail</h3></div>
    <div class="panel-body nopad">
      <table class="tbl">
        <thead><tr><th>Période</th><th>Acquis</th><th>Pris</th><th>Solde</th></
        <tbody>
          <tr><td>N-1 (année précédente)</td><td class="mono tabular">${(n.cpAc
          <tr><td>N (année en cours)</td><td class="mono tabular">${(n.cpAcquis
        </tbody>
      </table>
    </div>
  </div>

  <div class="panel" style="margin-top:14px;">
    <div class="panel-head"><h3>Demandes d'absence</h3></div>
    <div class="panel-body ${reqs.length?'nopad':''}">
      ${reqs.length === 0 ? '<div class="text-mute" style="text-align:center;pa
      <table class="tbl">
        <thead><tr><th>Date(s)</th><th>Jours</th><th>Type</th><th>Statut</th>
        <tbody>
          ${reqs.sort((a,b) => (b.createdAt||'').localeCompare(a.createdAt||'
          <tr>
            <td>${fmtDateShort(r.dateStart)} → ${fmtDateShort(r.dateEnd)}</
            <td class="mono">${r.nbJours || '?'}</td>
            <td>${esc(LEAVE_TYPES[r.type]?.label || r.type)}</td>
            <td>${reqStatusChip(r.status)}</td>

```

```

        </tr>
        `).join('')}
    </tbody>
</table>
`}
</div>
</div>

<div class="row" style="justify-content:center;margin-top:18px;gap:10px;">
    <button class="btn-sec" id="editCp">✎ Modifier les compteurs CP</button>
    <button class="btn-pri" id="addAbsence" style="width:auto;padding:10px 18px">Ajouter</button>
</div>
`;
}

function bindEmpTabConges() {
    $('#editCp').addEventListener('click', () => openCpEditor());
    $('#addAbsence').addEventListener('click', () => openAbsenceCreator(state.empDe
}

function openCpEditor() {
    const e = state.employees.find(x => x.id === state.empDetail);
    const n = normEmp(e);
    const body = `
        <div class="modal-section-title">N-1 (année précédente)</div>
        <div class="form-grid">
            <div class="field"><label class="field-label">CP acquis N-1</label><input c
            <div class="field"><label class="field-label">CP pris N-1</label><input cla
        </div>
        <div class="modal-section-title">N (année en cours)</div>
        <div class="form-grid">
            <div class="field"><label class="field-label">CP acquis N</label><input cla
            <div class="field"><label class="field-label">CP pris N (manuel)</label><in
        </div>
        <div class="modal-section-title">Autres compteurs</div>
        <div class="form-grid">
            <div class="field"><label class="field-label">Repos compensateur (heures)</
            <div class="field"><label class="field-label">Récup jours fériés (heures)</
        </div>
    `;
    const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
    const { close } = openModal({ title: 'Compteurs CP', body, footer });
    $('#cpSave').addEventListener('click', () => {
        const updated = {
            ...e,
            cpAcquisNm1: parseFloat($('#cAcq1').value) || 0,
            cpPrisNm1: parseFloat($('#cPri1').value) || 0,

```

```

        cpAcquisN: parseFloat($('#cAcq').value) || 0,
        cpPrisN: parseFloat($('#cPri').value) || 0,
        reposCompensateur: parseFloat($('#cRC').value) || 0,
        recupJoursFeries: parseFloat($('#cRJF').value) || 0,
    };
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Compteurs mis à jour', 'good');
    close();
    render();
  });
}

// — DOCS TAB —
function empTabDocs(n) {
  const docs = n.documents || [];
  // Group by category
  const byCat = {};
  docs.forEach(d => {
    const c = d.category || 'autre';
    if (!byCat[c]) byCat[c] = [];
    byCat[c].push(d);
  });

  const totalSize = docs.reduce((s,d) => s + (d.size||0), 0);

  return `
    <div class="row" style="gap:10px;margin-bottom:14px;flex-wrap:wrap;align-item
      <div class="row" style="gap:8px;flex:1;">
        <span class="chip">${docs.length} document${docs.length>1?'s':''}</span>
        <span class="chip">${formatBytes(totalSize)}</span>
      </div>
      ${docs.length > 0 ? `<button class="btn-sec" id="downloadAllDocs">↓ Tout té
        <button class="btn-pri" id="uploadDoc" style="width:auto;padding:9px 16px;"
      </div>

    ${docs.length === 0 ? `
      <div class="panel" style="background:#fafafa;border-style:dashed;">
        <div class="panel-body" style="text-align:center;padding:32px 18px;">
          <div style="font-size:36px;margin-bottom:8px;">📁</div>
          <div style="font-weight:500;margin-bottom:4px;">Aucun document pour le
          <div class="text-mute" style="font-size:13px;max-width:420px;margin:6px
            Téléverse contrat, avenants, navigo, arrêts maladie, fiches de paie,
          </div>
        </div>
      </div>
    ` : `
  `;

```



```

<div class="docs-grid">
  ${Object.entries(byCat).sort(([a],[b]) => a.localeCompare(b)).map(([cat,
    const c = DOC_CATEGORIES[cat] || DOC_CATEGORIES.autre;
    return `
      <div class="panel doc-cat-panel">
        <div class="panel-head">
          <h3>${c.icon} ${esc(c.label)} <span class="text-mute" style="font
        </div>
        <div class="panel-body" style="padding:0;">
          ${items.sort((a,b) => (b.uploadedAt||'').localeCompare(a.uploaded
            <div class="doc-item">
              <div class="doc-icon">${docFileIcon(d.type, d.name)}</div>
              <div class="doc-info">
                <div class="doc-name">${esc(d.name)}</div>
                <div class="doc-meta">${formatBytes(d.size||0)} · ${new Dat
              </div>
              <div class="doc-actions">
                <button class="btn-ghost" data-open-doc="${esc(d.id)}" data
                <button class="btn-ghost" data-dl-doc="${esc(d.id)}" data-f
                <button class="btn-ghost" data-del-doc="${esc(d.id)}" title
              </div>
            </div>
          `).join('')}
        </div>
      </div>
    `;
  }).join('')}
</div>
`}

${n.travailleurEtranger ? `
  <div class="panel" style="margin-top:14px;">
    <div class="panel-head"><h3> Titre de séjour – détails</h3></div>
    <div class="panel-body">
      ${kvRow('Type', n.titreSejourType || 'Non renseigné')}
      ${kvRow('Numéro', n.titreSejourNumero || 'Non renseigné')}
      ${kvRow('Début validité', n.titreSejourDebut ? fmtDateShort(n.titreSejo
      ${kvRow('Fin validité', n.titreSejourFin ? fmtDateShort(n.titreSejourFi
    </div>
  </div>
  ` : ''}
  `;
}

function docFileIcon(type, name) {
  const ext = (name||'').split('.').pop().toLowerCase();
  if (type && type.startsWith('image/')) return '';

```

```

    if (ext === 'pdf' || (type && type.includes('pdf'))) return '📄';
    if (['doc', 'docx'].includes(ext)) return '📑';
    if (['xls', 'xlsx', 'csv'].includes(ext)) return '📊';
    return '📁';
}

```

```

function bindEmpTabDocs() {
    $('#uploadDoc').addEventListener('click', () => openDocUploader());
    const dlAll = $('#downloadAllDocs');
    if (dlAll) dlAll.addEventListener('click', () => downloadAllEmpDocs());
    $$('[data-open-doc]').forEach(b => b.addEventListener('click', ev => {
        openDocInNewTab(ev.currentTarget.dataset.openDoc, ev.currentTarget.dataset.fn
    }));
    $$('[data-dl-doc]').forEach(b => b.addEventListener('click', ev => {
        downloadDocFile(ev.currentTarget.dataset.dlDoc, ev.currentTarget.dataset.fnam
    }));
    $$('[data-del-doc]').forEach(b => b.addEventListener('click', ev => {
        const docId = ev.currentTarget.dataset.delDoc;
        deleteEmpDoc(docId);
    }));
}

```

```

async function deleteEmpDoc(docId) {
    const e = state.employees.find(x => x.id === state.empDetail);
    const doc = (e.documents || []).find(d => d.id === docId);
    if (!doc) return;
    if (!confirm(`Supprimer "${doc.name}" ? Cette action est irréversible.`)) retur
    await deleteDocBlob(docId);
    const updated = { ...e, documents: (e.documents || []).filter(d => d.id !== docId
    state.employees = state.employees.map(x => x.id === e.id ? updated : x);
    fbSave('employees', state.employees);
    toast('Document supprimé', '');
    render();
}

```

```

async function downloadAllEmpDocs() {
    const e = state.employees.find(x => x.id === state.empDetail);
    const docs = e.documents || [];
    if (!docs.length) return;
    toast(`Téléchargement de ${docs.length} document${docs.length>1?'s':''}...`, ''
    for (const d of docs) {
        try {
            await downloadDocFile(d.id, d.name);
            await new Promise(r => setTimeout(r, 350));
        } catch (err) { console.warn(err); }
    }
    toast('Téléchargements terminés', 'good');
}

```

```
}
```

```
function openDocUploader() {
  const empId = state.empDetail;
  const e = state.employees.find(x => x.id === empId);
  const body = `
    <div class="form-grid">
      <div class="field full"><label class="field-label">Catégorie</label>
        <select class="input" id="dCat">
          ${Object.entries(DOC_CATEGORIES).map(([k,v]) => `<option value="${k}">${k}</option>`)}
        </select>
      </div>
      <div class="field full">
        <label class="field-label">Fichier(s)</label>
        <label class="file-drop" id="fileDrop">
          <input type="file" id="dFile" multiple accept="image/*,application/pdf," />
          <div class="file-drop-content">
            <div style="font-size:32px;"><img alt="upload icon" data-bbox="515 375 535 390"/></div>
            <div style="font-weight:500;margin-top:8px;">Cliquer pour choisir ou
              <div class="text-mute" style="font-size:12px;margin-top:4px;">>PDF, images, vidéos, etc.
              <div class="text-mute" style="font-size:11px;margin-top:2px;">Les images et vidéos sont limitées à 10 Mo.
            </div>
          </div>
        </label>
        <div id="fileList" style="margin-top:10px;"></div>
      </div>
      <div id="uploadProgress" style="display:none;margin-top:12px;">
        <div class="progress-bar"><div class="progress-fill" id="progFill"></div></div></div>
        <div class="text-mute" style="font-size:11.5px;margin-top:6px;" id="progLabel"></div>
      </div>
    `;
  const footer = `<div class="spacer"></div><button class="btn-sec" data-close>Annuler</button>`;
  const { close } = openModal({ title: `Ajouter un document - ${e.prenom} ${e.nom}` });

  let selectedFiles = [];

  const renderFileList = () => {
    const fl = $('#fileList');
    if (!selectedFiles.length) { fl.innerHTML = ''; return; }
    fl.innerHTML = selectedFiles.map((f,i) => `
      <div class="row" style="gap:8px;padding:6px 10px;background:#fafafa;border:1px solid #ccc;">
        <span style="flex:1;">${esc(f.name)}</span>
        <span class="text-mute mono">${formatBytes(f.size)}</span>
        <button class="btn-ghost" data-rm="${i}" style="padding:0 6px;">x</button>
      </div>
    `).join('');
    $('[data-rm]', fl).forEach(b => b.addEventListener('click', ev => {

```

```

        selectedFiles.splice(parseInt(ev.target.dataset.rm), 1);
        renderFileList();
        $('#dUpload').disabled = !selectedFiles.length;
    }));
};

$('#dFile').addEventListener('change', ev => {
    selectedFiles = [...ev.target.files];
    renderFileList();
    $('#dUpload').disabled = !selectedFiles.length;
});

// Drag & drop
const drop = $('#fileDrop');
['dragover', 'dragenter'].forEach(evt => drop.addEventListener(evt, ev => { ev.p
['dragleave', 'dragend', 'drop'].forEach(evt => drop.addEventListener(evt, ev =>
drop.addEventListener('drop', ev => {
    selectedFiles = [...ev.dataTransfer.files];
    renderFileList();
    $('#dUpload').disabled = !selectedFiles.length;
}));

$('#dUpload').addEventListener('click', async () => {
    if (!selectedFiles.length) return;
    const cat = $('#dCat').value;
    $('#dUpload').disabled = true;
    $('#uploadProgress').style.display = '';
    const e2 = state.employees.find(x => x.id === empId);
    const newDocs = [...(e2.documents || [])];
    let done = 0;
    for (const file of selectedFiles) {
        $('#progLabel').textContent = `Upload ${done+1}/${selectedFiles.length} : $
        $('#progFill').style.width = `${(done/selectedFiles.length)*100}%`;
        const meta = await uploadEmployeeDoc(empId, file, cat);
        if (meta) newDocs.push(meta);
        done++;
    }
    $('#progFill').style.width = '100%';
    $('#progLabel').textContent = 'Terminé !';
    const updated = { ...e2, documents: newDocs };
    state.employees = state.employees.map(x => x.id === empId ? updated : x);
    fbSave('employees', state.employees);
    toast(`${done} document${done>1?'s':''} ajouté${done>1?'s':''}`, 'good');
    setTimeout(close, 600);
    render();
});
}

```

```
// — ROLE TAB —
function empTabRole(n) {
  return `
    <div class="panel">
      <div class="panel-head"><h3>Compte</h3></div>
      <div class="panel-body">
        ${kvRow('Username', n.username || 'Non renseigné')}
        ${kvRow('Code PIN', n.code || '-')}
        ${kvRow('Accès au compte', n.peutSeConnecter !== false ? 'Activé' : 'Désa
      </div>
    </div>

    <div class="panel" style="margin-top:14px;">
      <div class="panel-head"><h3>Statut</h3></div>
      <div class="panel-body">
        ${kvRow('Statut', n.statut || 'Actif')}
        ${kvRow('Date de sortie', n.dateSortie ? fmtDateShort(n.dateSortie) : '-')}
        ${kvRow('Motif de sortie', n.motifSortie || '-')}
        ${kvRow('Travailleur étranger', n.travailleurEtranger ? 'Oui' : 'Non')}
      </div>
    </div>

    <div class="row" style="justify-content:center;margin-top:18px;gap:10px;">
      <button class="btn-sec" id="editRole">✎ Modifier le compte</button>
      <button class="btn-sec" id="markExit">Enregistrer une sortie</button>
      <button class="btn-danger" id="delEmp">Supprimer le salarié</button>
    </div>
  `;
}

function bindEmpTabRole() {
  $('#editRole').addEventListener('click', () => openRoleEditor());
  $('#markExit').addEventListener('click', () => openExitEditor());
  $('#delEmp').addEventListener('click', () => {
    const e = state.employees.find(x => x.id === state.empDetail);
    if (!confirm(`Supprimer ${e.prenom} ${e.nom} ? Cette action est irréversible.
    state.employees = state.employees.filter(x => x.id !== state.empDetail);
    fbSave('employees', state.employees);
    state.empDetail = null;
    toast('Salarié supprimé', '');
    render();
  });
}

function openRoleEditor() {
  const e = state.employees.find(x => x.id === state.empDetail);

```

```

const n = normEmp(e);
const body = `
  <div class="form-grid">
    <div class="field"><label class="field-label">Username</label><input class=
    <div class="field"><label class="field-label">Code PIN</label><input class=
    <div class="field full"><label class="field-label"><input type="checkbox" i
    <div class="field full"><label class="field-label"><input type="checkbox" i
  </div>
  <div id="foreignWrap" style="{n.travailleurEtranger?'':'display:none;'}">
    <div class="modal-section-title">Titre de séjour</div>
    <div class="form-grid">
      <div class="field"><label class="field-label">Type</label><input class="i
      <div class="field"><label class="field-label">Numéro</label><input class=
      <div class="field"><label class="field-label">Début validité</label><inpu
      <div class="field"><label class="field-label">Fin validité</label><input
    </div>
  </div>
`;
const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
const { close } = openModal({ title: 'Compte & permissions', body, footer });
$('#rForeign').addEventListener('change', ev => {
  $('#foreignWrap').style.display = ev.target.checked ? '' : 'none';
});
$('#rSave').addEventListener('click', () => {
  const updated = {
    ...e,
    username: $('#rUser').value.trim().toLowerCase(),
    code: $('#rCode').value.trim(),
    peutSeConnecter: $('#rConnect').checked,
    travailleurEtranger: $('#rForeign').checked,
    titreSejourType: $('#rTSType').value.trim(),
    titreSejourNumero: $('#rTSNum').value.trim(),
    titreSejourDebut: $('#rTSDeb').value,
    titreSejourFin: $('#rTSFin').value,
  };
  state.employees = state.employees.map(x => x.id === e.id ? updated : x);
  fbSave('employees', state.employees);
  toast('Compte mis à jour', 'good');
  close();
  render();
});
}

function openExitEditor() {
  const e = state.employees.find(x => x.id === state.empDetail);
  const n = normEmp(e);
  const body = `

```

```

<div class="form-grid">
  <div class="field"><label class="field-label">Date de sortie</label><input
  <div class="field"><label class="field-label">Motif</label>
    <select class="input" id="eMotif">
      <option value=""></option>
      <option ${n.motifSortie==='Démission'? 'selected':''}>Démission</option>
      <option ${n.motifSortie==='Fin de CDD'? 'selected':''}>Fin de CDD</optio
      <option ${n.motifSortie==='Licenciement'? 'selected':''}>Licenciement</o
      <option ${n.motifSortie==='Rupture conventionnelle'? 'selected':''}>Rupt
      <option ${n.motifSortie==='Fin de période d'essai'? 'selected':''}>Fin d
      <option ${n.motifSortie==='Autre'? 'selected':''}>Autre</option>
    </select>
  </div>
  <div class="field full" style="margin-top:8px;">
    <label class="field-label"><input type="checkbox" id="eInactif" checked>
  </div>
</div>
`;
const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
const { close } = openModal({ title: 'Enregistrer une sortie', body, footer });
$('#eSave').addEventListener('click', () => {
  const updated = {
    ...e,
    dateSortie: $('#eDate').value,
    motifSortie: $('#eMotif').value,
    statut: $('#eInactif').checked ? 'Inactif' : e.statut,
  };
  state.employees = state.employees.map(x => x.id === e.id ? updated : x);
  fbSave('employees', state.employees);
  toast('Sortie enregistrée', 'good');
  close();
  render();
});
}

// ————— ABSENCE REQUESTS —————
function reqStatusChip(s) {
  if (s === 'approved') return '<span class="chip good">✓ Validée</span>';
  if (s === 'rejected') return '<span class="chip alert">✗ Refusée</span>';
  return '<span class="chip warn">⌚ En attente</span>';
}

function pageAbsenceRequests() {
  const all = Object.entries(state.absenceRequests || {}).map(([id,r]) => ({...r,
  const tab = state.reqTab || 'pending';
  const filtered = all.filter(r => r.status === tab);
  filtered.sort((a,b) => (b.createdAt||'').localeCompare(a.createdAt||''));

```

```

return `
  <div class="page-head">
    <div>
      <div class="uppercase-eyebrow">Suivi RH</div>
      <h1 class="h-1">Demandes d'absence</h1>
    </div>
  </div>

  <div class="emp-tabs" style="margin-bottom:14px;">
    <button class="emp-tab ${tab==='pending'? 'active':''}" data-req-tab="pendin
    <button class="emp-tab ${tab==='approved'? 'active':''}" data-req-tab="appro
    <button class="emp-tab ${tab==='rejected'? 'active':''}" data-req-tab="rejec
  </div>

  <div class="panel">
    <div class="panel-body ${filtered.length?'nopad':''}">
      ${filtered.length === 0 ? `<div class="text-mute" style="text-align:cente
      <table class="tbl">
        <thead>
          <tr>
            <th>Salarié</th>
            <th>Type</th>
            <th>Dates</th>
            <th>Jours</th>
            <th>Demandée le</th>
            <th>Motif</th>
            ${tab==='pending'? '<th>Actions</th>': '<th>Décision</th>'}
          </tr>
        </thead>
        <tbody>
          ${filtered.map(r => {
            const emp = state.employees.find(e => e.id === r.empId);
            const lt = LEAVE_TYPES[r.type];
            return `
              <tr>
                <td><div class="emp-cell"><div class="av-emp sm">${emp?initia
                <td><span class="chip ${r.status==='approved'? 'good':r.status
                <td>${fmtDateShort(r.dateStart)} → ${fmtDateShort(r.dateEnd)}
                <td class="mono">${r.nbJours||'?'} j</td>
                <td class="mono" style="font-size:11.5px;">${r.createdAt?new
                <td class="text-mute" style="font-size:12px;">${esc(r.motif||
                ${tab==='pending'? `
                  <td>
                    <button class="btn-sec" data-approve="${r.id}" style="pad
                    <button class="btn-sec" data-reject="${r.id}" style="padd
                  </td>
            `
          }
        `
      `
    `
  `

```



```

      : `
      <td>
        <div style="display:flex;flex-direction:column;gap:4px;">
          <span class="text-mute" style="font-size:11.5px;">${r.d
          <div style="display:flex;gap:4px;">
            <button class="btn-ghost" data-edit-req="${r.id}" sty
            <button class="btn-ghost" data-revert-req="${r.id}" s
          </div>
        </div>
      </td>
    `}
  </tr>
  `;
  }).join('')
</tbody>
</table>
`}
</div>
</div>
`;
}

```

```

function bindAbsenceRequests() {
  $$('[data-req-tab]').forEach(b => b.addEventListener('click', ev => {
    state.reqTab = ev.currentTarget.dataset.reqTab;
    render();
  }));
  $$('[data-approve]').forEach(b => b.addEventListener('click', ev => {
    const id = ev.currentTarget.dataset.approve;
    approveAbsenceRequest(id);
  }));
  $$('[data-reject]').forEach(b => b.addEventListener('click', ev => {
    const id = ev.currentTarget.dataset.reject;
    rejectAbsenceRequest(id);
  }));
  $$('[data-edit-req]').forEach(b => b.addEventListener('click', ev => {
    openAbsenceEditor(ev.currentTarget.dataset.editReq);
  }));
  $$('[data-revert-req]').forEach(b => b.addEventListener('click', ev => {
    revertAbsenceRequest(ev.currentTarget.dataset.revertReq);
  }));
}

```

// — Édition d'une demande validée/refusée —

```

function openAbsenceEditor(id) {
  const r = state.absenceRequests[id];
  if (!r) return;

```

```

const emp = state.employees.find(e => e.id === r.empId);
const wasApproved = r.status === 'approved';

const body = `
  <div class="text-mute" style="font-size:12.5px;margin-bottom:14px;line-height
    <strong>${emp?esc(emp.prenom+' '+emp.nom):'(inconnu)'}</strong> · demande c
    ${wasApproved ? 'Les jours déjà validés seront retirés du planning, puis le
  </div>

  <div class="form-grid">
    <div class="field"><label class="field-label">Type</label>
      <select class="input" id="abEditType">
        ${Object.entries(LEAVE_TYPES).map(([k,v]) => `<option value="${k}" ${r.
      </select>
    </div>
    <div class="field"><label class="field-label">Date de début</label><input c
    <div class="field"><label class="field-label">Date de fin</label><input cla
    <div class="field full"><label class="field-label">Motif</label><textarea c
    <div class="field full"><label class="field-label">Commentaire admin</label
  </div>
`;

const footer = `
  <button class="btn-danger" id="abEditDelete" style="margin-right:auto;">Suppr
  <button class="btn-sec" data-close>Annuler</button>
  <button class="btn-pri" id="abEditSave" style="width:auto;">Enregistrer</butt
`;

const { close } = openModal({ title: 'Modifier la demande', body, footer });

$('#abEditSave').addEventListener('click', () => {
  const newType = $('#abEditType').value;
  const newStart = $('#abEditStart').value;
  const newEnd = $('#abEditEnd').value;
  if (!newStart || !newEnd || newEnd < newStart) { toast('Dates invalides', 'er

  const updates = {};

  // Si validée : on retire les anciens shifts d'abord
  if (wasApproved) {
    const oldStart = new Date(r.dateStart);
    const oldEnd = new Date(r.dateEnd);
    for (let d = new Date(oldStart); d <= oldEnd; d = addDays(d, 1)) {
      const wk = weekKey(d);
      const dayIdx = (d.getDay() + 6) % 7;
      const key = `${r.empId}_${dayIdx}_${wk}`;
      delete state.shifts[key];
      updates[`shifts/${key}`] = null;
    }
  }

```

```

    // Pose les nouveaux
    const ns = new Date(newStart);
    const ne = new Date(newEnd);
    for (let d = new Date(ns); d <= ne; d = addDays(d, 1)) {
      const wk = weekKey(d);
      const dayIdx = (d.getDay() + 6) % 7;
      const key = `${r.empId}_${dayIdx}_${wk}`;
      const newSh = [{ leaveType: newType, label: LEAVE_TYPES[newType]?.label |
        state.shifts[key] = newSh;
      updates[`shifts/${key}`] = newSh;
    }
  }

const nbJours = Math.round((new Date(newEnd) - new Date(newStart))/86400000)
const updatedReq = {
  ...r,
  type: newType,
  dateStart: newStart,
  dateEnd: newEnd,
  nbJours,
  motif: $('#abEditMotif').value.trim(),
  commentAdmin: $('#abEditComment').value.trim(),
  modifiedAt: new Date().toISOString(),
  modifiedBy: 'admin',
};
state.absenceRequests[id] = updatedReq;
updates[`absenceRequests/${id}`] = updatedReq;
if (db) db.ref().update(updates).catch(e => console.warn(e));
toast(wasApproved ? 'Demande et planning mis à jour' : 'Demande modifiée', 'g
close();
render();
});

$('#abEditDelete').addEventListener('click', () => {
  if (!confirm('Supprimer définitivement cette demande ?\n\nSi elle était valid
  const updates = {};
  if (wasApproved) {
    const s = new Date(r.dateStart);
    const e_ = new Date(r.dateEnd);
    for (let d = new Date(s); d <= e_; d = addDays(d, 1)) {
      const wk = weekKey(d);
      const dayIdx = (d.getDay() + 6) % 7;
      const key = `${r.empId}_${dayIdx}_${wk}`;
      delete state.shifts[key];
      updates[`shifts/${key}`] = null;
    }
  }
}

```

```

    delete state.absenceRequests[id];
    updates[`absenceRequests/${id}`] = null;
    if (db) db.ref().update(updates).catch(e => console.warn(e));
    toast('Demande supprimée', '');
    close();
    render();
  });
}

// Annule la validation et remet en attente
function revertAbsenceRequest(id) {
  const r = state.absenceRequests[id];
  if (!r) return;
  if (!confirm('Repasser cette demande en "En attente" ?\n\nLes jours validés ser

const updates = {};
if (r.status === 'approved') {
  const s = new Date(r.dateStart);
  const e_ = new Date(r.dateEnd);
  for (let d = new Date(s); d <= e_; d = addDays(d, 1)) {
    const wk = weekKey(d);
    const dayIdx = (d.getDay() + 6) % 7;
    const key = `${r.empId}_${dayIdx}_${wk}`;
    delete state.shifts[key];
    updates[`shifts/${key}`] = null;
  }
}

const updated = { ...r, status: 'pending', decidedAt: null, decidedBy: null, co
state.absenceRequests[id] = updated;
updates[`absenceRequests/${id}`] = updated;
if (db) db.ref().update(updates).catch(e => console.warn(e));
state.reqTab = 'pending';
toast('Demande remise en attente', '');
render();
}

function approveAbsenceRequest(id) {
  const r = state.absenceRequests[id];
  if (!r) return;
  if (!confirm(`Valider cette demande ?\n\nLes jours seront automatiquement marqu

// Inject shifts as leave on each day between dateStart and dateEnd
const start = new Date(r.dateStart);
const end = new Date(r.dateEnd);
const updates = {};
for (let d = new Date(start); d <= end; d = addDays(d, 1)) {
  const wk = weekKey(d);

```

```

    const dayIdx = (d.getDay() + 6) % 7;
    const key = `${r.empId}_${dayIdx}_${wk}`;
    const newSh = [{ leaveType: r.type, label: LEAVE_TYPES[r.type]?.label || r.type, state: { shifts: {} } };
    state.shifts[key] = newSh;
    updates[`shifts/${key}`] = newSh;
  }
  // Update request status
  const updatedReq = { ...r, status: 'approved', decidedAt: new Date().toISOString() };
  state.absenceRequests[id] = updatedReq;
  updates[`absenceRequests/${id}`] = updatedReq;
  if (db) db.ref().update(updates).catch(e => console.warn(e));
  toast('Demande validée et planning mis à jour', 'good');
  render();
}

```

```

function rejectAbsenceRequest(id) {
  const r = state.absenceRequests[id];
  if (!r) return;
  const reason = prompt('Motif du refus (optionnel) :');
  if (reason === null) return;
  const updated = { ...r, status: 'rejected', decidedAt: new Date().toISOString() };
  state.absenceRequests[id] = updated;
  fbSave(`absenceRequests/${id}`, updated);
  toast('Demande refusée', '');
  render();
}

```

```

function openAbsenceCreator(empId) {
  const e = state.employees.find(x => x.id === empId);
  const body = `
    <div class="form-grid">
      <div class="field"><label class="field-label">Type</label>
      <select class="input" id="abType">
        ${Object.entries(LEAVE_TYPES).map(([k,v]) => `<option value="${k}">${v}`)}
      </select>
    </div>
    <div class="field"><label class="field-label">Date de début</label><input class="date">
    <div class="field"><label class="field-label">Date de fin</label><input class="date">
    <div class="field full"><label class="field-label">Motif (optionnel)</label>
    <div class="field full text-mute" style="font-size:12px;">Cette absence sera
  </div>
  `;
  const footer = `<div class="spacer"></div><button class="btn-sec" data-close>Annuler</button>`;
  const { close } = openModal({ title: `Absence pour ${e.prenom} ${e.nom}`, body, $('#abSave').addEventListener('click', () => {
    const start = $('#abStart').value;
    const endVal = $('#abEnd').value;
  })});
}

```

```

    if (!start || !endVal) { toast('Renseigne les dates', 'error'); return; }
    if (endVal < start) { toast('Date de fin avant date de début', 'error'); retu
    const sD = new Date(start), eD = new Date(endVal);
    const nbJours = Math.round((eD - sD)/86400000) + 1;
    const id = 'r' + Date.now();
    const r = {
      empId, type: $('#abType').value,
      dateStart: start, dateEnd: endVal,
      motif: $('#abMotif').value.trim(),
      nbJours,
      status: 'pending', createdAt: new Date().toISOString(),
    };
    state.absenceRequests[id] = r;
    fbSave(`absenceRequests/${id}`, r);
    close();
    approveAbsenceRequest(id);
  });
}

```

// _____ COMPTEURS CP _____

```

function pageCpCompteurs() {
  const actives = state.employees.filter(e => e.statut === 'Actif' && !normEmp(e)
  return `
    <div class="page-head">
      <div>
        <div class="uppercase-eyebrow">Suivi RH</div>
        <h1 class="h-1">Compteurs de congés payés</h1>
      </div>
      <div class="page-actions">
        <button class="btn-sec" id="exportCp">↓ Exporter CSV</button>
      </div>
    </div>

    <div class="panel">
      <div class="panel-body nopad" style="overflow-x:auto;">
        <table class="tbl">
          <thead>
            <tr>
              <th>Salarié</th>
              <th class="mono">Acquis N-1</th>
              <th class="mono">Pris N-1</th>
              <th class="mono">Solde N-1</th>
              <th class="mono">Acquis N</th>
              <th class="mono">Pris N</th>
              <th class="mono">Solde N</th>
              <th class="mono">Solde total</th>
            <th></th>

```

```

    </tr>
  </thead>
  <tbody>
    ${actives.map(e => {
      const n = normEmp(e);
      const sNm1 = (n.cpAcquisNm1||0) - (n.cpPrisNm1||0);
      const sN = (n.cpAcquisN||0) - (n.cpPrisN||0);
      const total = sNm1 + sN;
      return `
        <tr>
          <td><div class="emp-cell"><div class="av-emp sm">${initials(e)}
          <td class="mono tabular">${(n.cpAcquisNm1||0).toFixed(1)}</td>
          <td class="mono tabular">${(n.cpPrisNm1||0).toFixed(1)}</td>
          <td class="mono tabular"><strong>${sNm1.toFixed(1)}</strong></td>
          <td class="mono tabular">${(n.cpAcquisN||0).toFixed(1)}</td>
          <td class="mono tabular">${(n.cpPrisN||0).toFixed(1)}</td>
          <td class="mono tabular"><strong>${sN.toFixed(1)}</strong></td>
          <td class="mono tabular"><strong>${total.toFixed(1)}</strong></td>
          <td><button class="btn-ghost" data-view-emp-cp="${e.id}">Modifi
        </tr>
      `;
    })).join('')}
  </tbody>
</table>
</div>
</div>
`;
}

```

```

function bindCpCompteurs() {
  $('#exportCp').addEventListener('click', () => exportCpCSV());
  $('[data-view-emp-cp]').forEach(b => b.addEventListener('click', ev => {
    state.empDetail = parseInt(ev.currentTarget.dataset.viewEmpCp);
    state.empTab = 'conges';
    state.page = 'employees';
    render();
  }));
}

```

```

function exportCpCSV() {
  const lines = ['Salarié;Acquis N-1;Pris N-1;Solde N-1;Acquis N;Pris N;Solde N;S
  state.employees.filter(e => e.statut === 'Actif').forEach(e => {
    const n = normEmp(e);
    const sNm1 = (n.cpAcquisNm1||0) - (n.cpPrisNm1||0);
    const sN = (n.cpAcquisN||0) - (n.cpPrisN||0);
    lines.push([
      csvEsc(`${e.prenom} ${e.nom}`),

```

```

        (n.cpAcquisNm1||0).toFixed(1).replace('.', ','),
        (n.cpPrisNm1||0).toFixed(1).replace('.', ','),
        sNm1.toFixed(1).replace('.', ','),
        (n.cpAcquisN||0).toFixed(1).replace('.', ','),
        (n.cpPrisN||0).toFixed(1).replace('.', ','),
        sN.toFixed(1).replace('.', ','),
        (sNm1+sN).toFixed(1).replace('.', ','),
    ].join(';'));
});
const blob = new Blob(['\u0000' + lines.join('\r\n')], { type: 'text/csv;chase
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url; a.download = `compteurs_cp_${dateISO(new Date())}.csv`;
document.body.appendChild(a); a.click(); document.body.removeChild(a);
setTimeout(() => URL.revokeObjectURL(url), 1000);
toast('CSV téléchargé', 'good');
}

```

// ————— SUIVI RH (Entrées, Sorties, Fins essai, Profils incomplets) —————

```

function pageRH() {
    const tab = state.rhTab || 'entries';
    return `
        <div class="page-head">
            <div>
                <div class="uppercase-eyebrow">Suivi RH</div>
                <h1 class="h-1">${tab==='entries'?'Entrées':tab==='exits'?'Sorties':tab===
                </div>
            </div>

            <div class="emp-tabs" style="margin-bottom:14px;">
                <button class="emp-tab ${tab==='entries'?'active':''}" data-rh-tab="entries
                <button class="emp-tab ${tab==='exits'?'active':''}" data-rh-tab="exits">So
                <button class="emp-tab ${tab==='essai'?'active':''}" data-rh-tab="essai">Fi
                <button class="emp-tab ${tab==='incomplete'?'active':''}" data-rh-tab="inco
            </div>

            ${tab === 'entries' ? rhEntries() :
            tab === 'exits' ? rhExits() :
            tab === 'essai' ? rhEssai() :
            rhIncomplete()}
        `;
}

```

```

function bindRH() {
    $$('[data-rh-tab]').forEach(b => b.addEventListener('click', ev => {
        state.rhTab = ev.currentTarget.dataset.rhTab;
        render();
    }

```



```

    }));
    $$('[data-rh-view]').forEach(b => b.addEventListener('click', ev => {
        state.empDetail = parseInt(ev.currentTarget.dataset.rhView);
        state.empTab = 'info';
        state.page = 'employees';
        render();
    }));
}

// ----- PAGE RAPPELS / ÉCHÉANCES -----
function pageRappels() {
    const rappels = computeRappels();
    const byEmp = {};
    rappels.forEach(r => {
        if (!byEmp[r.empId]) byEmp[r.empId] = { empLabel: r.empLabel, items: [] };
        byEmp[r.empId].items.push(r);
    });

    const alertCount = rappels.filter(r => r.severity === 'alert').length;
    const warnCount = rappels.filter(r => r.severity === 'warn').length;
    const infoCount = rappels.filter(r => r.severity === 'info').length;

    if (rappels.length === 0) {
        return `
            <div class="page-head">
                <div>
                    <div class="uppercase-eyebrow">Conformité</div>
                    <h1 class="h-1">Rappels et échéances</h1>
                </div>
            </div>
            <div class="panel">
                <div class="panel-body" style="text-align:center;padding:64px 24px;">
                    <div style="font-size:56px;line-height:1;margin-bottom:16px;">🌟</div>
                    <h2 class="h-2" style="margin-bottom:8px;">Tout est à jour</h2>
                    <div class="text-mute">Aucune échéance à anticiper sur les 90 prochains
                </div>
            </div>
        `;
    }

    return `
        <div class="page-head">
            <div>
                <div class="uppercase-eyebrow">Conformité</div>
                <h1 class="h-1">Rappels et échéances</h1>
                <div class="text-mute" style="margin-top:6px;font-size:13px;">Anticipatio
            </div>
        </div>
    `;
}

```

```

</div>

<div class="kpi-grid" style="grid-template-columns:repeat(3,1fr);">
  <div class="kpi alert">
    <div class="kpi-label">Urgent</div>
    <div class="kpi-value">${alertCount}</div>
    <div class="kpi-meta">à traiter immédiatement</div>
  </div>
  <div class="kpi warn">
    <div class="kpi-label">À anticiper</div>
    <div class="kpi-value">${warnCount}</div>
    <div class="kpi-meta">échéance proche</div>
  </div>
  <div class="kpi">
    <div class="kpi-label">À surveiller</div>
    <div class="kpi-value">${infoCount}</div>
    <div class="kpi-meta">informationnel</div>
  </div>
</div>

<div class="panel">
  <div class="panel-head"><h3>Échéances triées par urgence</h3></div>
  <div class="panel-body nopad">
    ${rappels.map(r => `
      <div class="rappel-row sev-${r.severity}">
        <div class="rappel-icon">${RAPPEL_ICONS[r.type] || '🕒'}</div>
        <div class="rappel-body">
          <div class="rappel-head">
            <span class="rappel-emp">${esc(r.empLabel)}</span>
            <span class="chip ${r.severity==='alert'? 'alert':r.severity==='wa
              ${r.daysLeft} >= 0
              ? `<span class="text-mute" style="font-size:12px;">${r.daysLeft
                : `<span style="font-size:12px;color:var(--c-alert-text);font-w
              </div>
            <div class="rappel-detail">${esc(r.detail)}</div>
            <div class="rappel-action">➔ ${esc(r.action)}</div>
          </div>
          <button class="btn-ghost" data-open-emp="${r.empId}" title="Ouvrir la
        </div>
      `).join('')}
    </div>
  </div>
</div>
`;
}

function bindRappels() {
  $$('[data-open-emp]').forEach(b => b.addEventListener('click', ev => {

```

```

    state.empDetail = parseInt(ev.currentTarget.dataset.openEmp);
    state.empTab = 'contrats';
    state.page = 'employees';
    render();
  }));
}

// ----- ÉMARGEMENTS -----
// ----- ÉMARGEMENTS - Prévisionnel + Effectif + Invalidation auto -----
// Calcule un hash simple du planning d'un salarié pour une semaine donnée.
// Sert à détecter si le planning a changé après émargement → invalide la signatu
function weeklyPlanningHash(empId, wkKey) {
  const parts = [];
  for (let d = 0; d < 7; d++) {
    const shifts = (state.shifts[`${empId}_${d}_${wkKey}`] || []).map(s => {
      if (s.leaveType) return `L:${s.leaveType}`;
      return `S:${s.type}||':${s.start}||':${s.end}||':${s.pauseDuration||0}:$
    });
    parts.push(d + '=' + shifts.join('|'));
  }
  // Hash simple (somme de codes de caractères × position)
  const str = parts.join(';');
  let h = 0;
  for (let i = 0; i < str.length; i++) {
    h = ((h << 5) - h + str.charCodeAt(i)) | 0;
  }
  return h.toString(36);
}

function emargementStatus(empId, wkKey) {
  const sigKey = `${empId}_${wkKey}`;
  const sig = state.emargements && state.emargements[sigKey];
  const currentHash = weeklyPlanningHash(empId, wkKey);
  if (!sig) return { state: 'unsigned', hash: currentHash };
  if (sig.planningHash && sig.planningHash !== currentHash) {
    return { state: 'invalidated', sig, currentHash, oldHash: sig.planningHash };
  }
  return { state: 'signed', sig, hash: currentHash };
}

function emargementEffectifStatus(empId, wkKey) {
  const key = `${empId}_${wkKey}_effectif`;
  const sig = state.emargements && state.emargements[key];
  // Only available after the week ended
  const wkDate = new Date(wkKey + 'T00:00:00');
  const wkEnd = addDays(wkDate, 6); wkEnd.setHours(23,59,59,999);
  const now = new Date();

```

```

    if (now < wkEnd) return { state: 'too_early', wkEnd };
    if (sig) return { state: 'signed', sig };
    return { state: 'unsigned' };
}

```

```

function countUnsignedEmargements() {
    let count = 0;
    const publishedWeeks = Object.keys(state.publications || {});
    const actives = state.employees.filter(e => e.statut === 'Actif');
    publishedWeeks.forEach(wk => {
        actives.forEach(emp => {
            const hasShifts = [0,1,2,3,4,5,6].some(d => (state.shifts[`${emp.id}_${d}_${wk}`]));
            if (!hasShifts) return;
            const prev = emargementStatus(emp.id, wk);
            const eff = emargementEffectifStatus(emp.id, wk);
            if (prev.state !== 'signed') count++;
            if (eff.state === 'unsigned') count++;
        });
    });
    return count;
}

```

```

function pageEmargements() {
    const publishedWeeks = Object.keys(state.publications || {})
        .sort((a, b) => b.localeCompare(a))
        .slice(0, 12);

    const actives = state.employees.filter(e => e.statut === 'Actif');

    return `
        <div class="page-head">
            <div>
                <div class="uppercase-eyebrow">Conformité légale</div>
                <h1 class="h-1">Émargements des plannings</h1>
                <div class="text-mute" style="margin-top:6px;font-size:13px;">Signature é
            </div>
        </div>

        <div class="panel" style="background:var(--c-paper);border:1px solid var(--c-
            <div class="panel-body" style="padding:16px 22px;">
                <div style="display:flex;align-items:flex-start;gap:14px;">
                    <div style="font-size:24px;line-height:1;">i</div>
                    <div style="flex:1;font-size:13px;color:var(--c-ink-3);line-height:1.55"
                        <strong>Deux émargements par semaine</strong> pour couvrir l'article
                        <ul style="margin:8px 0 0 18px;">
                            <li><strong>Prévisionnel</strong> : à la publication du planning. S
                            <li><strong>Effectif</strong> : disponible côté salarié à partir du

```

```

        </ul>
      </div>
    </div>
  </div>
</div>

${publishedWeeks.length === 0 ? `
  <div class="panel"><div class="panel-body" style="text-align:center;padding
    <div class="text-mute">Aucun planning publié pour le moment.</div>
  </div></div>
` : `
  <div class="panel">
    <div class="panel-head"><h3>Suivi des émargements · 12 dernières semaines
    <div class="panel-body nopad" style="overflow-x:auto;">
      <table class="tbl tbl-compact">
        <thead>
          <tr>
            <th rowspan="2" style="vertical-align:middle;">Semaine</th>
            <th rowspan="2" style="vertical-align:middle;">Publication</th>
            ${actives.map(e => `<th colspan="2" style="text-align:center;font
              <th rowspan="2" style="vertical-align:middle;">État global</th>
            </tr>
            <tr>
              ${actives.map(() => `<th style="text-align:center;font-size:9.5px
            </tr>
          </thead>
          <tbody>
            ${publishedWeeks.map(wk => {
              const pub = state.publications[wk];
              const wkDate = new Date(wk + 'T00:00:00');
              const wkEnd = addDays(wkDate, 6);
              let signedPrev = 0, signedEff = 0, expectedPrev = 0, expectedEff
              const cells = actives.map(emp => {
                const hasShifts = [0,1,2,3,4,5,6].some(d => (state.shifts[`${em
                  if (!hasShifts) return `<td style="text-align:center;color:var(
                    expectedPrev++;
                  // Prévisionnel
                  const prev = emargementStatus(emp.id, wk);
                  let prevCell;
                  if (prev.state === 'signed') {
                    signedPrev++;
                    prevCell = `<td style="text-align:center;color:var(--c-good-d
                  } else if (prev.state === 'invalidated') {
                    prevCell = `<td style="text-align:center;color:var(--c-warn-d
                  } else {
                    prevCell = `<td style="text-align:center;color:var(--c-warn-d
                  }
                }
              }
            }
          }
        }
      }
    }
  }
`

```

```

// Effectif
const eff = emargementEffectifStatus(emp.id, wk);
let effCell;
if (eff.state === 'signed') {
  signedEff++;
  expectedEff++;
  effCell = `  |
```

```

    { key: 'urgence',          label: 'Numéros d\'urgence',          detai
    { key: 'harcèlement',      label: 'Lutte contre le harcèlement',      detai
    { key: 'discrimination',   label: 'Lutte contre les discriminations',   detai
    { key: 'egalite',          label: 'Égalité professionnelle H/F',          deta
    { key: 'reglement',        label: 'Règlement intérieur',        detai
    { key: 'duerp_dispo',       label: 'Modalités d\'accès au DUERP',       detai
    { key: 'fumeur',           label: 'Interdiction de fumer',           detai
    { key: 'cse_election',      label: 'Élection CSE',                  deta
  ];

```

```

function pageEtablissement() {
  const actives = state.employees.filter(e => e.statut === 'Actif');
  const haccpTeam = actives.filter(e => normEmp(e).certifHACCP);
  const haccpOK = haccpTeam.length >= 1;
  const cseSeuil = actives.length >= 11;
  const duerp = state.duerp || null;
  const today = new Date();
  const duerpEcheance = duerp && duerp.updatedAt ? (() => {
    const d = new Date(duerp.updatedAt);
    d.setFullYear(d.getFullYear() + 1);
    return d;
  })() : null;
  const duerpOK = duerp && duerpEcheance && duerpEcheance >= today;
  const affichagesCount = Object.values(state.affichages || {}).filter(a => a &&

  return `
    <div class="page-head">
      <div>
        <div class="uppercase-eyebrow">Conformité</div>
        <h1 class="h-1">Établissement</h1>
        <div class="text-mute" style="margin-top:6px;font-size:13px;">Obligations
      </div>
    </div>

    <div class="kpi-grid" style="grid-template-columns:repeat(4,1fr);">
      <div class="kpi ${duerpOK?'':'alert'}">
        <div class="kpi-label">DUERP</div>
        <div class="kpi-value" style="font-size:24px;">${duerpOK ? '✓' : '✗'}</di
        <div class="kpi-meta">${duerpOK ? `À renouveler le ${fmtDateShort(dateISO
      </div>
      <div class="kpi ${haccpOK?'':'warn'}">
        <div class="kpi-label">Personnes HACCP</div>
        <div class="kpi-value">${haccpTeam.length}</div>
        <div class="kpi-meta">${haccpOK ? haccpTeam.map(e => e.prenom).join(', '
      </div>
      <div class="kpi">
        <div class="kpi-label">Affichages OK</div>

```

```

    <div class="kpi-value">${affichagesCount}<span style="font-size:18px;colo
    <div class="kpi-meta">Vérification annuelle conseillée</div>
  </div>
  <div class="kpi ${cseSeuil?'warn':''}">
    <div class="kpi-label">Seuil CSE</div>
    <div class="kpi-value">${actives.length}</div>
    <div class="kpi-meta">${cseSeuil ? '> 11 → CSE obligatoire' : 'Sous le se
  </div>
</div>

<div class="panel">
  <div class="panel-head">
    <h3>Document Unique d'Évaluation des Risques (DUERP)</h3>
    <button class="btn-sec" id="updateDuerp">${duerp ? 'Marquer comme à jour'
  </div>
  <div class="panel-body">
    <div style="font-size:13px;color:var(--c-ink-3);line-height:1.55;margin-b
      Le DUERP est obligatoire dès le 1er salarié (article R4121-1). Il doit
    </div>
    ${duerp ? `
      ${kvRow('Dernière mise à jour', fmtDateShort(duerp.updatedAt))}
      ${kvRow('Prochaine échéance', `<span class="chip ${duerpOK?'good':'aler
      ${duerp.note ? kvRow('Note', esc(duerp.note)) : ''}
      ` : `<div class="text-mute" style="text-align:center;padding:16px 0;">Auc
    </div>
  </div>
</div>

<div class="panel" style="margin-top:14px;">
  <div class="panel-head">
    <h3>Affichages obligatoires</h3>
    <span class="text-mute" style="font-size:11.5px;">À cocher après vérifica
  </div>
  <div class="panel-body nopad">
    <table class="tbl">
      <thead><tr><th>Affichage</th><th>Détail</th><th>Présent</th><th>Vérifié
      <tbody>
        ${AFFICHAGES_OBLIGATOIRES.map(aff => {
          const state_aff = (state.affichages || {})[aff.key];
          const present = state_aff && state_aff.present;
          return `
            <tr>
              <td><strong>${esc(aff.label)}</strong></td>
              <td class="text-mute" style="font-size:12px;">${esc(aff.detail)
              <td><label style="display:flex;align-items:center;gap:6px;curso
                <input type="checkbox" data-aff="${aff.key}" ${present?'check
                <span class="chip ${present?'good':''}>${present?'✓ Présent'
              </label></td>

```



```
 >${state_af |
```

```

</tr>
`;
}).join('')}
</tbody>
</table>
</div>
</div>

<div class="panel" style="margin-top:14px;">
  <div class="panel-head"><h3>Équipe titulaire de la formation HACCP</h3></di
  <div class="panel-body">
    <div style="font-size:13px;color:var(--c-ink-3);line-height:1.55;margin-b
      Obligation depuis le décret n°2011-731 : <strong>au moins une personne<
    </div>
    ${haccpTeam.length === 0 ? `
      <div class="info-banner" style="background:var(--c-alert-bg);border-lef
        ⚠️ Aucun salarié n'est marqué comme titulaire HACCP. <strong>Obligati
      </div>
      ` : `
      <table class="tbl">
        <thead><tr><th>Salarié</th><th>Date formation</th></tr></thead>
        <tbody>
          ${haccpTeam.map(e => {
            const n = normEmp(e);
            return `
              <tr>
                <td><strong>${esc(e.prenom)} ${esc(e.nom)}</strong> – ${esc(e
                <td>${n.certifHACCPDate ? fmtDateShort(n.certifHACCPDate) : '
              </tr>
            `;
          }).join('')}
        </tbody>
      </table>
    `}
  </div>
</div>

```

```

${cseSeuil ? `
  <div class="panel" style="margin-top:14px;">
    <div class="panel-head"><h3>Comité Social et Économique (CSE)</h3></div>
    <div class="panel-body">
      <div class="info-banner" style="background:var(--c-warn-bg);border-left
        ⚠️ <strong>Tu as atteint le seuil de 11 salariés.</strong> Si cet eff
      <ul style="margin:10px 0 0 18px;">
        <li>Informar les salariés de l'organisation des élections (LR + aff
        <li>Inviter les organisations syndicales représentatives</li>
      </ul>
    </div>
  </div>

```

```

        <li>Organiser les élections dans les 90 jours</li>
        <li>Mettre en place le CSE et tenir au moins une réunion mensuelle<
    </ul>
    <div style="margin-top:10px;">Sanction si défaut : délit d'entrave pu
    </div>
  </div>
  </div>
  ` : ''}

<div class="panel" style="margin-top:14px;background:var(--c-paper);">
  <div class="panel-head"><h3>Notifications push & WhatsApp aux salariés</h3>
  <div class="panel-body">
    <div style="font-size:13px;color:var(--c-ink-3);line-height:1.55;">
      Les notifications push web nécessitent un backend dédié (Firebase Cloud
      <ul style="margin:10px 0 0 18px;">
        <li><strong>FCM via Firebase Cloud Functions</strong> (gratuit jusqu'
        <li><strong>WhatsApp Business API</strong> via Twilio ou Vonage : ~0,
        <li><strong>SMS basique</strong> via Twilio/OVH : ~0,06€ par SMS. Le
      </ul>
      <div style="margin-top:10px;">Pour le moment l'app est en mode <strong>
    </div>
  </div>
</div>
`;
}

function bindEtablissement() {
  $('#updateDuerp')?.addEventListener('click', () => openDuerpEditor());
  $('[data-aff]').forEach(cb => cb.addEventListener('change', ev => {
    const key = ev.currentTarget.dataset.aff;
    const present = ev.currentTarget.checked;
    state.affichages = state.affichages || {};
    state.affichages[key] = { present, updatedAt: new Date().toISOString() };
    if (db) db.ref(`affichages/${key}`).set(state.affichages[key]);
    toast(present ? 'Affichage marqué présent' : 'Affichage retiré', '');
  }));
}

function openDuerpEditor() {
  const current = state.duerp || {};
  const todayISO = dateISO(new Date());
  const body = `
    <div style="font-size:13px;color:var(--c-ink-3);line-height:1.55;margin-botto
      Marque ici la mise à jour annuelle du DUERP. Le document physique (ou PDF)
    </div>
    <div class="form-grid">
      <div class="field full"><label class="field-label">Date de mise à jour</lab

```

```

    <div class="field full"><label class="field-label">Note interne (optionnel)
  </div>
`;
const footer = `<button class="btn-sec" data-close>Annuler</button><button clas
const { close } = openModal({ title: 'Mise à jour DUERP', body, footer });
$('#duerpSave').addEventListener('click', () => {
  const updated = {
    updatedAt: new Date($('#duerpDate').value + 'T00:00:00').toISOString(),
    note: $('#duerpNote').value.trim(),
  };
  state.duerp = updated;
  if (db) db.ref('duerp').set(updated);
  toast('DUERP mis à jour', 'good');
  close();
  render();
});
}

// ————— PARAMÈTRES ADMIN — Identifiant + mot de passe —————
function openAdminSettings() {
  const current = state.adminCredentials;
  const usingDefault = !current;
  const body = `
    <div class="text-mute" style="font-size:13px;line-height:1.55;margin-bottom:1
      ${usingDefault
        ? `<div class="info-banner" style="background:var(--c-warn-bg);border-lef
          : `Identifiant actuel : <strong>${esc(current.username)}</strong> · Derni
        </div>

    <div class="modal-section-title">Vérification</div>
    <div class="form-grid">
      <div class="field full">
        <label class="field-label">Mot de passe actuel</label>
        <input class="input mono" id="asCurrentPwd" type="password" autocomplete=
      </div>
    </div>

    <div class="modal-section-title">Nouveaux identifiants</div>
    <div class="form-grid">
      <div class="field full">
        <label class="field-label">Nouvel identifiant</label>
        <input class="input" id="asNewUser" value="${esc((current?current.usernam
        <div class="text-mute" style="font-size:11.5px;margin-top:4px;">Sera mis
      </div>
    </div>
    <div class="field">
      <label class="field-label">Nouveau mot de passe</label>
      <input class="input mono" id="asNewPwd" type="password" autocomplete="new

```

```

    </div>
    <div class="field">
      <label class="field-label">Confirmer le mot de passe</label>
      <input class="input mono" id="asNewPwd2" type="password" autocomplete="ne
    </div>
  </div>

  <div class="info-banner" style="background:var(--c-paper);border:1px solid va
     Le mot de passe est stocké sous forme de <strong>hash SHA-256</strong> c
  </div>
`;
const footer = `
  <button class="btn-sec" data-close>Annuler</button>
  <button class="btn-pri" id="asSave" style="width:auto;">Enregistrer</button>
`;
const { close } = openModal({ title: 'Paramètres administrateur', body, footer

$('#asSave').addEventListener('click', async () => {
  const currentPwd = $('#asCurrentPwd').value.trim();
  const newUser = $('#asNewUser').value.trim().toLowerCase();
  const newPwd = $('#asNewPwd').value.trim();
  const newPwd2 = $('#asNewPwd2').value.trim();

  if (!currentPwd) { toast('Saisis le mot de passe actuel', 'error'); return; }
  if (!newUser) { toast('Identifiant requis', 'error'); return; }
  if (!newPwd) { toast('Nouveau mot de passe requis', 'error'); return; }
  if (newPwd.length < 4) { toast('Mot de passe trop court (min 4 caractères)',
  if (newPwd !== newPwd2) { toast('Les deux mots de passe ne correspondent pas'

  // Vérifier le mot de passe actuel
  let currentOk = false;
  if (current && current.passwordHash) {
    const hashCurrent = await sha256(currentPwd);
    currentOk = hashCurrent === current.passwordHash;
  } else {
    currentOk = currentPwd === ADMIN_CODE;
  }
  if (!currentOk) {
    toast('Mot de passe actuel incorrect', 'error');
    return;
  }

  // Hasher et sauvegarder
  const newHash = await sha256(newPwd);
  const updated = {
    username: newUser,
    passwordHash: newHash,

```

```

        updatedAt: new Date().toISOString(),
    };
    state.adminCredentials = updated;
    if (db) {
        try {
            await db.ref('adminCredentials').set(updated);
            toast(`Identifiants mis à jour. Reconnecte-toi avec "${newUser}".`, 'good');
            close();
            // Force logout pour utiliser les nouveaux credentials
            setTimeout(() => logout(), 1500);
        } catch (e) {
            toast('Erreur sauvegarde Firebase : ' + (e.message||e.code), 'error', 600);
        }
    } else {
        toast('Firebase non disponible – changement non persistant', 'error');
    }
});
}

```

```

// ————— PACK SORTIE – STC + Certif + Attestation France Travail —————
function openPackSortie(empId) {
    const emp = state.employees.find(e => e.id === empId);
    if (!emp) return;
    const n = normEmp(emp);

    const today = new Date();
    const todayISO = dateISO(today);

    const body = `
        <div class="text-mute" style="font-size:13px;line-height:1.55;margin-bottom:20px">
            Génère les 3 documents légaux obligatoires à remettre au salarié à sa sortie :
            <strong>certificat de travail</strong>, <strong>solde de tout compte</strong>
        </div>

        <div class="form-grid">
            <div class="field full">
                <label class="field-label">Salarié</label>
                <input class="input" value="${esc(emp.prenom)} ${esc(emp.nom)}" readonly>
            </div>
            <div class="field">
                <label class="field-label">Date de sortie</label>
                <input class="input" id="psDateSortie" type="date" value="${n.dateSortie}">
            </div>
            <div class="field">
                <label class="field-label">Motif de sortie</label>

```

```

    <select class="input" id="psMotif">
      <option ${n.motifSortie=== 'Démission'? 'selected': ''}>Démission</option>
      <option ${n.motifSortie=== 'Fin de CDD'? 'selected': ''}>Fin de CDD</optio
      <option ${n.motifSortie=== 'Rupture conventionnelle'? 'selected': ''}>Rupt
      <option ${n.motifSortie=== 'Licenciement'? 'selected': ''}>Licenciement</o
      <option ${n.motifSortie=== 'Fin de période d\'essai'? 'selected': ''}>Fin
      <option ${n.motifSortie=== 'Retraite'? 'selected': ''}>Retraite</option>
      <option ${n.motifSortie=== 'Décès'? 'selected': ''}>Décès</option>
    </select>
  </div>
  <div class="field">
    <label class="field-label">Solde CP restant à payer (j)</label>
    <input class="input mono" id="psSoldeCp" type="number" step="0.5" value="
  </div>
  <div class="field">
    <label class="field-label">Indemnité de précarité (€)</label>
    <input class="input mono" id="psPrecarite" type="number" step="0.01" valu
    <div class="text-mute" style="font-size:11px;margin-top:4px;">10% du brut
  </div>
  <div class="field">
    <label class="field-label">Indemnité légale licenciement (€)</label>
    <input class="input mono" id="psIndemLic" type="number" step="0.01" value
  </div>
  <div class="field full">
    <label class="field-label">Documents à générer</label>
    <div style="display:flex;gap:8px;flex-wrap:wrap;">
      <label style="display:flex;align-items:center;gap:6px;font-size:13px;cu
      <label style="display:flex;align-items:center;gap:6px;font-size:13px;cu
      <label style="display:flex;align-items:center;gap:6px;font-size:13px;cu
    </div>
  </div>
</div>

<div class="info-banner" style="margin-top:14px;background:var(--c-warn-bg);b
  <strong>Note :</strong> ces documents sont générés en HTML imprimable (Cmd/
</div>
`;
const footer = `
  <button class="btn-sec" data-close>Annuler</button>
  <button class="btn-pri" id="psGenerate" style="width:auto;">Générer les docum
`;

const { close } = openModal({ title: 'Pack de sortie', body, footer });

$('#psGenerate').addEventListener('click', () => {
  const data = {
    emp,

```

```

    dateSortie: $('#psDateSortie').value,
    motif: $('#psMotif').value,
    soldeCp: parseFloat($('#psSoldeCp').value) || 0,
    precarite: parseFloat($('#psPrecarite').value) || 0,
    indemLic: parseFloat($('#psIndemLic').value) || 0,
  };
  const docs = {
    certif: $('#psDocCertif').checked,
    stc: $('#psDocStc').checked,
    attest: $('#psDocAttest').checked,
  };
  // Save sortie info on employee
  const updated = { ...emp, dateSortie: data.dateSortie, motifSortie: data.moti
  state.employees = state.employees.map(x => x.id === emp.id ? updated : x);
  fbSave('employees', state.employees);

  generatePackSortieDocuments(data, docs);
  close();
});
}

```

```

function generatePackSortieDocuments(data, docs) {
  const { emp, dateSortie, motif, soldeCp, precarite, indemLic } = data;
  const n = normEmp(emp);
  const win = window.open('', '_blank');
  if (!win) { toast('Pop-up bloquée – autorise les pop-ups', 'error'); return; }
  const dateSortieFmt = new Date(dateSortie).toLocaleDateString('fr-FR', {day:'nu
  const dateDebutFmt = n.contratDebut ? new Date(n.contratDebut).toLocaleDateStri
  const todayFmt = new Date().toLocaleDateString('fr-FR', {day:'numeric',month:'l
  const empNom = `${emp.prenom} ${emp.nom}`;
  const adresseSal = [n.adresse, n.complementAdresse, `${n.codePostal} ${n.ville}

  const css = `
    body { font-family: Georgia, serif; max-width: 720px; margin: 30px auto; padd
    h1 { font-size: 22px; margin: 30px 0 16px; border-bottom: 2px solid #1a1817;
    h2 { font-size: 16px; margin-top: 20px; }
    .doc-section { page-break-after: always; padding-bottom: 40px; }
    .doc-section:last-child { page-break-after: auto; }
    .header { display: flex; justify-content: space-between; margin-bottom: 24px;
    .from { font-size: 12.5px; }
    .to { text-align: right; font-size: 12.5px; }
    .signature-line { margin-top: 50px; display: flex; justify-content: space-bet
    .signature-line .box { width: 240px; }
    .signature-line .box strong { display: block; margin-bottom: 40px; font-size:
    .signature-line .line { border-top: 1px solid #555; margin-top: 30px; padding
    table { width: 100%; border-collapse: collapse; margin: 14px 0; font-size: 13
    table th, table td { padding: 8px 10px; border: 1px solid #d0c8b8; text-align
  `;

```

```

    table th { background: #f5f2ea; }
    table .num { text-align: right; font-family: monospace; }
    @media print { body { margin: 0; padding: 18mm; } @page { margin: 0; size: A4
`;

```

```

let html = `<!doctype html><html><head><meta charset="utf-8"><title>Pack sortie

```

```

if (docs.certif) {
    html += `
        <div class="doc-section">
            <div class="header">
                <div class="from">
                    <strong>SAS Man'ouché</strong><br>
                    ${esc(emp.adresseRestau || ' ')}<br>
                    SIRET / RCS
                </div>
                <div class="to">
                    <strong>${esc(empNom)}</strong><br>
                    ${esc(adresseSal)}
                </div>
            </div>
            <h1>Certificat de travail</h1>
            <p>Je soussigné, gérant de la société Man'ouché, certifie que <strong>${e
            <ul>
                <li>en qualité de <strong>${esc(emp.poste || '-')}</strong></li>
                <li>du <strong>${dateDebutFmt}</strong> au <strong>${dateSortieFmt}</st
                <li>sous contrat de type <strong>${esc(emp.contrat || '-')}</strong></li>
            </ul>
            <p>${esc(empNom)} nous quitte ce jour libre de tout engagement.</p>
            <p>Le présent certificat est délivré pour servir et valoir ce que de droi
            <p style="margin-top:24px;">Fait à Paris, le ${todayFmt}.</p>
            <div class="signature-line">
                <div class="box"><strong>Signature de l'employeur</strong><div class="1
            </div>
        </div>
    `;
}

```

```

if (docs.stc) {
    const totalDu = soldeCp * (n.taux || 12) * 7 + precarite + indemLic;
    html += `
        <div class="doc-section">
            <div class="header">
                <div class="from">
                    <strong>SAS Man'ouché</strong><br>
                    SIRET / RCS
                </div>

```



```

        <div class="to">
            <strong>${esc(empNom)}</strong><br>
            ${esc(adresseSal)}
        </div>
    </div>
<h1>Reçu pour solde de tout compte</h1>
<p>Je soussigné(e), <strong>${esc(empNom)}</strong>, reconnais avoir reçu
<table>
    <tr><th>Élément de rémunération</th><th class="num">Montant brut (€)</th></tr>
    <tr><td>Solde de congés payés (${soldeCp.toFixed(1)} jours)</td><td class="num">${precarite > 0 ? `<tr><td>Indemnité de fin de contrat (précarité)</td><td class="num">${indemLic > 0 ? `<tr><td>Indemnité légale de licenciement</td><td class="num">${totalDu.toFixed(2)}</td></tr>
</table>
<p style="font-size:12px;font-style:italic;color:#555;">Le présent reçu p
<p style="margin-top:24px;">Établi en deux exemplaires originaux, à Paris
<div class="signature-line">
    <div class="box"><strong>Signature du salarié</strong> <span style="font-size:12px;font-style:italic;color:#555;">
    <div class="box"><strong>Signature de l'employeur</strong><div class="box">
</div>
</div>
`;
}

if (docs.attest) {
    html += `
        <div class="doc-section">
            <h1>Attestation France Travail</h1>
            <p style="font-size:13px;color:#666;font-style:italic;">Document brouillon
            <h2>Identification du salarié</h2>
            <p>
                <strong>${esc(empNom)}</strong><br>
                Né(e) le ${n.dateNaissance ? new Date(n.dateNaissance).toLocaleDateString() : ''}
                ${esc(adresseSal)}<br>
                Nationalité : ${esc(n.nationalite || '-')}<br>
            </p>
            <h2>Identification de l'employeur</h2>
            <p>
                <strong>SAS Man'ouché</strong><br>
                SIRET : à renseigner<br>
                Code NAF : 5610A (Restauration traditionnelle)<br>
                Convention collective : HCR (IDCC 1979)
            </p>
            <h2>Période d'emploi</h2>
            <ul>
                <li>Du <strong>${dateDebutFmt}</strong> au <strong>${dateSortieFmt}</strong>
                <li>Emploi occupé : <strong>${esc(emp.poste || '-')}</strong></li>
            </ul>
        </div>
    `;
}

```

```

        <li>Type de contrat : <strong>${esc(emp.contrat || '-')}</strong></li>
        <li>Durée hebdomadaire : <strong>${emp.heures || 35} h</strong></li>
    </ul>
    <h2>Motif de la rupture</h2>
    <p><strong>${esc(motif)}</strong></p>
    <h2>Salaires des 12 derniers mois (à compléter par le comptable)</h2>
    <p style="font-size:12px;color:#666;">Cette section sera renseignée depuis
    <p style="margin-top:24px;">Fait à Paris, le ${todayFmt}.</p>
    <div class="signature-line">
        <div class="box"><strong>Signature de l'employeur</strong><div class="l
    </div>
</div>
`;
}

html += `</body></html>`;
win.document.write(html);
win.document.close();
setTimeout(() => win.print(), 600);
toast('Documents prêts - Cmd+P pour imprimer/PDF', 'good', 5000);
}

function rhEntries() {
    // Entrées : salariés avec contratDebut dans les 90 derniers jours
    const now = new Date();
    const cutoff = addDays(now, -90);
    const list = state.employees.map(normEmp).filter(n => {
        if (!n.contratDebut) return false;
        const d = new Date(n.contratDebut);
        return d >= cutoff && d <= addDays(now, 30);
    }).sort((a,b) => (b.contratDebut||'').localeCompare(a.contratDebut||''));
    return rhTable(list, [
        { label: 'Date d\'entrée', val: e => fmtDateShort(e.contratDebut) },
        { label: 'Salarié', val: e => `${e.prenom} ${e.nom}` },
        { label: 'Contrat', val: e => e.contrat||'-' },
        { label: 'Poste', val: e => e.poste||'-' },
        { label: 'Email', val: e => e.email||'-' },
        { label: 'Tél.', val: e => e.telMobile||'-' },
    ], 'Aucune entrée dans les 90 derniers jours');
}

function rhExits() {
    const list = state.employees.map(normEmp).filter(n => n.dateSortie).sort((a,b)
    return rhTable(list, [
        { label: 'Date de sortie', val: e => fmtDateShort(e.dateSortie) },
        { label: 'Salarié', val: e => `${e.prenom} ${e.nom}` },
        { label: 'Motif', val: e => e.motifSortie||'-' },
    ]

```

```

    { label: 'Contrat', val: e => e.contrat||'-' },
    { label: 'Email', val: e => e.email||'-' },
  ], 'Aucune sortie enregistrée');
}

```

```

function rhEssai() {
  // Salariés en période d'essai : contratDebut + periodeEssaiJours > today
  const now = new Date();
  const cutoff = addDays(now, 60); // afficher ceux dont la fin d'essai arrive da
  const list = state.employees.map(normEmp).filter(n => {
    if (!n.contratDebut || !n.periodeEssaiJours) return false;
    const finEssai = addDays(new Date(n.contratDebut), n.periodeEssaiJours);
    return finEssai >= addDays(now, -7) && finEssai <= cutoff;
  }).map(n => ({...n, finEssai: addDays(new Date(n.contratDebut), n.periodeEssaiJ
    .sort((a,b) => a.finEssai - b.finEssai);
  return rhTable(list, [
    { label: 'Fin période essai', val: e => fmtDateShort(dateISO(e.finEssai)) },
    { label: 'Date d\'entrée', val: e => fmtDateShort(e.contratDebut) },
    { label: 'Salarié', val: e => `${e.prenom} ${e.nom}` },
    { label: 'Contrat', val: e => e.contrat||'-' },
  ], 'Aucune fin de période d\'essai à venir');
}

```

```

function rhIncomplete() {
  const list = state.employees.map(normEmp).filter(n => !isProfileComplete(n));
  return rhTable(list, [
    { label: 'Salarié', val: e => `${e.prenom} ${e.nom}` },
    { label: 'Manque email', val: e => e.email?'✓':<span style="color:var(--c-al
    { label: 'Manque tél.', val: e => e.telMobile?'✓':<span style="color:var(--c
    { label: 'Manque date naiss.', val: e => e.dateNaissance?'✓':<span style="co
    { label: 'Manque adresse', val: e => e.adresse?'✓':<span style="color:var(--
    { label: 'Manque CP/ville', val: e => (e.codePostal && e.ville)?'✓':<span st
    { label: 'Manque contrat', val: e => e.contratDebut?'✓':<span style="color:v
  ], 'Tous les profils sont complets ✓');
}

```

```

function rhTable(list, cols, emptyMsg) {
  return `
    <div class="panel">
      <div class="panel-body ${list.length?'nopad':''}" style="${list.length?'ove
        ${list.length === 0 ? `<div class="text-mute" style="text-align:center;pa
          <table class="tbl">
            <thead><tr>${cols.map(c => `<th>${esc(c.label)}</th>`).join('')}<th><
            <tbody>
              ${list.map(e => `
                <tr>
                  ${cols.map(c => `<td>${c.val(e)}</td>`).join('')}

```

```

        <td><button class="btn-ghost" data-rh-view="\${e.id}">Voir →</bu
    </tr>
    \).join('')}
</tbody>
</table>
`}
</div>
</div>
`;
}

```

// — Small helper for key-value rows in the detail panels —

```

function kvRow(label, value) {
    // value peut contenir du HTML volontaire (chips) — les appelants échappent leu
    return `<div class="kv-row"><div class="kv-label">\${esc(label)}</div><div class
}

```

// ————— EMPLOYEE MOBILE SHELL —————

```

function viewEmpShell() {
    return `
        <div class="emp-shell">
            <div class="emp-topbar">
                <div class="greet">
                    <div class="gtitle">\${esc(state.user.prenom)}</div>
                    <div class="gsub">\${new Date().toLocaleDateString('fr-FR', { weekday: '
                </div>
                <button class="btn-icon" data-logout>
                    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="cur
                </button>
            </div>

            <div class="emp-content fade-in" id="empBody">
                \${renderEmpPage()}
            </div>

            <nav class="emp-tabbar">
                <button class="emp-tab \${state.page==='home'? 'active':''}" data-page="hom
                    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
                    Pointer
                </button>
                <button class="emp-tab \${state.page==='myweek'? 'active':''}" data-page="m
                    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
                    Planning
                </button>
                <button class="emp-tab \${state.page==='myreqs'? 'active':''}" data-page="m
                    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
                    Absences
                </button>
            </nav>
        `
    }
}

```

```

        <button class="emp-tab ${state.page==='myhours'?'active':''}" data-page="
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
                Heures
            </button>
        <button class="emp-tab ${state.page==='profile'?'active':''}" data-page="
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
                Profil
            </button>
        </nav>
    </div>`;
}

```

```

function renderEmpPage() {
    switch (state.page) {
        case 'home': return empHome();
        case 'myweek': return empWeek();
        case 'myreqs': return empMyRequests();
        case 'myhours': return empHours();
        case 'profile': return empProfile();
        default: return empHome();
    }
}

```

```

function bindEmp() {
    $$('[data-page]').forEach(b => b.addEventListener('click', e => {
        state.page = e.currentTarget.dataset.page;
        render();
    }));
    $$('[data-logout]').forEach(b => b.addEventListener('click', logout));

    switch (state.page) {
        case 'home': bindEmpHome(); break;
        case 'myweek': bindEmpWeek(); break;
        case 'myreqs': bindEmpMyRequests(); break;
        case 'myhours': break;
        case 'profile': break;
    }
}

```

```

function bindEmpWeek() {
    $$('[data-week]').forEach(b => b.addEventListener('click', e => {
        const a = e.currentTarget.dataset.week;
        if (a === 'prev') state.weekStart = addDays(state.weekStart, -7);
        else if (a === 'next') state.weekStart = addDays(state.weekStart, 7);
        else state.weekStart = getMonday(new Date());
        render();
    }));
}

```

```

// Émargement prévisionnel (planning)
const emargerBtn = $('#emargerBtn');
if (emargerBtn) {
  emargerBtn.addEventListener('click', () => {
    const empId = state.user.empId;
    const wk = weekKey(state.weekStart);
    const sigKey = `${empId}_${wk}`;
    if (!confirm('Tu confirmes avoir pris connaissance de ton planning pour cet
const sig = {
  signedAt: new Date().toISOString(),
  userAgent: navigator.userAgent.slice(0, 200),
  empId, weekKey: wk,
  planningHash: weeklyPlanningHash(empId, wk),
  type: 'previsionnel',
});
state.emargements = state.emargements || {};
state.emargements[sigKey] = sig;
if (db) db.ref(`emargements/${sigKey}`).set(sig);
toast('Planning émargé. Merci !', 'good');
render();
});
}

// Émargement effectif (heures réellement faites - fin de semaine)
const effBtn = $('#emargerEffectifBtn');
if (effBtn) {
  effBtn.addEventListener('click', () => {
    const empId = state.user.empId;
    const wk = weekKey(state.weekStart);
    const effKey = `${empId}_${wk}_effectif`;
    // Calcule les heures effectivement faites selon le planning
    let totalH = 0;
    for (let d = 0; d < 7; d++) {
      const shifts = (state.shifts[`${empId}_${d}_${wk}`] || []).map(normShift)
      shifts.forEach(s => totalH += shiftHours(s));
    }
    if (!confirm(`Tu confirmes avoir effectivement travaillé ${totalH.toFixed(1)
const sig = {
  signedAt: new Date().toISOString(),
  userAgent: navigator.userAgent.slice(0, 200),
  empId, weekKey: wk,
  planningHash: weeklyPlanningHash(empId, wk),
  totalHeures: totalH,
  type: 'effectif',
});
state.emargements = state.emargements || {};

```

```

    state.emargements[effKey] = sig;
    if (db) db.ref(`emargements/${effKey}`).set(sig);
    toast('Heures confirmées. Merci !', 'good');
    render();
  });
}
}

// ----- EMPLOYEE - MY REQUESTS -----
function empMyRequests() {
  const empId = state.user.empId;
  const reqs = Object.entries(state.absenceRequests||{})
    .map(([id,r]) => ({...r, id}))
    .filter(r => r.empId === empId)
    .sort((a,b) => (b.createdAt||'').localeCompare(a.createdAt||''));

  return `
    <div class="row" style="justify-content:space-between;margin-bottom:14px;">
      <div>
        <div class="uppercase-eyebrow">Mes absences</div>
        <div class="serif" style="font-size:24px;letter-spacing:-0.02em;">Demande
      </div>
    </div>

    <button class="punch-btn" id="newReq" style="margin-bottom:16px;">+ Nouvelle

    ${reqs.length === 0 ? `
      <div class="panel" style="background:#fafafa;border-style:dashed;">
        <div class="panel-body" style="text-align:center;padding:24px 18px;">
          <div style="font-size:32px;margin-bottom:8px;">🌴</div>
          <div class="text-mute" style="font-size:13px;">Aucune demande pour le m
        </div>
      </div>
    ` : reqs.map(r => {
      const lt = LEAVE_TYPES[r.type];
      return `
        <div class="panel" style="margin-bottom:10px;">
          <div class="panel-body" style="padding:14px 16px;">
            <div class="row" style="justify-content:space-between;margin-bottom:6px">
              <div style="font-weight:500;">${esc(lt?.label||r.type)}</div>
              ${reqStatusChip(r.status)}
            </div>
            <div class="text-mute mono" style="font-size:12px;">
              ${fmtDateShort(r.dateStart)} → ${fmtDateShort(r.dateEnd)} · ${r.nbJ}
            </div>
            ${r.motif ? `<div style="margin-top:6px;font-size:12.5px;color:var(--`
              : `<div style="margin-top:6px;font-size:12px;color:v`

```

```

        </div>
    </div>
    `;
    }).join('')
    `;
}

function bindEmpMyRequests() {
    $('#newReq').addEventListener('click', () => openEmpAbsenceRequest());
}

function openEmpAbsenceRequest() {
    const empId = state.user.empId;
    const body = `
        <div class="form-grid">
            <div class="field full"><label class="field-label">Type d'absence</label>
                <select class="input" id="erType">
                    ${Object.entries(LEAVE_TYPES).map(([k,v]) => `<option value="${k}">${es
                </select>
            </div>
            <div class="field"><label class="field-label">Du</label><input class="input
            <div class="field"><label class="field-label">Au</label><input class="input
            <div class="field full"><label class="field-label">Motif (optionnel)</label>
            <div class="field full text-mute" style="font-size:11.5px;">Ta demande sera
        </div>
    `;
    const footer = `<div class="spacer"></div><button class="btn-sec" data-close>An
    const { close } = openModal({ title: 'Nouvelle demande d\'absence', body, foote
    $('#erSave').addEventListener('click', () => {
        const start = $('#erStart').value;
        const endVal = $('#erEnd').value;
        if (!start || !endVal) { toast('Renseigne les dates', 'error'); return; }
        if (endVal < start) { toast('Date de fin avant date de début', 'error'); retu
        const sD = new Date(start), eD = new Date(endVal);
        const nbJours = Math.round((eD - sD)/86400000) + 1;
        const id = 'r' + Date.now();
        const r = {
            empId, type: $('#erType').value,
            dateStart: start, dateEnd: endVal,
            motif: $('#erMotif').value.trim(),
            nbJours,
            status: 'pending', createdAt: new Date().toISOString(),
        };
        state.absenceRequests[id] = r;
        fbSave(`absenceRequests/${id}`, r);
        toast('Demande envoyée à la direction', 'good');
        close();
    }

```



```

    render();
  });
}

```

```

// ----- EMPLOYEE - HOME (PUNCH) -----
function empHome() {
  const empId = state.user.empId;
  const now = new Date();
  const iso = dateISO(now);
  const dayIdx = (now.getDay() + 6) % 7;
  const wk = weekKey(now);
  const wkPublished = weekIsPublished(wk);
  const today = wkPublished ? (state.shifts[`${empId}_${dayIdx}_${wk}`] || []).map(s => s) : [];
  const punches = state.punches[`${empId}_${iso}`] || [];
  const last = punches[punches.length-1];
  const ongoing = last && last.in && !last.out;

  const pub = state.publications[wk];
  const showPubBanner = pub && (Date.now() - new Date(pub.publishedAt).getTime() < 1000 * 60 * 60 * 24);

  const todayLeave = today.find(s => s.leaveType);

  let statusText = '';
  if (!wkPublished) statusText = `Planning de la semaine pas encore publié.`;
  else if (todayLeave) statusText = `Aujourd'hui : ${LEAVE_TYPES[todayLeave.leaveType]} ${todayLeave.leaveStart} - ${todayLeave.leaveEnd}`;
  else if (today.length === 0) statusText = `Aucun shift prévu aujourd'hui.`;
  else if (ongoing) statusText = `Pointé à ${last.in} - en service.`;
  else if (last && last.out) statusText = `Dernier pointage : ${last.in} → ${last.out}`;
  else statusText = `Shift prévu : ${today[0].start} → ${today[0].end}`;

  return `
    ${showPubBanner ? `
      <div class="pub-banner">
        <div style="flex:1;">
          <div style="font-weight:500;font-size:13px;">🔔 Nouveau planning publié
          <div style="font-size:11.5px;opacity:.85;margin-top:2px;">Publié ${new Date(pub.publishedAt).toLocaleDateString()}
        </div>
        <button class="btn-ghost" data-page="myweek" style="color:inherit;font-weight:bold;">My Week
      </div>
    ` : ''}

    <div class="punch-card">
      <div class="punch-time" id="punchClock">${pad(now.getHours())}:${pad(now.getMinutes())}
      <div class="punch-date">${now.toLocaleDateString('fr-FR', { weekday: 'long' })}
      <div class="punch-status">${statusText}</div>
    </div>
  `;
}

```

```

    ${todayLeave ? '' :
      (ongoing
        ? `<button class="punch-btn" id="btnPunchOut">Pointer la sortie</button
        : (today.length > 0 || punches.length > 0)
        ? `<button class="punch-btn" id="btnPunchIn">Pointer l'entrée</button
        : `<button class="punch-btn" id="btnPunchIn">Pointer l'arrivée (hors
      )
    }
    ${punches.length ? `<button class="punch-btn secondary" id="btnSignal">Sign
  </div>

  ${today.length && !todayLeave ? `
    <div class="panel">
      <div class="panel-head"><h3>Shift d'aujourd'hui</h3></div>
      <div class="panel-body">
        ${today.map(s => `
          <div class="row" style="justify-content:space-between;">
            <div>
              <div style="font-family:var(--f-mono);font-size:16px;">${s.start}
              <div class="text-mute" style="font-size:12px;margin-top:2px;">${e
            </div>
            <div class="mono tabular" style="font-size:13px;color:var(--c-ink-4
          </div>`).join('')}
        </div>
      </div>` : ''}

    ${punches.length ? `
      <div class="panel" style="margin-top:12px;">
        <div class="panel-head"><h3>Pointages d'aujourd'hui</h3></div>
        <div class="panel-body" style="padding:0;">
          ${punches.map((p,i) => `
            <div class="row" style="padding:12px 16px;border-bottom:1px solid var
              <div class="mono">${p.in || '-'} → ${p.out || '<span class="text-di
                ${p.meta ? `<div class="text-mute" style="font-size:11px;">${esc(p.
              </div>`).join('')}
            </div>
          </div>` : ''}

        </div>` : ''}
    `;
  }

function bindEmpHome() {
  // live clock
  const tick = () => {
    const el = $('#punchClock');
    if (!el) return;
    const n = new Date();
    el.textContent = `${pad(n.getHours())}:${pad(n.getMinutes())}`;
  };

```

```

const interval = setInterval(tick, 30000);
setTimeout(() => clearInterval(interval), 600000); // stop after 10min idle

const inBtn = $('#btnPunchIn');
const outBtn = $('#btnPunchOut');
const sigBtn = $('#btnSignal');

if (inBtn) inBtn.addEventListener('click', () => doPunch('in'));
if (outBtn) outBtn.addEventListener('click', () => doPunch('out'));
if (sigBtn) sigBtn.addEventListener('click', openSignalModal);
}

// Coordonnées du restaurant Man'ouché (à ajuster). Si null, pas de check géo.
const RESTO_GPS = {
  lat: 48.8566, // Paris fallback - à remplacer par les coords précises du resto
  lng: 2.3522,
  radius: 150, // mètres de tolérance
  active: false, // mettre true pour activer le check géo
};

function haversineDistance(lat1, lng1, lat2, lng2) {
  const R = 6371000;
  const toRad = x => x * Math.PI / 180;
  const dLat = toRad(lat2 - lat1);
  const dLng = toRad(lng2 - lng1);
  const a = Math.sin(dLat/2)**2 + Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
  return 2 * R * Math.asin(Math.sqrt(a));
}

async function getGeoCheck() {
  if (!RESTO_GPS.active || !navigator.geolocation) return { ok: true, skip: true }
  return new Promise((resolve) => {
    const timeout = setTimeout(() => resolve({ ok: false, error: 'timeout' })), 80
    navigator.geolocation.getCurrentPosition(
      pos => {
        clearTimeout(timeout);
        const d = haversineDistance(pos.coords.latitude, pos.coords.longitude, RE
        resolve({
          ok: d <= RESTO_GPS.radius,
          distance: Math.round(d),
          lat: pos.coords.latitude,
          lng: pos.coords.longitude,
        });
      },
      err => {
        clearTimeout(timeout);
        resolve({ ok: false, error: err.message });
      }
    );
  });
}

```

```

    },
    { enableHighAccuracy: true, timeout: 7000, maximumAge: 30000 }
  );
});
}

```

```

async function doPunch(kind) {
  const empId = state.user.empId;
  const now = new Date();
  const iso = dateISO(now);
  const hhmm = `${pad(now.getHours())}:${pad(now.getMinutes())}`;
  const key = `${empId}_${iso}`;
  const punches = state.punches[key] ? [...state.punches[key]] : [];

  // Géolocalisation check
  let geo = null;
  if (RESTO_GPS.active) {
    toast('Vérification de la position...', '', 1500);
    const check = await getGeoCheck();
    if (!check.ok && !check.skip) {
      const msg = check.distance
        ? `Tu es à ${check.distance}m du restaurant (max ${RESTO_GPS.radius}m). P`
        : `Position non disponible. Active la géolocalisation et autorise l'app.`
      if (!confirm(msg + '\n\nForcer le pointage quand même ? (sera flaggé pour v`
      geo = { ...check, forced: true };
    } else {
      geo = check.skip ? null : check;
    }
  }

  if (kind === 'in') {
    const punch = { in: hhmm };
    if (geo) punch.geoIn = geo;
    punches.push(punch);
  } else {
    if (punches.length === 0 || punches[punches.length-1].out) {
      toast('Aucune entrée à clôturer', 'error');
      return;
    }
    punches[punches.length-1].out = hhmm;
    if (geo) punches[punches.length-1].geoOut = geo;
  }
  state.punches[key] = punches;
  fbSave(`punches/${key}`, punches);
  toast(kind === 'in' ? `Entrée enregistrée à ${hhmm}` : `Sortie enregistrée à ${
  render();
}

```

```

function openSignalModal() {
  const body = `
    <div class="form-grid">
      <div class="field"><label class="field-label">Entrée réelle</label><input c
    <div class="field"><label class="field-label">Sortie réelle</label><input c
    <div class="field full"><label class="field-label">Motif</label>
      <textarea class="input" id="sigReason" rows="3" placeholder="ex: pause ét
    </div>
    </div>
    <div class="text-mute" style="font-size:12px;margin-top:10px;">Ta demande ser
  `;
  const footer = `
    <div class="spacer"></div>
    <button class="btn-sec" data-close>Annuler</button>
    <button class="btn-pri" id="sigSend" style="width:auto;padding:10px 18px;">En
  `;
  const { close } = openModal({ title: 'Signaler une modification', body, footer
    $('#sigSend').addEventListener('click', () => {
      const empId = state.user.empId;
      const iso = dateISO(new Date());
      const key = `${empId}_${iso}`;
      const inT = $('#sigIn').value, outT = $('#sigOut').value, reason = $('#sigRea
      if (!inT && !outT) { toast('Renseigne au moins une heure', 'error'); return;
      const punches = state.punches[key] ? [...state.punches[key]] : [];
      punches.push({ in: inT, out: outT, meta: 'Signalé : ' + (reason || 'sans moti
      state.punches[key] = punches;
      fbSave(`punches/${key}`, punches);
      toast('Signalement envoyé', 'good');
      close();
      render();
    });
  }

  // _____ EMPLOYEE - MY WEEK _____
  function empWeek() {
    const empId = state.user.empId;
    const wk = weekKey(state.weekStart);
    const wkPublished = weekIsPublished(wk);
    const todayISO = dateISO(new Date());
    const wkEnd = addDays(state.weekStart, 6);

    let totalH = 0;
    const cards = !wkPublished ? '' : [0,1,2,3,4,5,6].map(i => {
      const d = addDays(state.weekStart, i);
      const iso = dateISO(d);
      const shifts = (state.shifts[`${empId}_${i}_${wk}`] || []).map(normShift);

```

```

const isToday = iso === todayISO;
const fer = holidayFor(d);
const hours = shifts.reduce((s, sh) => s + shiftHours(sh), 0);
totalH += hours;
const ferieTag = fer ? `<span class="ferie-pill">🇫🇷 ${esc(fer.short)}</span>`
if (!shifts.length) {
  return `<div class="day-card off ${isToday?'today':''} ${fer?'ferie':''}">
    <div><div class="day-num">${d.getDate()}</div><div class="day-name">${DAY}
    <div class="day-info"><div class="day-shift">Repos</div>${ferieTag}</div>
    </div>`;
}
return shifts.map(s => {
  if (s.leaveType) {
    const lt = LEAVE_TYPES[s.leaveType];
    return `<div class="day-card ${isToday?'today':''} ${fer?'ferie':''}" sty
      <div><div class="day-num">${d.getDate()}</div><div class="day-name">${D
      <div class="day-info">
        <div class="day-shift" style="font-family:var(--f-body);">${esc(lt?.l
        <div class="day-meta">Non travaillé</div>
        ${ferieTag}
      </div>
    </div>`;
  }
  return `
<div class="day-card ${isToday?'today':''} ${fer?'ferie':''}">
  <div><div class="day-num">${d.getDate()}</div><div class="day-name">${DAY}
  <div class="day-info">
    <div class="day-shift">${s.start} → ${s.end}</div>
    <div class="day-meta">${esc(s.label || (s.type==='midi'? 'Service midi':
    ${ferieTag}
  </div>
</div>`;
}).join('');
}).join('');

return `
<div class="row" style="justify-content:space-between;margin-bottom:14px;">
  <div>
    <div class="uppercase-eyebrow">Ma semaine</div>
    <div class="serif" style="font-size:24px;letter-spacing:-0.02em;">${fmtRa
  </div>
  <div class="text-mute mono tabular" style="font-size:13px;">${totalH.toFixe
</div>

<div class="row" style="margin-bottom:14px;gap:6px;">
  <button class="btn-sec" data-week="prev" style="padding:7px 12px;">←</butto
  <button class="btn-sec" data-week="today" style="padding:7px 14px;flex:1;">

```

```

    <button class="btn-sec" data-week="next" style="padding:7px 12px;"></button>
  </div>

```

```

    ${wkPublished ? cards : `
      <div class="panel" style="background:#fafafa;border-style:dashed;">
        <div class="panel-body" style="text-align:center;padding:32px 18px;">
          <div style="font-size:32px;margin-bottom:8px;"><img alt="calendar icon" data-bbox="665 162 682 178"/></div>
          <div style="font-weight:500;margin-bottom:4px;">Planning non publié</div>
          <div class="text-mute" style="font-size:12.5px;">Le planning de cette s
        </div>
      </div>
    `}

```

```

    ${wkPublished && totalH > 0 ? (() => {
      const prev = emargementStatus(empId, wk);
      const eff = emargementEffectifStatus(empId, wk);

      // Carte 1 : émargement prévisionnel (planning)
      let prevCard = '';
      if (prev.state === 'signed') {
        prevCard = `
          <div class="emargement-card signed">
            <div style="font-size:22px;line-height:1;">✓</div>
            <div style="flex:1;">
              <div style="font-weight:600;font-size:13.5px;">Planning prévisionne
              <div class="text-mute" style="font-size:12px;">Signé le ${new Date(
            </div>
          </div>
        `;
      } else if (prev.state === 'invalidated') {
        prevCard = `
          <div class="emargement-card invalidated">
            <div style="font-size:22px;line-height:1;">⚠</div>
            <div style="flex:1;">
              <div style="font-weight:600;font-size:14px;margin-bottom:4px;">Plan
              <div class="text-mute" style="font-size:12.5px;line-height:1.45;">T
            </div>
            <button class="btn-pri" id="emargerBtn" style="width:auto;flex-shrink
          </div>
        `;
      } else {
        prevCard = `
          <div class="emargement-card unsigned">
            <div style="flex:1;">
              <div style="font-weight:600;font-size:14px;margin-bottom:4px;">Plan
              <div class="text-mute" style="font-size:12.5px;line-height:1.45;">E
            </div>

```

```

        <button class="btn-pri" id="emargerBtn" style="width:auto;flex-shrink
    </div>
    `;
}

// Carte 2 : émargement effectif (heures réellement faites – fin de semaine
let effCard = '';
if (eff.state === 'too_early') {
    effCard = `
        <div class="emargement-card future">
            <div style="font-size:22px;line-height:1;">🕒</div>
            <div style="flex:1;">
                <div style="font-weight:600;font-size:13.5px;">Émargement des heure
                <div class="text-mute" style="font-size:12px;">Disponible à partir
            </div>
        </div>
    `;
} else if (eff.state === 'signed') {
    effCard = `
        <div class="emargement-card signed">
            <div style="font-size:22px;line-height:1;">✓✓</div>
            <div style="flex:1;">
                <div style="font-weight:600;font-size:13.5px;">Heures effectuées –
                <div class="text-mute" style="font-size:12px;">${eff.sig.totalHeure
            </div>
        </div>
    `;
} else {
    effCard = `
        <div class="emargement-card unsigned-effectif">
            <div style="flex:1;">
                <div style="font-weight:600;font-size:14px;margin-bottom:4px;">🇫🇷 C
                <div class="text-mute" style="font-size:12.5px;line-height:1.45;">L
            </div>
            <button class="btn-pri" id="emargerEffectifBtn" style="width:auto;fle
        </div>
    `;
}

return prevCard + effCard;
}) () : ''}
`;
}

// ————— EMPLOYEE — MY HOURS —————
function empHours() {
    const empId = state.user.empId;

```



```

const emp = state.employees.find(e => e.id === empId);
const now = new Date();
const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
const monthEnd = new Date(now.getFullYear(), now.getMonth()+1, 0);

let totalReal = 0, totalPlanned = 0, daysWorked = 0;
for (let d = new Date(monthStart); d <= monthEnd; d = addDays(d, 1)) {
  const iso = dateISO(d);
  const dayIdx = (d.getDay() + 6) % 7;
  const wk = weekKey(d);
  const shifts = state.shifts[`${empId}_${dayIdx}_${wk}`] || [];
  const punches = state.punches[`${empId}_${iso}`] || [];
  shifts.forEach(s => totalPlanned += shiftHours(s));
  let dayReal = 0;
  punches.forEach(p => {
    if (p.in && p.out) {
      let dur = timeToMin(p.out) - timeToMin(p.in);
      if (dur < 0) dur += 24*60;
      dayReal += dur / 60;
    }
  });
  if (dayReal > 0) daysWorked++;
  totalReal += dayReal;
}

const monthLabel = now.toLocaleDateString('fr-FR', { month: 'long', year: 'nume
const expected = (emp.heures || 35) * 4.33;
const pct = Math.min(100, Math.round((totalReal / expected) * 100));

return `
  <div class="row" style="justify-content:space-between;margin-bottom:14px;">
    <div>
      <div class="uppercase-eyebrow">Mes heures</div>
      <div class="serif" style="font-size:24px;letter-spacing:-0.02em;text-tran
    </div>
  </div>

  <div class="punch-card" style="text-align:left;">
    <div class="uppercase-eyebrow" style="color:rgba(255,255,255,.55);margin-bo
    <div class="punch-time">${totalReal.toFixed(1)}<span style="font-size:24px;
    <div class="punch-status" style="margin-top:8px;">
      ${daysWorked} jour${daysWorked>1?'s':''} travaillé${daysWorked>1?'s':''}
    </div>
    <div style="height:6px;background:rgba(255,255,255,.1);border-radius:10px;o
      <div style="width:${pct}%;height:100%;background:#fff;transition:width .4
    </div>
  </div>

```

```

<div class="panel">
  <div class="panel-head"><h3>Comparaison</h3></div>
  <div class="panel-body" style="padding:0;">
    <div class="row" style="padding:14px 16px;border-bottom:1px solid var(--c
      <div><div class="text-mute" style="font-size:11px;letter-spacing:0.06em
      <div><div class="text-mute" style="font-size:11px;letter-spacing:0.06em
      <div><div class="text-mute" style="font-size:11px;letter-spacing:0.06em
    </div>
  </div>
</div>
`;
}

// _____ EMPLOYEE - PROFILE _____
function empProfile() {
  const emp = state.employees.find(e => e.id === state.user.empId);
  if (!emp) return `<div class="empty"><h4>Profil indisponible</h4></div>`;
  return `
    <div class="row" style="margin-bottom:18px;gap:14px;">
      <div class="av-emp lg">${initials(emp)}</div>
      <div>
        <div class="serif" style="font-size:24px;letter-spacing:-0.02em;line-heig
          <div class="text-mute" style="font-size:13px;margin-top:2px;">${esc(emp.p
          </div>
        </div>

    <div class="panel">
      <div class="panel-body" style="padding:0;">
        ${profileRow('Contrat', esc(emp.contrat))}
        ${profileRow('Heures hebdo', `<span class="mono">${emp.heures} h</span>`)
        ${profileRow('Taux horaire', `<span class="mono">${emp.taux.toFixed(2)} €
        ${profileRow('Congés acquis', `<span class="mono">${emp.cpAcquis || 0} jo
        ${profileRow('Congés pris', `<span class="mono">${emp.cpPris || 0} jours<
        ${emp.email ? profileRow('Email', `<span class="mono" style="font-size:12
      </div>
    </div>

    <button class="btn-sec" data-logout style="width:100%;margin-top:16px;padding
  `;
}

function profileRow(label, value) {
  return `<div class="row" style="padding:14px 16px;border-bottom:1px solid var(-
    <div class="text-mute" style="font-size:12px;letter-spacing:0.04em;text-trans
    <div>${value}</div>
  </div>`;
}

```

```

// Week nav for employee
document.addEventListener('click', e => {
  const t = e.target.closest('[data-week]');
  if (!t) return;
  if (state.user?.role !== 'emp') return;
  const a = t.dataset.week;
  if (a === 'prev') state.weekStart = addDays(state.weekStart, -7);
  else if (a === 'next') state.weekStart = addDays(state.weekStart, 7);
  else state.weekStart = getMonday(new Date());
  render();
});

// _____ BOOT _____
function finishBoot() {
  if (!state.loading) return; // already booted
  state.loading = false;
  if (!state.employees.length) state.employees = DEFAULT_EMPLOYEES;
  if (state.user?.role === 'emp' && !state.employees.find(e => e.id === state.user.id))
    state.employees = DEFAULT_EMPLOYEES;
}
console.log('[boot] rendering', { fbReady: state.fbReady, employees: state.employees });
render();
}

(async function boot() {
  console.log('[boot] starting');
  // Restore session if any
  const saved = sessionStorage.getItem('mu_user');
  if (saved) {
    try { state.user = JSON.parse(saved); } catch {}
  }

  // Failsafe: always render within 2.5s regardless of Firebase status
  const failsafe = setTimeout(() => {
    console.warn('[boot] failsafe triggered - Firebase took too long');
    finishBoot();
  }, 2500);

  // Try Firebase with timeout
  try {
    const fbPromise = initFirebase();
    const timeoutPromise = new Promise(r => setTimeout(() => r(false), 4000));
    const ok = await Promise.race([fbPromise, timeoutPromise]);
    if (ok) {
      fbListen();
      try {

```

```
        await Promise.race([seedIfEmpty(), new Promise(r => setTimeout(r, 1500))]
    } catch (e) { console.warn('[boot] seed failed', e); }
    }
} catch (e) {
    console.error('[boot] firebase error', e);
}

clearTimeout(failsafe);
// Small delay so first listener snapshot can arrive
setTimeout(finishBoot, 300);
})();

// Service worker registration
if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
        navigator.serviceWorker.register('sw.js').catch(e => console.warn('SW failed'
    });
}
```